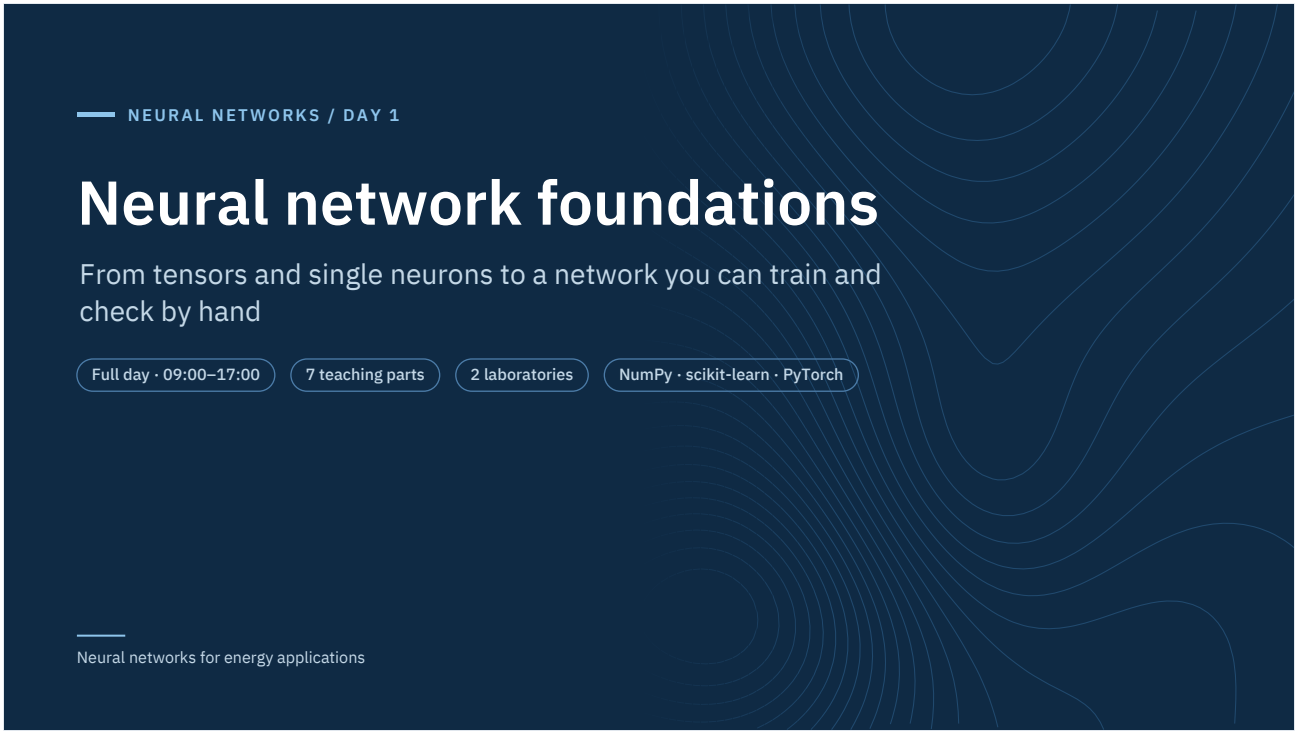
Neural network foundations

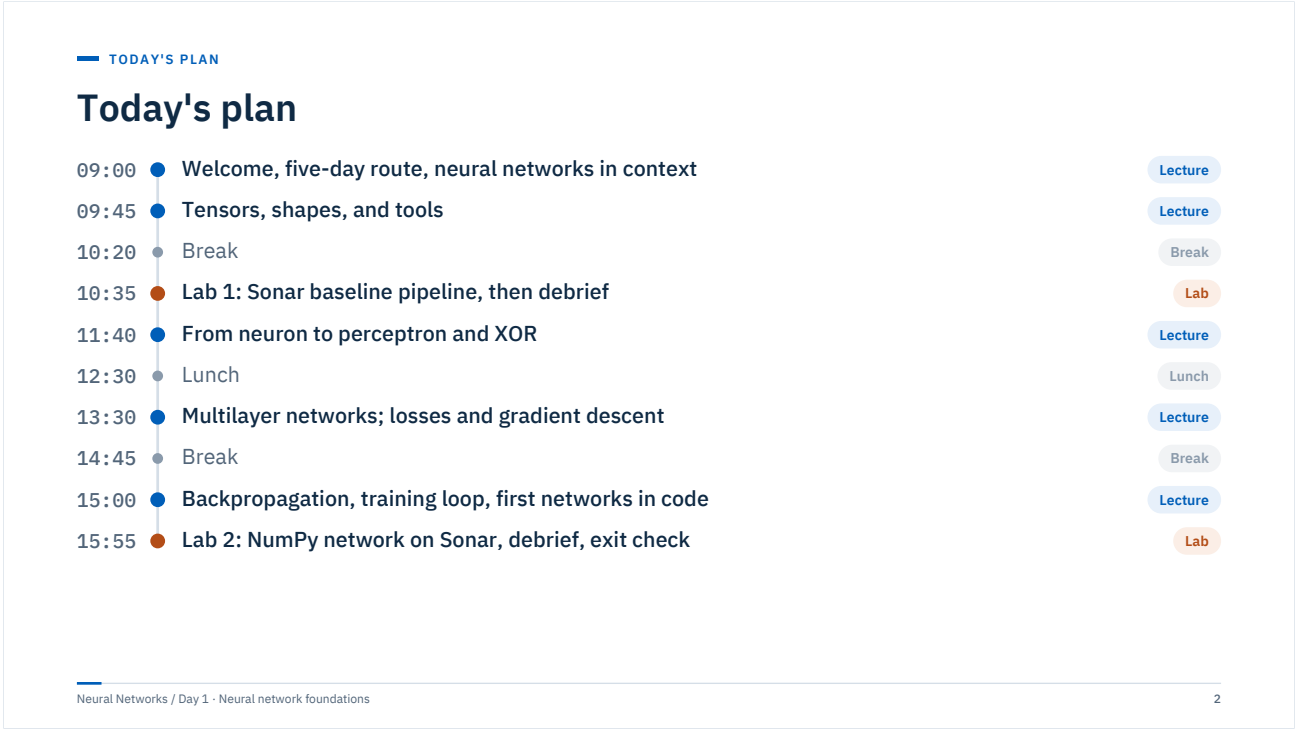


[2 min] Welcome the group and set expectations. Prerequisites: basic Python, NumPy arrays, algebra at the level of weighted sums and simple derivatives, and the idea of training, validation and test data.

This is the first of five days. Day 1 builds the machinery every later day relies on: tensors and shapes, the artificial neuron, multilayer networks, losses, gradient descent, backpropagation and the training loop. Two laboratories anchor the day: a scikit-learn baseline on the Sonar data this morning and a NumPy network written from scratch this afternoon.

The course places a dense-network capstone on Day 3, before CNNs (Day 4) and sequence models (Day 5). Day 3 is a midpoint application, not the final assessment of all architectures. Ask participants to open the exercises folder now so the environment is ready before Lab 1.

Today's plan

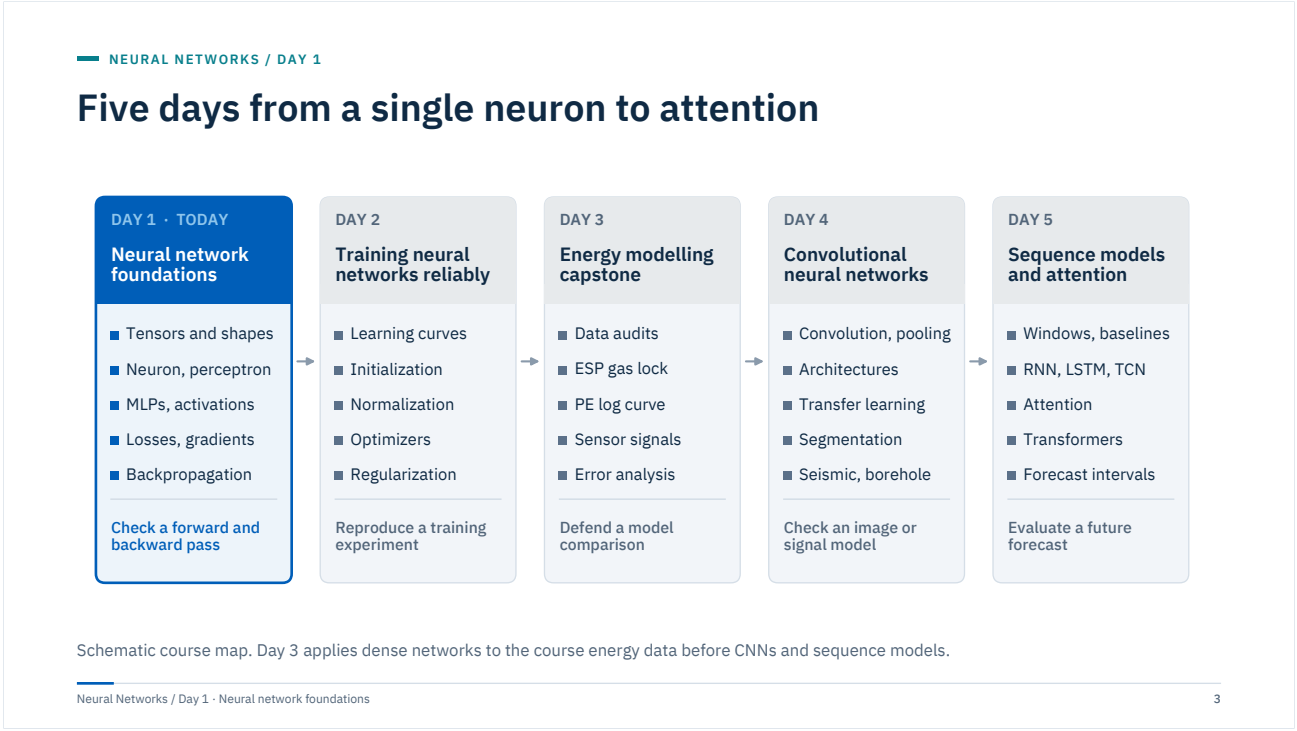


[2 min] Walk through the day and point out that the parts on the agenda are the seven section slides.

Opening and Part 1 (context) run 09:00–09:45; Part 2 (tensors and tools) 09:45–10:20. Lab 1 runs 10:35–11:25 with a 15-minute debrief to 11:40. Part 3 (neuron, perceptron, XOR) runs 11:40–12:30. After lunch, Part 4 (multilayer networks) runs 13:30–14:10 and Part 5 (losses and gradient descent) 14:10–14:45. Part 6 (backpropagation) runs 15:00–15:30 and Part 7 (training loop and first networks in code) 15:30–15:55. Lab 2 runs 15:55–16:40, its debrief 16:40–16:50, and the recap and exit check close the day at 17:00.

The in-lecture exercises are active time too: roughly one every 10–15 minutes. If a part runs long, shorten the history and hardware slides, never the exercises or the lab debriefs.

Five days from a single neuron to attention

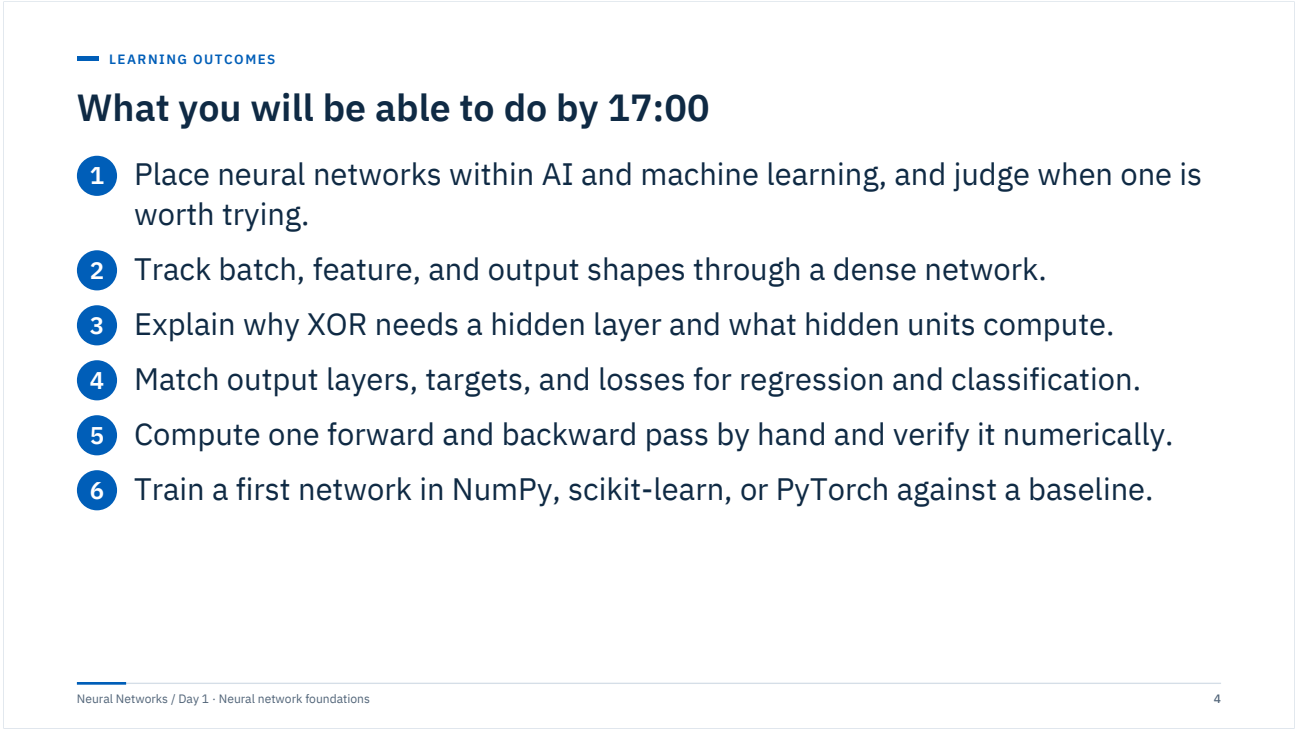


[3 min] The five-day route; each card carries the day's title. Today, Neural network foundations. Day 2, Training neural networks reliably: learning curves and diagnostics, initialization and normalization, a PyTorch training loop you can reproduce, optimizers and schedules, then regularization and autoencoders. Day 3, the Energy modelling capstone: three supplied projects (ESP gas lock, PE log-curve prediction and anonymous sensor signals), each a dense-network candidate compared with a baseline. Day 4, Convolutional neural networks, for images and 1-D signals. Day 5, Sequence models and attention: forecasting, recurrent networks, attention, transformers and forecast uncertainty.

Each day ends in a demonstrated outcome (bottom row of each card) rather than a list of topics covered. Some laboratories contain more material than one session can complete, and their lab slides mark the extensions (today, sections 12 and 13 of Lab 2). Finishing and debriefing one sound experiment is worth more than rushing every notebook cell.

Ask: which of the five outcomes is closest to something you need at work? Transition: today's outcome is concrete. By 17:00 you will have checked a forward and a backward pass by hand and in code.

What you will be able to do by 17:00

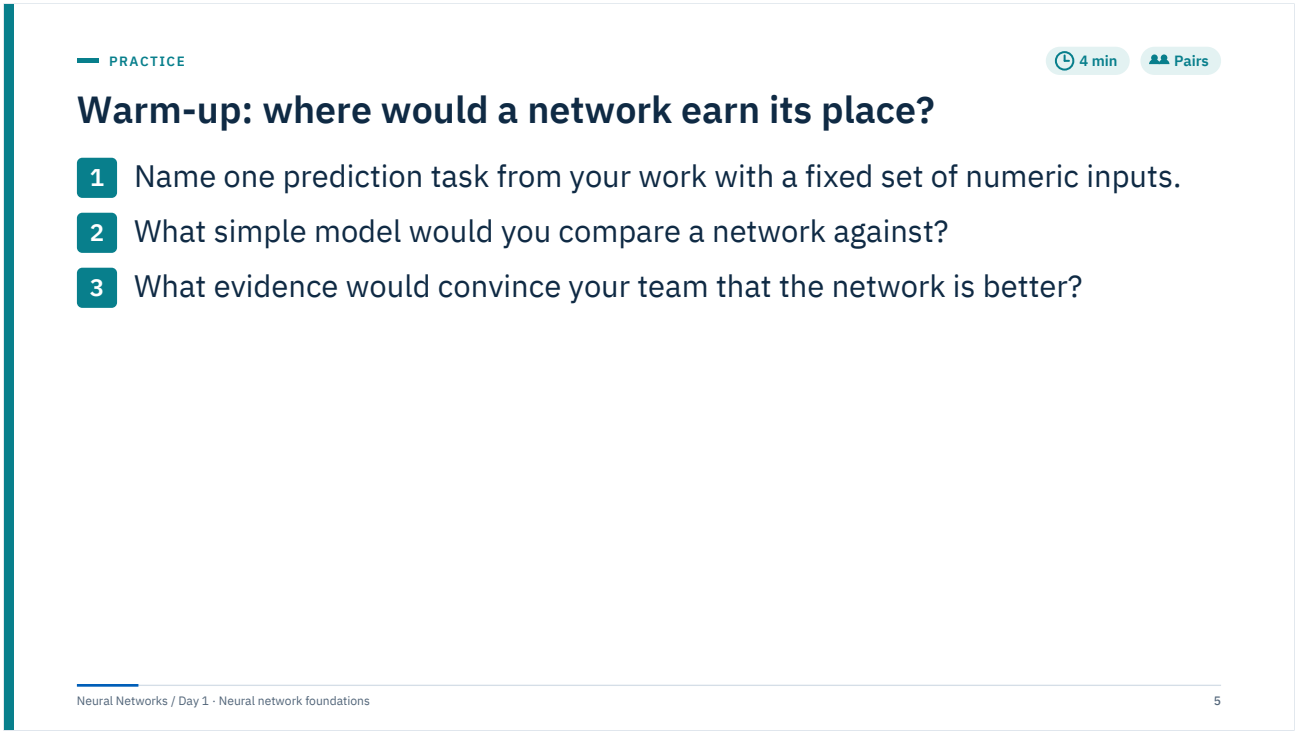


[2 min] Read the outcomes as observable skills: each one is something you can be asked to do, not something you have been told.

Assessment for the day: the exit check at 16:50 and the two lab deliverables, a baseline comparison on the Sonar task (Lab 1) and a NumPy network with shape assertions and a gradient check (Lab 2).

Outcome 5 matters most for the rest of the week. Every framework automates the forward and backward pass; knowing how to check them by hand is what lets you trust, and debug, the automation. Outcome 6 insists on "against a baseline": a network that has not beaten a simple model on the same split has not yet earned its complexity.

Warm-up: where would a network earn its place?

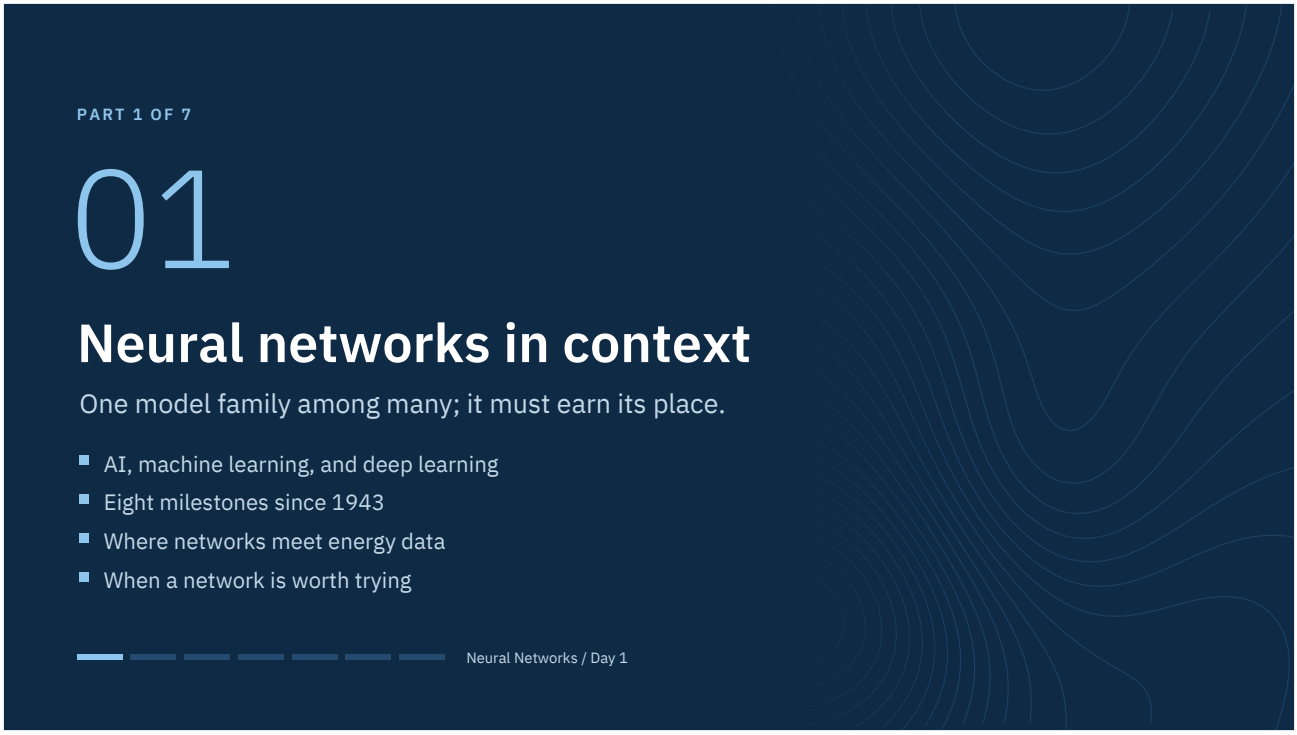


[4 min] Purpose: activate prior experience and plant the idea that a neural network is one candidate model that must win a fair comparison. Give pairs two minutes, then collect three or four answers on a flip chart and keep them visible; revisit them at the end of Part 1 and in the exit check.

Typical answers: predicting a missing log curve from other logs (compare with linear regression or gradient boosting); flagging ESP gas lock from pump sensor readings (compare with the thresholds engineers already use, or logistic regression); day-ahead wind power from weather forecasts (compare with persistence or a power-curve model).

Good evidence: the same held-out wells, assets or future period for every candidate; a metric tied to the decision; errors inspected by group. Weak evidence: one random split of correlated rows, or a training-set score. Transition: first, where do neural networks sit in the wider field?

Neural networks in context



[1 min] Part 1 runs until 09:45, about 30 minutes including one exercise. The goal is orientation, not history for its own sake: by the end of the part learners should be able to say what a neural network is, why it became practical, where it is used in energy work, and what evidence it must produce before anyone should trust it.

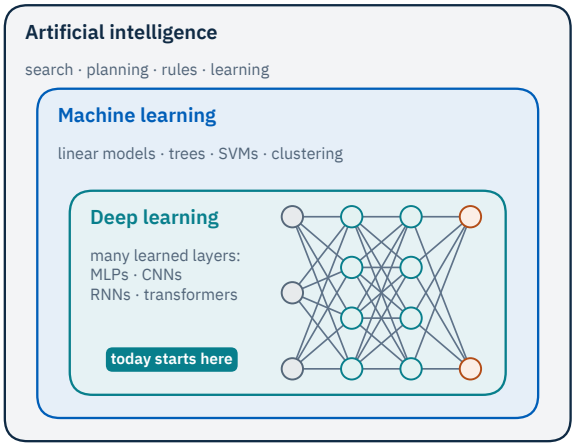
Keep the history slide brisk (four minutes at most). Spend the saved time on the closing exercise, which asks pairs to decide whether a network should be the first model for four energy tasks.

Deep learning is one branch of machine learning

NEURAL NETWORKS IN CONTEXT

Deep learning is one branch of machine learning

- AI is the broad field; not all of it learns from data.
- Machine learning fits a model's parameters to data.
- **Deep learning** composes many learned layers.
- The same networks serve classification, regression, and representation learning.



Schematic. The examples in each field are illustrative, not exhaustive.

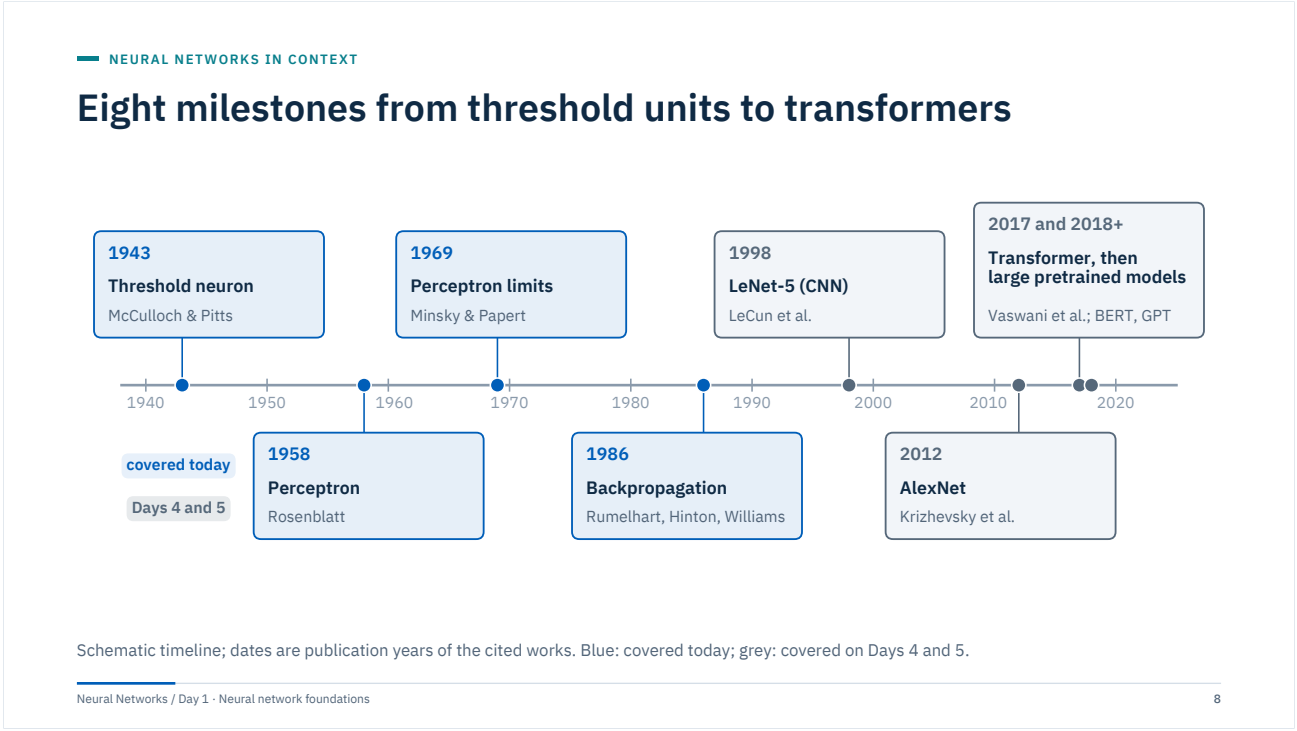
Neural Networks / Day 1 · Neural network foundations 7

[3 min] Artificial intelligence is the broad field of computational systems that act intelligently: search, planning and rule-based expert systems belong here even though nothing is fitted to data. Machine learning is the subset that fits parameters to data or experience. Deep learning uses compositions of many learned transformations; "deep" refers to the number of layers. A network with one hidden layer is a neural network but not usually called deep.

Avoid equating neural networks with human cognition. Biological neurons were historical inspiration (Part 3), but the unit we use is a mathematical function. Reinforcement learning and generative modelling are related settings, not prerequisites for today.

Ask: is a rule-based alarm on ESP intake pressure AI? It is AI in the broad historical sense if it encodes expert decisions, but it is not machine learning, because nothing is fitted to data.

Eight milestones from threshold units to transformers

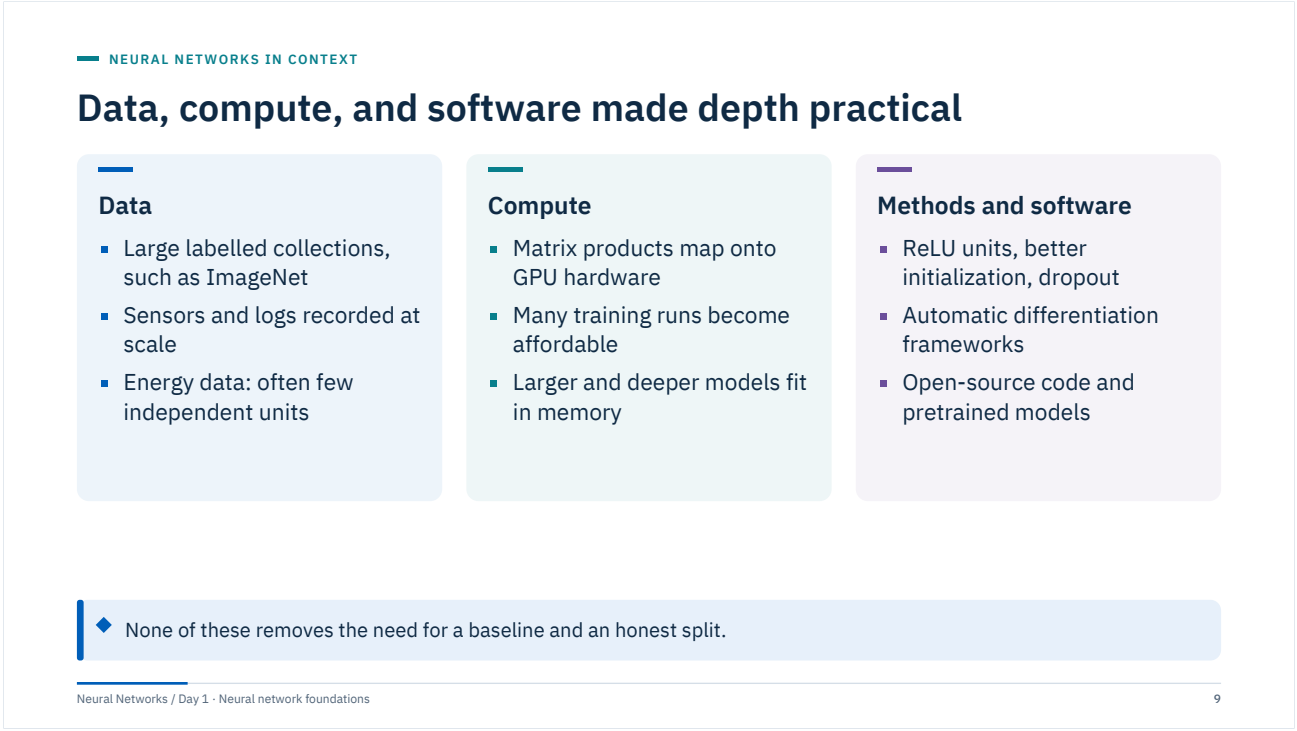


[4 min] One sentence each, left to right. 1943: McCulloch and Pitts describe a binary threshold neuron, logic from weighted sums. 1958: Rosenblatt's perceptron learns its weights from examples. 1969: Minsky and Papert's book Perceptrons analyses what single-layer perceptrons cannot compute, including parity problems such as XOR. 1986: Rumelhart, Hinton and Williams popularize backpropagation for training hidden layers; reverse-mode differentiation itself has earlier roots, for example Werbos's 1974 thesis.

1998: LeCun and colleagues publish LeNet-5, a convolutional network for document recognition. 2012: AlexNet, a GPU-trained deep CNN, wins the ImageNet challenge by a wide margin. 2017: the Transformer replaces recurrence with attention. From 2018: large models pretrained on broad data, such as BERT and GPT, are adapted to many tasks.

The quiet period after 1969 is often described as reduced interest and funding; do not attach precise dates to it. Today covers the blue boxes.

Data, compute, and software made depth practical

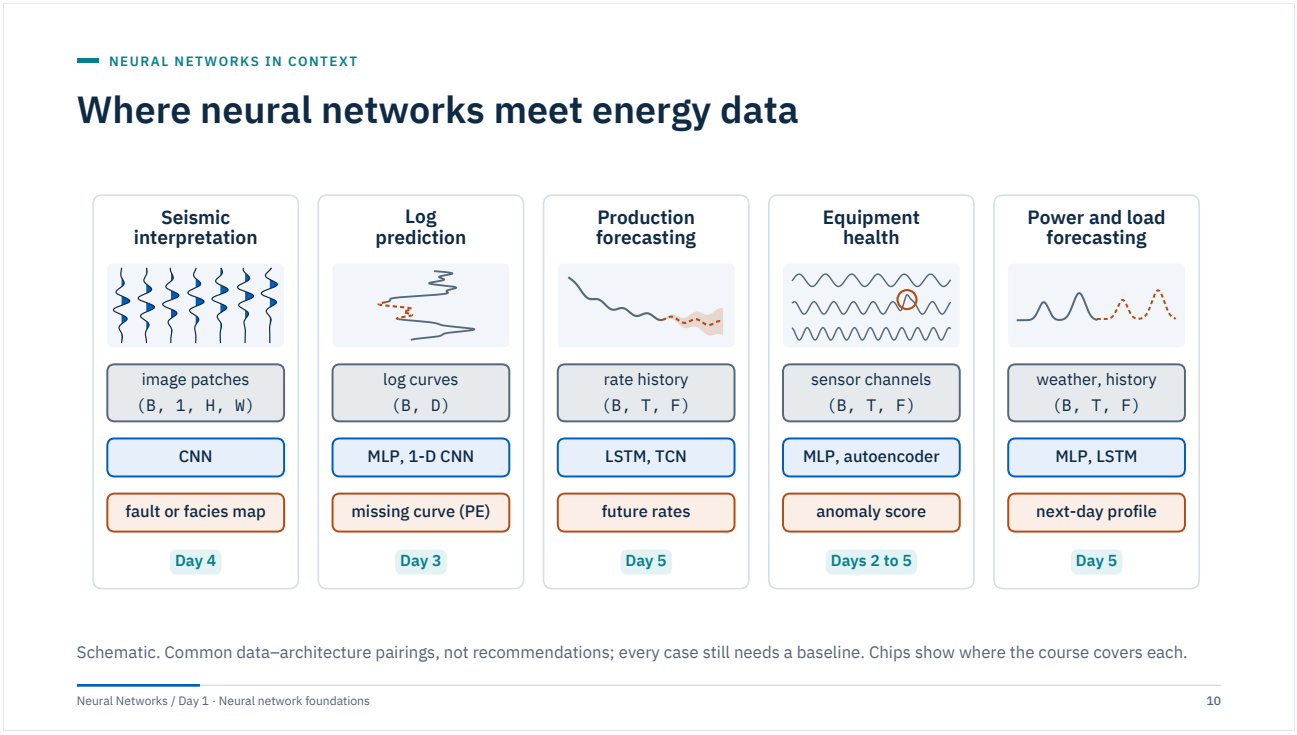


[3 min] Networks with hidden layers existed in the 1980s and 1990s. Why did they become dominant for images, speech and text after about 2010? Three enablers arrived together. Large labelled datasets gave flexible models enough examples. GPUs made the dense matrix arithmetic of training fast (Part 2 shows why). Methods such as ReLU activations, better weight initialization and dropout made deeper networks trainable, and frameworks with automatic differentiation removed the need to derive gradients by hand.

For energy data, the data card is the constraint that bites most. Many problems have few independent wells, pumps or events even when the row count is large. That is why Day 3 spends so much time on data contracts and audits.

Ask: in your data, how many independent units (wells, pumps, turbines) do you have, as opposed to rows?

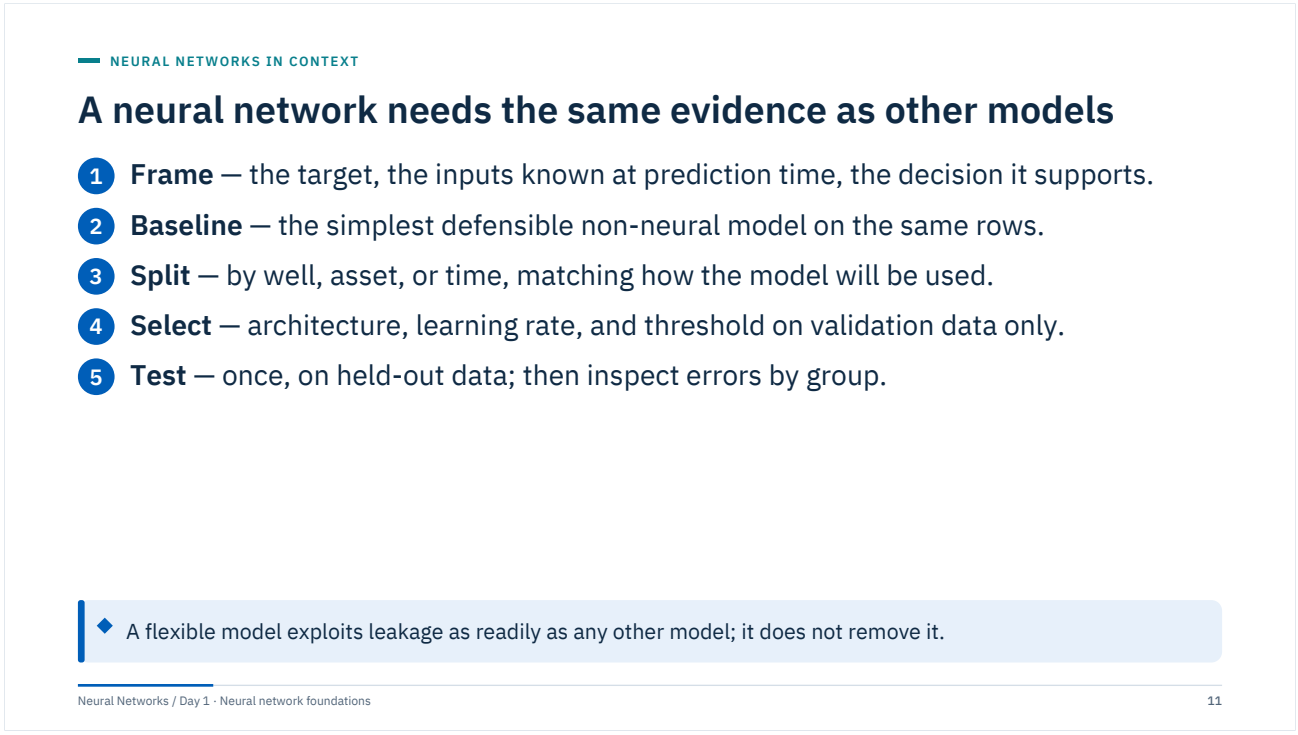
Where neural networks meet energy data



[4 min] Five application families, each with an input tensor shape (formalized in Part 2), a common architecture and an output. Seismic interpretation: image patches or volumes, CNNs, fault or facies maps (Day 4). Log prediction: depth samples of curves as a table, MLPs or 1-D CNNs, a missing curve such as PE (the Day 3 capstone). Production forecasting: rate and pressure histories, sequence models, future rates (Day 5). Equipment health: ESP, compressor or turbine sensor channels, MLPs or autoencoders, fault classes or anomaly scores (Days 2 to 5: autoencoders on Day 2, the ESP and sensor projects on Day 3, 1-D convolutions and reconstruction scores on Day 4, sequence anomalies on Day 5). Power and load forecasting: weather forecasts plus history, MLPs or recurrent networks such as LSTMs, next-day profiles (Day 5).

These pairings describe common practice, not guaranteed wins: a gradient-boosted model or a decline-curve fit may be the better tool. Ask participants to place their warm-up task in one of the columns.

A neural network needs the same evidence as other models

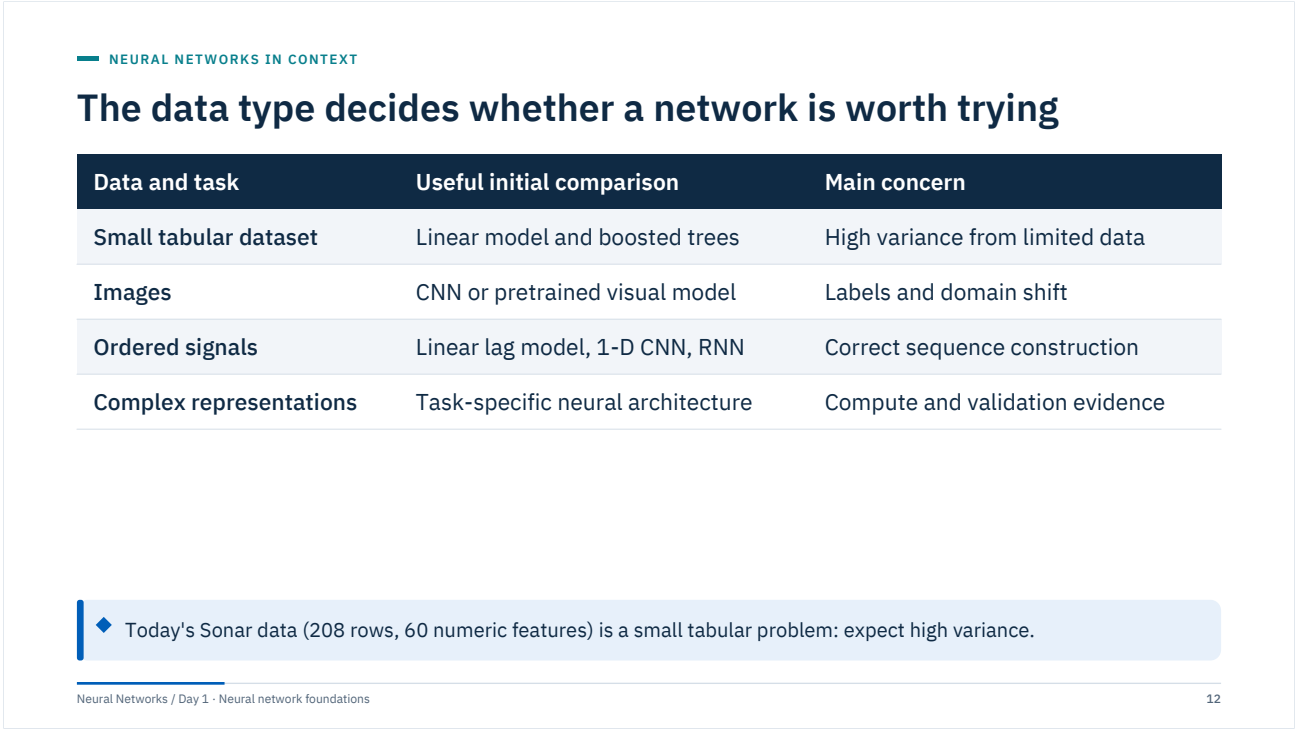


[3 min] The evaluation workflow does not change because the model is a network. Ask learners why a neural network does not remove leakage. Answer: leakage is a property of the data and the split, not of the model; a flexible model finds a leaked signal at least as easily as a simple one.

Use grouped splits for unseen wells and chronological splits for future observations. Multiple rows from one asset are not independent evidence of generalization. Select architectures, learning rates and thresholds on validation data, and evaluate the chosen model once on the final test set.

In Lab 1 you will see how much the split design alone can move a score on a small dataset, before any network is involved. Transition: so when is a network worth the extra effort?

The data type decides whether a network is worth trying



[3 min] There is no minimum sample count that makes a neural network universally appropriate. Transfer learning, task difficulty, noise, the number of independent groups and model capacity all change the data requirement. Use learning curves and held-out comparisons rather than rules of thumb.

For small tables, typical of well-level or asset-level energy data, boosted trees and regularized linear models are strong baselines, and a network must beat them on the same folds. Networks tend to pay off when the input has structure they can exploit (spatial for images, temporal for signals) or when a pretrained model exists.

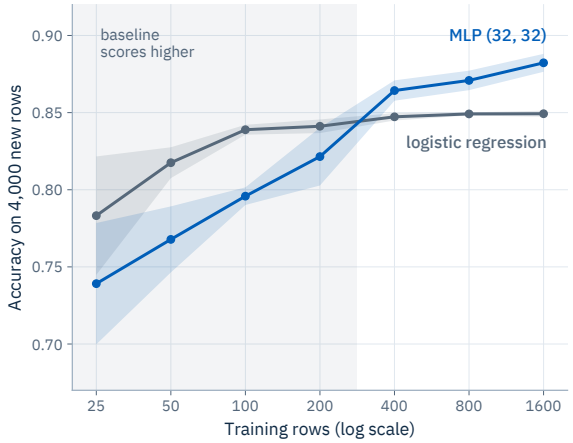
Ask: where does today's Sonar data (208 rows, 60 numeric features) sit in this table? Answer: small tabular dataset; high variance from limited data is the main concern, which is exactly what the Lab 1 debrief will show.

On small samples, a simple baseline can beat a network

NEURAL NETWORKS IN CONTEXT

On small samples, a simple baseline can beat a network

- Same synthetic task; only the number of training rows changes.
- Up to 200 rows, logistic regression scores higher.
- From 400 rows the MLP pulls ahead: 0.88 vs 0.85 at 1,600.
- In energy data, count independent wells or assets, not rows.



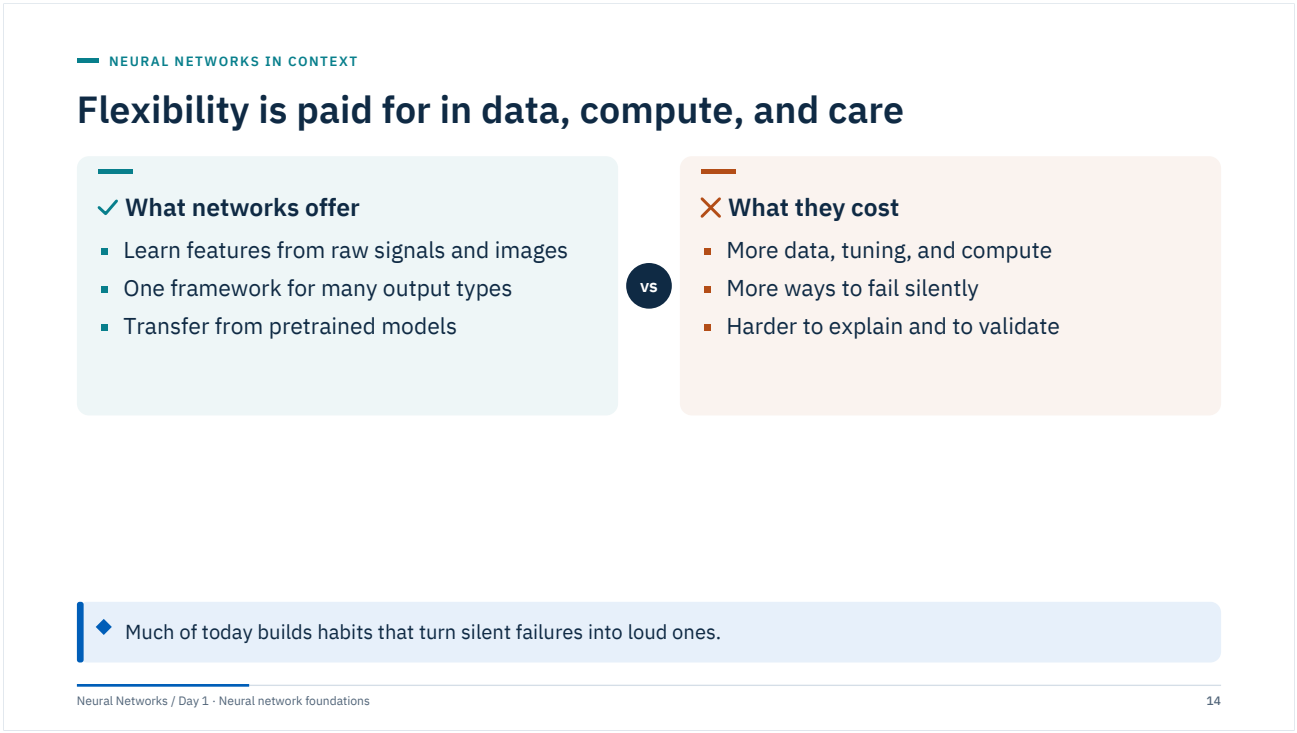
Computed: synthetic two moons plus 4 noise features; lines show the mean of 8 resamples (seeds 0–7), bands ±1 sd.

[3 min] A learning curve: the same task learned from more and more training rows, each model scored on 4,000 fresh rows. The data are synthetic: two interleaved half-moons (a curved boundary) plus four pure-noise features.

With 25 to 200 rows the logistic baseline wins (for example 0.839 against 0.796 at 100 rows): the network has more parameters than the data can pin down and fits some noise. From 50 rows on its band is also wider, so its results vary more from sample to sample. From 400 rows the network's flexibility pays off, and at 1,600 rows it reaches 0.882 while the straight-line baseline levels off at 0.849.

The crossover point belongs to this problem only; it is not a rule. It moves with noise, the number of features and how curved the true boundary is. The general lesson holds: small datasets favour simple models, and correlated rows (depth samples in one well, minutes from one pump) count for less than their number suggests. Ask: where would the 208-row Sonar dataset sit on this axis? Around the 200 mark, still inside the shaded region where the simple model wins on this task. Where Sonar's own crossover lies is unknown: it has 60 features, not 6, and a different boundary.

Flexibility is paid for in data, compute, and care



[2 min] Balance the enthusiasm. The left card explains why networks dominate images, text and many signal tasks: they learn useful features from raw inputs instead of relying on hand-crafted ones, one framework handles regression, classification and structured outputs, and pretrained models transfer knowledge from large datasets.

"More ways to fail silently" deserves an example: a target of shape (B,) against predictions of shape (B, 1) that broadcasts, a sigmoid applied twice, a learning rate that stalls, leakage through preprocessing. The code runs and prints a number. Much of today is about habits that turn silent failures into loud ones: shape contracts, gradient checks and baselines.

Transition to the exercise: apply this to four energy tasks.

Would you start with a neural network here?

PRACTICE

5 min Pairs

Would you start with a neural network here?

1. 300 wells, 12 log-derived features per depth sample → facies class.
2. Two years of 1-minute ESP sensor data → gas-lock warning an hour ahead.
3. 50,000 labelled seismic image patches → fault or no fault.
4. Daily wind-farm output and weather forecasts → next-day power.

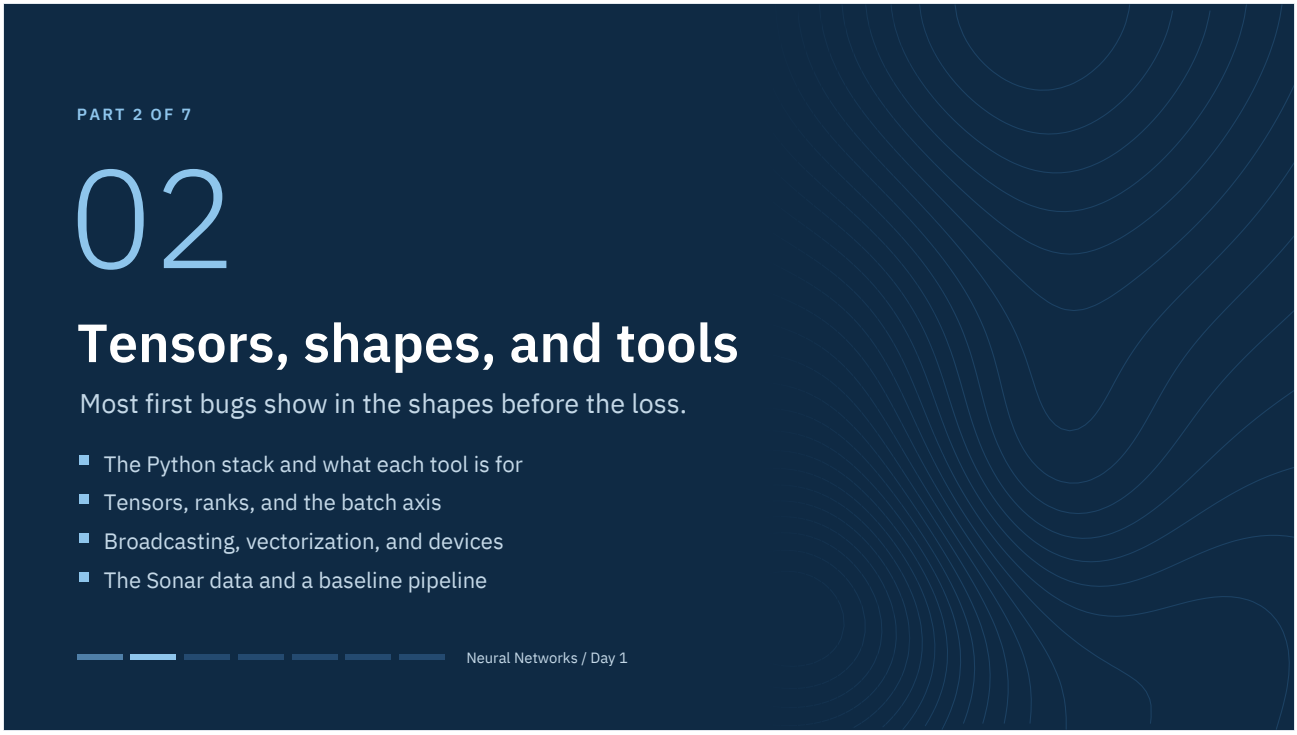
Neural Networks / Day 1 · Neural network foundations 15

[5 min] Ask pairs to label each task "network first", "baseline first, network maybe" or "not a network", and to name the split. Suggested answers (there is room for argument):

1. Facies from logs: baseline first (boosted trees or logistic regression on the 12 features) with a split grouped by well. An MLP is a reasonable candidate, but the 300 wells, not the number of depth samples, set the effective sample size. 2. ESP gas lock: baseline first (engineering thresholds, logistic regression on lagged features). A sequence model (Day 5) may help, but events are rare and correlated in time, so use a chronological split and event-level metrics. 3. Seismic patches: network first. Images with spatial structure are where CNNs (Day 4) earn their place; keep patches from the same survey area out of both training and test. 4. Wind power: baseline first (persistence and a power-curve model on forecast wind speed); a network may add value with many weather inputs. Split chronologically.

Close Part 1 at about 09:45.

Tensors, shapes, and tools



[1 min] Part 2 runs 09:45–10:20 and ends with the Lab 1 briefing. The message of the part: most first neural-network bugs are visible in array shapes and data types long before they are visible in the loss, so we make shapes explicit and check them.

PyTorch is the main teaching framework for code consistency; Day 2 provides a Keras translation so learners can recognize the same concepts in both frameworks. NumPy comes first because Lab 2 builds a network in plain NumPy, and because PyTorch tensors deliberately mirror NumPy arrays.

The Python stack has distinct responsibilities

TENSORS, SHAPES, AND TOOLS

The Python stack has distinct responsibilities

Tool	Responsibility	First check
NumPy	Arrays and vectorized calculations	Shape and dtype
pandas	Tables, labels, joins, and timestamps	Row alignment and schema
Matplotlib	Scientific plots	Axes, units, and sampling
scikit-learn	Baselines and preprocessing	Fitting inside training folds
PyTorch / Keras	Differentiable models and optimization	Output-loss compatibility

◆ Keep notebooks restartable: run top to bottom before trusting any number.

Neural Networks / Day 1 · Neural network foundations 17

[2 min] Each tool has one job and one characteristic first check. NumPy: print shape and dtype. pandas: confirm row alignment after joins and the schema (column names, types, units). Matplotlib: check axes, units and whether you are looking at a sample. scikit-learn: fit every preprocessing step inside the training folds. PyTorch or Keras: confirm that the output layer and the loss agree.

Keep notebooks restartable. Run from the exercise directory or use an explicit data root. Print a small sample and the shapes before training. Avoid notebook cells that depend on hidden execution history; "Restart and run all" before reporting any number.

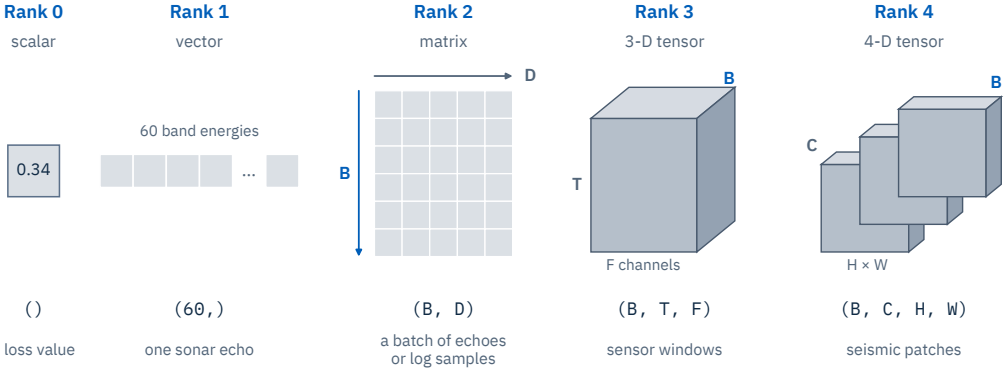
Ask: which of these tools have you used? Adjust the pace of the NumPy slides to the answer.

A tensor is an array with a fixed number of axes

TENSORS, SHAPES, AND TOOLS

A tensor is an array with a fixed number of axes

The **rank** is the number of axes; the shape lists the length of each axis.



Schematic. B = batch, D = features, T = time steps, F = channels, C = image channels, H × W = image size.

[3 min] "Tensor" in deep learning simply means an n-dimensional array. A scalar has rank 0 and shape (); a vector rank 1; a matrix rank 2; a batch of sequences rank 3; a batch of images rank 4.

By convention the batch axis B comes first in NumPy and PyTorch. PyTorch images are channels-first, (B, C, H, W); some other libraries use channels-last, (B, H, W, C), so always check. The training loss is a rank-0 tensor: the whole batch is reduced to one number before backpropagation.

Ask: what shape is one Sonar echo? (60,). A batch of 32 echoes? (32, 60). A batch of 16 seismic patches of 64 × 64 samples with one amplitude channel? (16, 1, 64, 64).

Energy data arrive in a few standard tensor shapes

TENSORS, SHAPES, AND TOOLS

Energy data arrive in a few standard tensor shapes

Data	Tensor shape	Axes
Log samples as a table	(B, D)	depth samples × curves
Sonar echoes (today)	(B, 60)	echoes × frequency bands
ESP sensor windows	(B, T, F)	windows × time steps × channels
Seismic patches	(B, 1, H, W)	patches × amplitude channel × height × width
Load or production history	(B, T, 1)	windows × time steps × one value

◆ First question for any dataset: what is one example, and what does B count?

Neural Networks / Day 1 · Neural network foundations 19

[3 min] The first design decision for any dataset is what counts as one example. For logs, one example may be one depth sample (and then neighbouring samples are strongly correlated) or one depth window. The choice changes both the tensor shape and the split: all samples from one well belong on the same side of the split, whatever the tensor shape.

For sensor data, one example is usually a window of T time steps with F channels; Day 5 builds these windows properly, including the gap between the window and the prediction target. For seismic, one example is a patch cut from a section or volume; Day 4 builds image tensors.

Ask: in the ESP case, if you cut overlapping windows every minute, are neighbouring windows independent? No: they share most of their time steps, so random splits would leak.

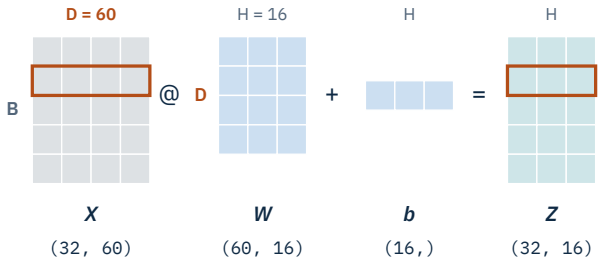
One batch, one shape contract

TENSORS, SHAPES, AND TOOLS

One batch, one shape contract

$$Z = XW + b$$

- Rows are examples: X is (B, D), Z is (B, H).
- Inner sizes must match: (B, D) @ (D, H).
- W never contains B, so the batch size is free.
- PyTorch `nn.Linear(D, H)` stores its weight as (H, D), the transpose of this W.



`(32, 60) @ (60, 16)` inner sizes must match

row i of Z uses only row i of X (outlined)

Schematic with B = 32, D = 60, H = 16. Row i of Z depends only on row i of X.

Neural Networks / Day 1 · Neural network foundations 20

[3 min] For batches we write each example as a row, and a dense layer computes Z = XW + b. X is (B, D) with B examples and D features; W is (D, H) for H hidden units; b has one entry per hidden unit and is added to every row; Z is (B, H). NumPy code, Lab 2 and most of today's shape checks use this row convention.

Part 4 also writes a single example as a column vector: z = Wx + b. There W has shape (H, D), one row per unit: it is the transpose of the (D, H) matrix used here. Stacking such examples as rows gives Z = X W^T + b, the same layer. PyTorch's nn.Linear(D, H) stores its weight in that (H, D) layout and, for a batch x of shape (B, D), computes x · weight^T + bias: the row form above with W = weight^T. State the convention before writing derivatives: most confusion in backpropagation code comes from mixing the two.

The batch size never appears in W, which is why the same trained network can predict one example or a million.

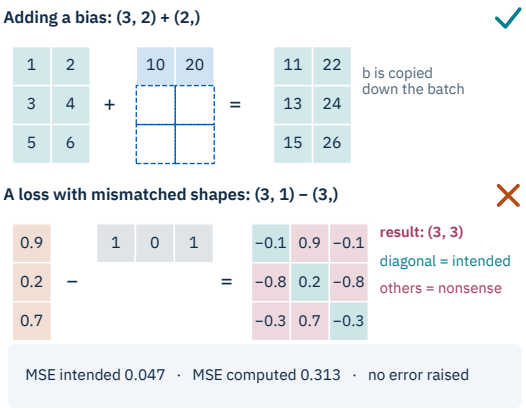
Ask: for H = 16 hidden units and 60 features, how many weights and biases? 60 × 16 = 960 weights plus 16 biases = 976 parameters.

Broadcasting stretches size-1 axes, sometimes silently

TENSORS, SHAPES, AND TOOLS

Broadcasting stretches size-1 axes, sometimes silently

- Shapes align from the last axis; missing or size-1 axes are copied.
- A bias of shape (H,) added to (B, H) is the intended use.
- (B, 1) minus (B,) silently becomes (B, B).
- Reshape targets to match predictions before any loss.



Exact NumPy arithmetic. The mismatched loss is 0.313 instead of 0.047, and no error is raised.

Neural Networks / Day 1 · Neural network foundations 21

[3 min] NumPy and PyTorch compare shapes from the trailing axis. Two sizes are compatible if they are equal or one of them is 1; a missing leading axis counts as size 1. So (3, 2) + (2,) treats b as (1, 2) and copies it down three rows: the intended bias.

The bottom example is the classic loss bug. Predictions of shape (3, 1) minus targets of shape (3,) treat the targets as (1, 3), so the result is (3, 3): every prediction minus every target. The diagonal holds the intended differences (−0.1, 0.2, −0.3); the rest is nonsense. The mean squared error comes out as 0.313 instead of 0.047 and nothing warns you.

PyTorch's BCEWithLogitsLoss raises an error for this mismatch, but MSELoss only warns and broadcasts. Ask: what is the shape of (32, 1) + (32,)? (32, 32).

Predict the shape before you run the cell

PRACTICE

4 min Individual

Predict the shape before you run the cell

X has shape (32, 60), W (60, 16), b (16,), y_pred (32, 1), and y (32,).

1. `X @ W + b`
2. `(X @ W).mean(axis=0)`
3. `y_pred - y`
4. `W @ X`

Neural Networks / Day 1 · Neural network foundations 22

[4 min] Give learners two minutes on paper, then reveal.

1. X @ W + b has shape (32, 16): the matrix product gives (32, 16) and b broadcasts across rows. 2. (X @ W).mean(axis=0) has shape (16,): averaging over axis 0 averages over the batch, leaving one value per hidden unit. 3. y_pred – y has shape (32, 32): the silent broadcast from the previous slide. The fix is y.reshape(-1, 1) or y_pred.ravel(). 4. W @ X raises a ValueError: the inner sizes are 16 and 32, which do not match. Order matters in matrix products.

Common errors: confusing axis=0 (down the rows, over examples) with axis=1 (across columns, over features); expecting item 3 to give (32,). Habit to build: print shapes in the first cell of every notebook and assert them where it matters.

Vectorized calculation replaces nested loops

TENSORS, SHAPES, AND TOOLS

Vectorized calculation replaces nested loops

```python
import numpy as np

X = np.array([[1., 2.], [3., 4.]])      # 2 examples, 2 features
W = np.array([[0.5, -1.], [1., 0.5]])   # 2 inputs -> 2 units
b = np.array([0., 1.])
Z = X @ W + b                           # (2, 2): one row per example
A = np.maximum(Z, 0.)                   # ReLU, elementwise
assert A.shape == (2, 2)
print(A)
```

OUTPUT

```
[[2.5 1. ]
 [5.5 0. ]]
```

One matrix product computes every unit for every example at once.

Neural Networks / Day 1 · Neural network foundations 23

[3 min] Ask learners to compute the first row before revealing the output. For example one, X W is (1·0.5 + 2·1, 1·(−1) + 2·0.5) = (2.5, 0); adding the bias (0, 1) gives Z = (2.5, 1); ReLU keeps both. For example two, X W = (3·0.5 + 4·1, 3·(−1) + 4·0.5) = (5.5, −1); adding the bias gives (5.5, 0); ReLU gives (5.5, 0).

Explain @ as matrix multiplication and * as elementwise multiplication in NumPy. A Python loop over examples and units would compute the same numbers, but each iteration runs in the interpreter; @ hands the whole product to optimized linear-algebra libraries, which is also what makes GPUs useful later.

The assert is a habit worth keeping: it documents the shape contract and fails loudly if it breaks.

PyTorch tensors mirror NumPy arrays

TENSORS, SHAPES, AND TOOLS

PyTorch tensors mirror NumPy arrays

```python
import numpy as np
import torch

a = np.arange(6, dtype=np.float32).reshape(2, 3)
t = torch.from_numpy(a)             # shares memory with a
print(t.shape, t.dtype, t.ndim)
print((t @ t.T).shape)              # (2, 3) @ (3, 2)
a[0, 0] = 100.                      # change the NumPy array...
print(t[0, 0])                      # ...and the tensor sees it
```

OUTPUT

```
torch.Size([2, 3]) torch.float32 2
torch.Size([2, 2])
tensor(100.)
```

`torch.from_numpy` shares memory; `torch.tensor` copies. NumPy defaults to float64, PyTorch layers to float32.

Neural Networks / Day 1 · Neural network foundations 24

[2 min] PyTorch tensors have the same shape, dtype and indexing ideas as NumPy arrays, plus two additions we need: a device (CPU or GPU) and the ability to record operations for automatic differentiation.

The memory-sharing demonstration matters in practice: torch.from_numpy(a) returns a tensor that views a's memory, so changing a changes t. torch.tensor(a) always copies; torch.as_tensor(a, dtype=torch.float32) copies when the dtype changes.

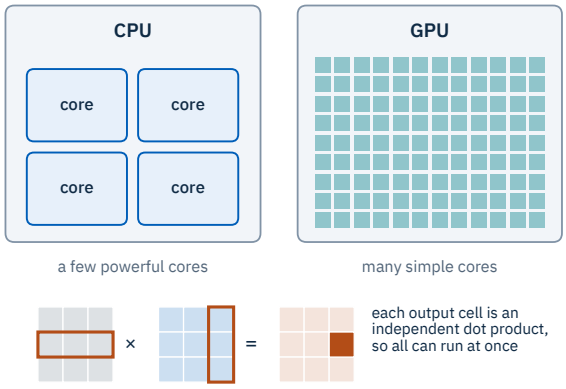
The dtype trap: pandas and NumPy produce float64 by default, while PyTorch layers are created in float32. Feeding float64 inputs to a float32 layer raises a dtype error, so convert explicitly (Part 7 shows the check). float32 halves memory and is what GPUs are fastest at; the gradient check in Part 6 is the exception, where float64 is required.

GPUs run many identical operations at once

TENSORS, SHAPES, AND TOOLS

GPUs run many identical operations at once

- A matrix product is many independent dot products.
- GPUs trade per-core flexibility for massive parallelism.
- Small tabular networks, like today's, train quickly on a CPU.
- Model and data must be on the same device.



Schematic; core counts are illustrative, not a specific device.

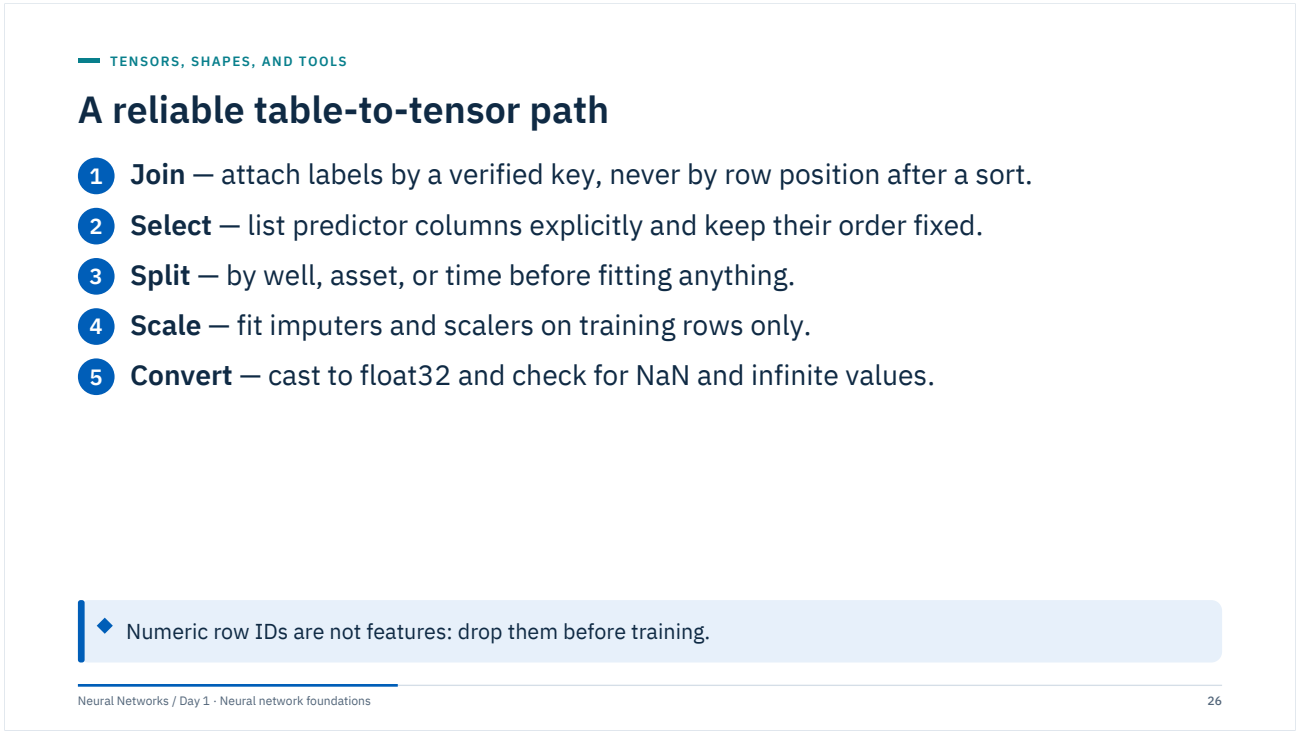
Neural Networks / Day 1 · Neural network foundations 25

[2 min] A CPU has a few powerful cores designed for varied, branching work. A GPU has many simpler cores that execute the same instruction on many numbers. Training is dominated by matrix products (and convolutions on Day 4), and every output cell of a matrix product is an independent dot product, so the work maps naturally onto a GPU.

Device selection in PyTorch: device = "cuda" if torch.cuda.is_available() else "cpu", then model.to(device) and X.to(device) for every batch. Apple silicon offers an "mps" backend. The most common first GPU error is "Expected all tensors to be on the same device".

Speed-ups are largest for big matrix products, convolutions and large batches. For a 60-feature, 208-row problem, the overhead of moving data can outweigh the gain; today's labs run comfortably on a laptop CPU. No benchmark numbers here: they depend entirely on hardware.

A reliable table-to-tensor path



[3 min] Walk the five steps. Join labels by a verified key whenever a key exists, never by row position after a sort. Select predictor columns explicitly and preserve their order, because a network has no column names: column 7 must mean the same curve at training and prediction time. Split before fitting imputation or scaling, and fit those steps on training rows only. Convert numeric predictors to float32 and check for NaN and infinite values, which silently poison a gradient.

Targets need the same care. For a binary target, float values with the same shape as the logits suit BCEWithLogitsLoss. For multiclass CrossEntropyLoss with class indices, targets should be integer class indices.

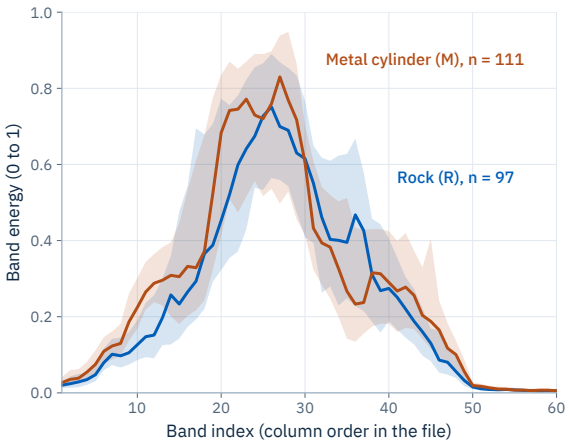
Do not include row IDs, well IDs or timestamps simply because they are numeric: they invite leakage and memorization.

Lab data: 208 sonar echoes with 60 band energies each

TENSORS, SHAPES, AND TOOLS

Lab data: 208 sonar echoes with 60 band energies each

- Labels: R = rock, M = metal cylinder, not "mineral".
- 97 rock and 111 metal echoes; values already in [0, 1].
- The classes overlap heavily in every single band.
- With 208 rows, one random split gives a noisy score.



Computed from sonar.all-data.csv: median and interquartile range of each band for each class.

Neural Networks / Day 1 · Neural network foundations 27

[3 min] Today's labs use the UCI "Connectionist Bench (Sonar, Mines vs. Rocks)" data: 208 sonar returns, each summarized as 60 numbers in [0, 1], the energy in a frequency band integrated over time. According to the UCI documentation, 111 returns come from a metal cylinder (label M) and 97 from rocks under similar conditions (label R). The source task is rocks versus metal cylinders, not minerals.

It is not an energy dataset, but the task shape, a fixed-length spectrum mapped to a class, matches classifying acoustic, vibration or seismic signatures. The plot shows why a model must combine bands: the interquartile ranges overlap in every band, so no single threshold separates the classes.

The file has no acquisition metadata (dates, angles, sites), so we cannot group related echoes. Keep that limitation in mind when you interpret any score today. The x-axis is the column order in the file.

Baseline pipeline for Sonar

TENSORS, SHAPES, AND TOOLS

Baseline pipeline for Sonar

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y_raw, random_state=42)
baseline = make_pipeline(StandardScaler(),
                         LogisticRegression(max_iter=1000))
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
scores = cross_val_score(baseline, X_train, y_train, cv=cv)
print(f"CV accuracy {scores.mean():.3f} +/- {scores.std():.3f}")
```

OUTPUT

```
CV accuracy 0.753 +/- 0.061
```

The 42 test rows stay untouched until one final evaluation.

Neural Networks / Day 1 · Neural network foundations 28

[3 min] X holds the 60 band energies, y_raw the M/R labels and y their explicit encoding, Mine = 1 and Rock = 0, as in both Day 1 labs. The labs make the same 80/20 split (stratified on the M/R column, random_state 42), so their 42 test rows are the ones held out here. The scaler sits inside the pipeline, so each cross-validation fold refits it on its own training rows: no statistics leak from the validation fold.

The five shuffled, stratified folds use only the 166 training rows. The output, 0.753 ± 0.061, is the logistic-regression baseline that every later model must beat on the same folds. The ± is the standard deviation across five folds of about 33 echoes, a reminder of how noisy small-data estimates are. A majority-class predictor scores 0.536 on the same folds.

Small sample size makes a single split noisy. Report uncertainty, and do not treat related measurements as independent when metadata to check is absent. Transition: in Lab 1 you will build this and two stronger baselines.

Lab 1: a Sonar baseline you can defend

LABORATORY

50 min

Lab 1: a Sonar baseline you can defend

- ☐ Load, explore, and encode the labels; print the label mapping.
- ☐ Build three pipelines: logistic regression, random forest, RBF SVM.
- ☐ Compare them with shuffled 5-fold CV on the training rows only.
- ☐ Tune the best of them with GridSearchCV on the same folds.
- ☐ Test the tuned model once; note what 42 rows can establish.

NOTEBOOK

`exercises/01_sklearn_pipeline/sklearn_Pipeline_Student.ipynb`

EVIDENCE TO SUBMIT

- Score table: CV mean ± sd per model, one final test score
- The printed label mapping and class counts
- Two sentences on the uncertainty of your best score

Neural Networks / Day 1 · Neural network foundations 29

[50 min lab, then 15 min debrief] Suggested timing: 5 minutes to open the notebook and run the imports; 10 minutes for sections 2–5 (load, explore, map the labels, split and define the folds); 15 minutes for the three pipelines and the CV comparison (sections 6–9); 10 minutes for the fold plot, the row-order check and grid search (sections 10–11); 10 minutes for the single test evaluation with its confusion matrix (section 12) and the written deliverables. If time is short, skip the row-order check in section 10, never the test evaluation in section 12.

Common problems. The labels are mapped explicitly, Mine = 1 and Rock = 0, as in Lab 2 and the Day 2 PyTorch lab; LabelEncoder would sort them alphabetically and make M = 0, so the notebook prints the mapping. Learners who pass cv=5 instead of the shared cv object get unshuffled folds that differ from everyone else's. Matplotlib 3.9 renamed boxplot's labels argument to tick_labels, and 3.11 rejects the old name; the notebook uses tick_labels.

Protocol: every choice is made with the same five shuffled, stratified folds on the 166 training rows (the code on the previous slide): the three pipelines in sections 6–8, the model family in section 9 (highest mean CV accuracy), and the hyperparameters in section 11. The 42 test rows are used once, in section 12, for the tuned model. Section 10's row-order check reruns the pipelines on the training rows in file order with unshuffled folds; ask learners to compare it with the shuffled folds. Expected numbers are in the debrief notes; do not reveal them early.

Lab 1 debrief: the evaluation protocol moves the score

DISCUSSION

Lab 1 debrief: the evaluation protocol moves the score



Computed from sonar.all-data.csv with the Lab 1 pipelines. Dots: single folds; bars: fold means; diamonds: the 42-row test split.

Neural Networks / Day 1 · Neural network foundations 30

[8 min] The figure scores the same three pipelines under three protocols. With shuffled 5-fold CV on the 166 training rows, the protocol the notebook uses to choose, they reach 0.75 to 0.83. With unshuffled folds that follow the file order they drop to 0.64 to 0.66 over all 208 rows, where one logistic-regression fold scores 0.40 (17 of 42), and to 0.59 to 0.65 in the notebook's row-order check, which uses the training rows only. The diamonds show what each pipeline would score on the 42-row test split (35, 37 and 36 of 42 for logistic regression, random forest and SVM); they were computed for this debrief, because in the lab only the final model touches the test rows.

The unshuffled folds follow the file order. Their lower, more variable scores suggest that neighbouring rows in the file are more alike than random pairs, so row order may carry structure that a random split hides. One plausible source: the UCI documentation says the original pattern labels were numbered in increasing aspect angle. Our file carries no such labels, so we cannot confirm it; treat it like wells or time periods in energy data, where contiguous blocks really are related.

Each test echo is worth 2.4 percentage points (1/42), so the model differences on the test split are one or two echoes. In the notebook the random forest and the SVM tie at 0.831 in CV, and the rule "highest mean" takes the forest by less than 0.001. GridSearchCV on the same folds reaches 0.837, slightly optimistic as the best of 18 settings, and the tuned forest scores 0.881 in its single test evaluation (37 of 42). A majority-class guess scores 0.52 (22 of 42).

Lab 1 debrief: three habits to keep all week



[5 min] Three habits, each from something learners did in the lab.

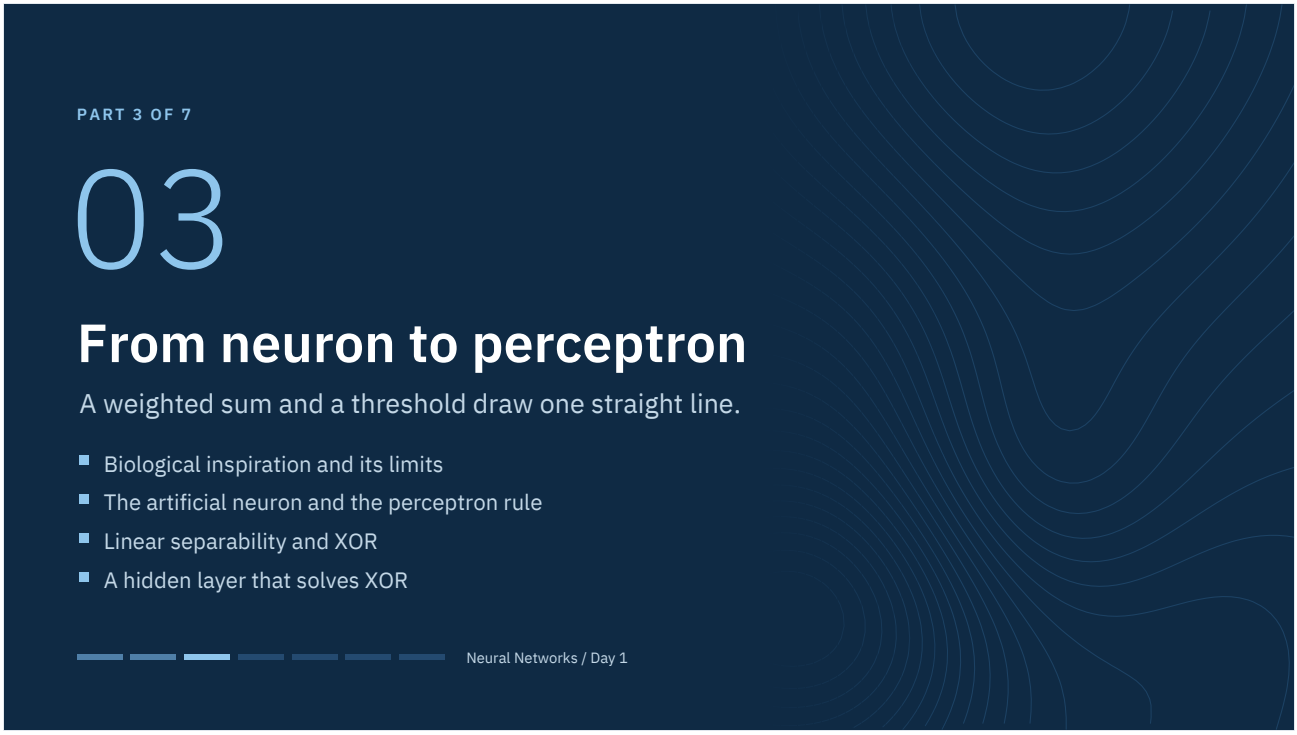
First, print the label mapping. Every Sonar lab this week maps Mine = 1 and Rock = 0; LabelEncoder would have sorted the labels alphabetically and made M = 0 without saying so. Metrics such as precision and recall depend on which class is positive; a silent flip between notebooks inverts them.

Second, keep the test set for last. Once a score has influenced a decision, such as which model family to tune, it no longer estimates performance on new data. Use cross-validation on training rows to choose, then evaluate once.

Third, treat row order as possible structure. Shuffled and unshuffled folds disagreed by more than ten accuracy points on the same data. In energy work the equivalent structure is known: wells, pads, pumps and time. Use GroupKFold or time-ordered splits whenever that information exists.

The callout gives the numbers to beat this afternoon on the same shuffled folds: 0.753 for logistic regression and 0.831 for both the random forest and the SVM.

From neuron to perceptron



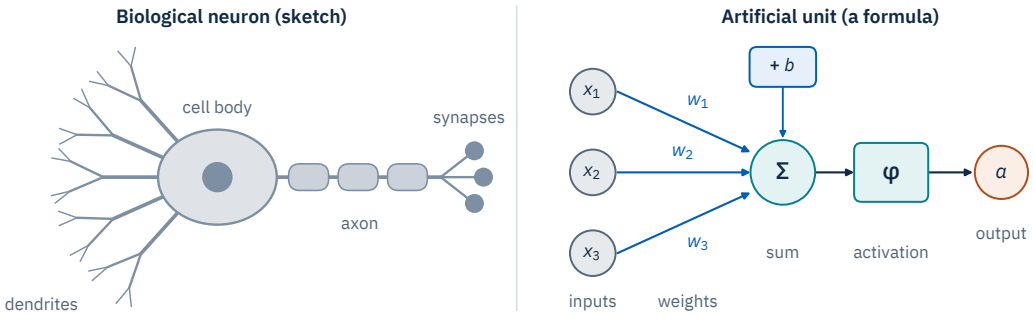
[1 min] Part 3 runs 11:40–12:30 and has three short exercises. The storyline follows the history slide: a neuron is a weighted sum plus a nonlinearity; the perceptron thresholds it and learns its weights; a single perceptron can only draw a straight line, so it cannot compute XOR; a hidden layer fixes that by re-mapping the inputs.

Keep the biology slide short. The important ideas are geometric: a weight vector, a boundary line, and a change of representation. Everything in the afternoon (multilayer networks, losses, backpropagation) builds on these pictures.

The artificial neuron borrows a loose biological metaphor

FROM NEURON TO PERCEPTRON

The artificial neuron borrows a loose biological metaphor



What the analogy leaves out: spike timing · computation inside dendrites · chemical signalling · adaptation over time

Schematic drawing, not anatomically accurate. The analogy motivated the model; the equations define it.

Neural Networks / Day 1 · Neural network foundations 33

[3 min] The historical mapping: dendrites receive signals, which play the role of inputs; synaptic strengths play the role of weights; the cell body integrates the incoming signals, like a weighted sum plus a bias; the neuron fires when the integrated signal is large enough, like an activation function.

Be explicit about the limits. Real neurons communicate with spikes whose timing matters; dendrites perform their own nonlinear computation; chemical signalling changes behaviour; synapses adapt continuously. The artificial unit is a fixed formula, $a = \varphi(\Sigma\, w_i x_i + b)$. Do not use the analogy to suggest biological fidelity or consciousness; all our reasoning today follows the equations.

Ask: which parts of the artificial unit are learned from data, and which are chosen by the designer? Learned: the weights and the bias. Chosen: the activation function and the number of inputs.

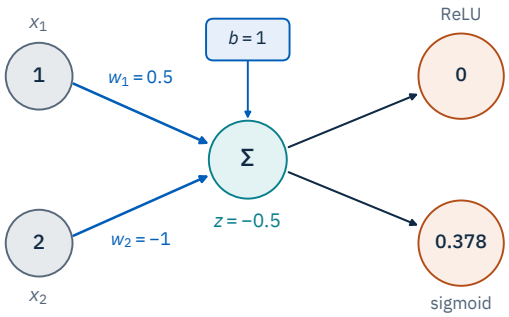
Weighted inputs, bias, and activation

FROM NEURON TO PERCEPTRON

Weighted inputs, bias, and activation

$$z = \sum_{j=1}^{d} w_j x_j + b, \qquad a = \phi(z)$$

- The weights set each input's contribution.
- The bias shifts the operating point.
- The **activation** φ shapes the response.



$z = 0.5 \cdot 1 + (-1) \cdot 2 + 1 = -0.5$

Exact arithmetic for inputs (1, 2), weights (0.5, −1), and bias 1.

[3 min] Define the symbols: x_j are the d inputs, w_j the weights, b the bias, z the pre-activation (the weighted sum) and a the output after the activation function φ.

Work the example with the room. For inputs (1, 2), weights (0.5, −1) and bias 1: z = 0.5·1 + (−1)·2 + 1 = −0.5. A ReLU outputs max(0, −0.5) = 0. A sigmoid outputs 1 / (1 + e^0.5) ≈ 0.378. The same weighted sum gives different outputs depending on φ; the choice of activation is part of the model design.

A negative weight means the input pushes the unit's output down. The bias lets the unit fire (or not) even when all inputs are zero; without it every boundary would have to pass through the origin.

Ask: which parts are learned and which are chosen? The weights and bias are learned; φ is chosen.

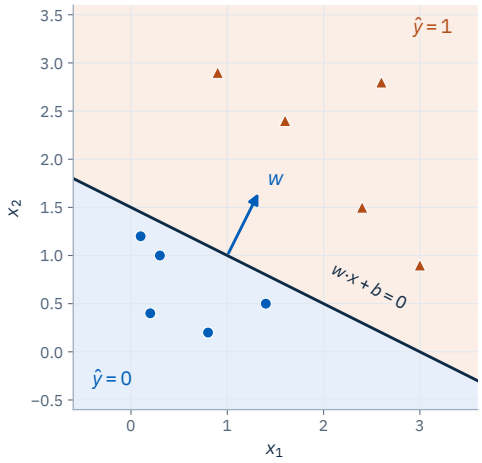
A perceptron thresholds a weighted sum into a class

FROM NEURON TO PERCEPTRON

A perceptron thresholds a weighted sum into a class

$$\hat{y} = \text{step}(w \cdot x + b)$$

- step(z) = 1 if z > 0, otherwise 0.
- The boundary $w \cdot x + b = 0$ is a straight line (a plane in more dimensions).
- The weight vector w is perpendicular to the boundary; b shifts it.



Exact geometry for w = (1, 2) and b = −3; illustrative points.

Neural Networks / Day 1 · Neural network foundations 35

[3 min] The perceptron is an artificial neuron with a step activation and a learning rule. Its prediction is 1 on one side of the boundary w·x + b = 0 and 0 on the other. In two dimensions the boundary is a line; in d dimensions a hyperplane.

Geometry worth drawing on the board: w points toward the ŷ = 1 side and is perpendicular to the boundary; changing b slides the line without rotating it. With w = (1, 2) and b = −3 the line is $x_1 + 2x_2 = 3$.

The perceptron is a linear classifier, like logistic regression. The differences are the hard threshold (no probability) and the learning rule on the next slide.

Ask: what happens to the boundary if b changes from −3 to −1? It shifts toward the origin, to $x_1 + 2x_2 = 1$, keeping its orientation.

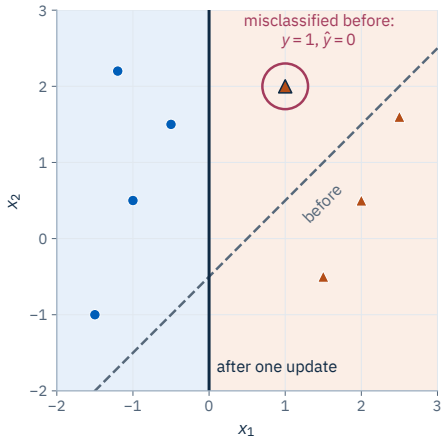
The perceptron rule moves the boundary toward mistakes

FROM NEURON TO PERCEPTRON

The perceptron rule moves the boundary toward mistakes

$$w \leftarrow w + \eta\,(y - \hat{y})\,x, \qquad b \leftarrow b + \eta\,(y - \hat{y})$$

- Correct predictions leave the weights unchanged.
- Worked update: x = (1, 2), y = 1, $\hat{y}$ = 0, η = 0.5.
- w: (1, −1) → (1.5, 0); b: −0.5 → 0.
- On separable data the rule stops after finitely many mistakes.



Exact update. Dashed: boundary before; solid: after one update. The ringed point is now classified correctly.

Neural Networks / Day 1 · Neural network foundations 36

[4 min] Walk the worked update. Before: z = 1·1 + (−1)·2 − 0.5 = −1.5, so ŷ = 0 while y = 1; the error y − ŷ is 1. The update adds η x = 0.5·(1, 2) = (0.5, 1.0) to w and η = 0.5 to b, giving w = (1.5, 0) and b = 0. After: z = 1.5·1 + 0·2 + 0 = 1.5, so ŷ = 1. The boundary has rotated and moved so the point is on the correct side; the other points stay correct.

η is the learning rate, the same symbol we will use for gradient descent. When y − ŷ = −1 (a false alarm) the rule subtracts η x instead.

Novikoff's convergence theorem (1962) guarantees that if a separating line exists, the perceptron makes only finitely many mistakes. If no separating line exists, the weights never settle: that is the XOR story two slides ahead.

Update a perceptron by hand

PRACTICE

5 min · Individual

Update a perceptron by hand

Start from $w = (0.2, -0.4)$, $b = 0$, and learning rate $\eta = 0.5$.

1. Present $x = (1, 1)$ with $y = 1$: compute z, $\hat{y}$, and the new w and b.
2. Then present $x = (2, -1)$ with $y = 0$ and update again.
3. Are both examples now classified correctly?

Neural Networks / Day 1 · Neural network foundations 37

[5 min] Worked answer.

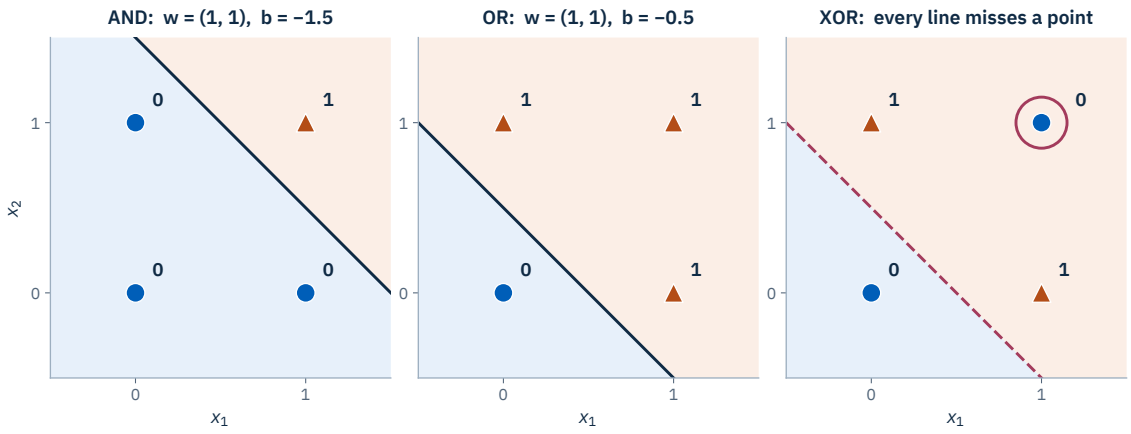
1. $z = 0.2 \cdot 1 + (-0.4) \cdot 1 + 0 = -0.2$, so $\hat{y} = 0$ while $y = 1$; error $+1$. New $w = (0.2, -0.4) + 0.5 \cdot (1, 1) = (0.7, 0.1)$; new $b = 0 + 0.5 = 0.5$. 2. With the new weights, $z = 0.7 \cdot 2 + 0.1 \cdot (-1) + 0.5 = 1.8$, so $\hat{y} = 1$ while $y = 0$; error -1. New $w = (0.7, 0.1) - 0.5 \cdot (2, -1) = (-0.3, 0.6)$; new $b = 0.5 - 0.5 = 0$. 3. Check: for $(1, 1)$, $z = -0.3 + 0.6 + 0 = 0.3 > 0$, so $\hat{y} = 1$, correct. For $(2, -1)$, $z = -0.6 - 0.6 + 0 = -1.2$, so $\hat{y} = 0$, correct. Both are now right.

Common errors: forgetting to update the bias; using the old weights for the second example; flipping the sign of the second update. Point out that the second update partly undid the first, which is why the perceptron can take many passes on harder data.

AND and OR are linearly separable; XOR is not

FROM NEURON TO PERCEPTRON

AND and OR are linearly separable; XOR is not



Exact threshold units. On XOR even the best line, here the OR line, misclassifies (1, 1); the perceptron rule keeps making mistakes on every pass.

Neural Networks / Day 1 · Neural network foundations 38

[3 min] AND: with w = (1, 1) and b = −1.5, only (1, 1) gives z = 0.5 > 0. OR: with b = −0.5, only (0, 0) gives z < 0. Keep these two units in mind: they reappear as the hidden units of the XOR network.

XOR outputs 1 when exactly one input is 1. The positive points sit on one diagonal of the square and the negatives on the other, so no single line puts both positives on one side and both negatives on the other. Any line gets at least one of the four points wrong; the OR line shown gets (1, 1) wrong.

Computed check: the perceptron rule from a zero start with η = 0.5 made 2 mistakes in the first pass, 3 in the second and 4 in each of the remaining 18 passes; its weights cycle instead of settling. This is the limitation Minsky and Papert analysed in 1969. The next exercise proves it.

Show that no single threshold unit computes XOR

PRACTICE

5 min Pairs

Show that no single threshold unit computes XOR

A unit outputs 1 when $w_1x_1 + w_2x_2 + b > 0$ and 0 otherwise.

1. Write the four inequalities that XOR requires.
2. Add the two inequalities for the inputs (0, 1) and (1, 0).
3. Show that the result contradicts the other two.

Neural Networks / Day 1 · Neural network foundations 39

[5 min] Worked answer.

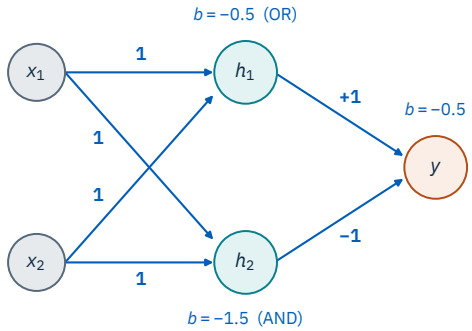
1. The four conditions: (0, 0) → 0 requires $b \le 0$. (0, 1) → 1 requires $w_2 + b > 0$. (1, 0) → 1 requires $w_1 + b > 0$. (1, 1) → 0 requires $w_1 + w_2 + b \le 0$. 2. Adding the middle two gives $w_1 + w_2 + 2b > 0$. 3. Rearranged, $w_1 + w_2 + b > -b$. Because $b \le 0$, $-b \ge 0$, so $w_1 + w_2 + b > 0$, which contradicts $w_1 + w_2 + b \le 0$. No weights and bias satisfy all four conditions.

Ask one pair to present at the board. Common difficulty: learners try specific numbers instead of reasoning with inequalities; nudge them to add the two "must fire" conditions. The proof does not depend on the threshold convention: with "fires when ≥ 0" the inequalities flip strictness but the contradiction remains.

Two hidden units solve XOR: OR and not AND

FROM NEURON TO PERCEPTRON

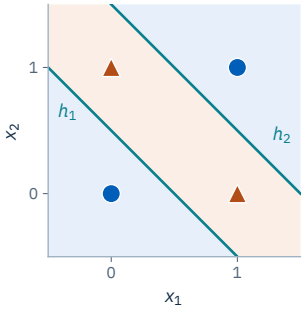
Two hidden units solve XOR: OR and not AND



all units: step(z) = 1 if z > 0, else 0

x_1	x_2	h_1	h_2	y
0	0	0	0	**0**
0	1	1	0	**1**
1	0	1	0	**1**
1	1	1	1	**0**

$y = h_1$ AND NOT h_2 = XOR



Exact hand-set weights with step activations. Hidden unit 1 is the OR line, hidden unit 2 the AND line; the output fires in the band between them.

Neural Networks / Day 1 · Neural network foundations 40

[4 min] Trace the truth table with the room. For (0, 1): h₁ = step(0 + 1 − 0.5) = 1, h₂ = step(0 + 1 − 1.5) = 0, y = step(1 − 0 − 0.5) = 1. For (1, 1): h₁ = step(1.5) = 1, h₂ = step(0.5) = 1, y = step(1 − 1 − 0.5) = 0. For (0, 0): h₁ = h₂ = 0 and y = step(−0.5) = 0. So y = h₁ AND NOT h₂, which is XOR.

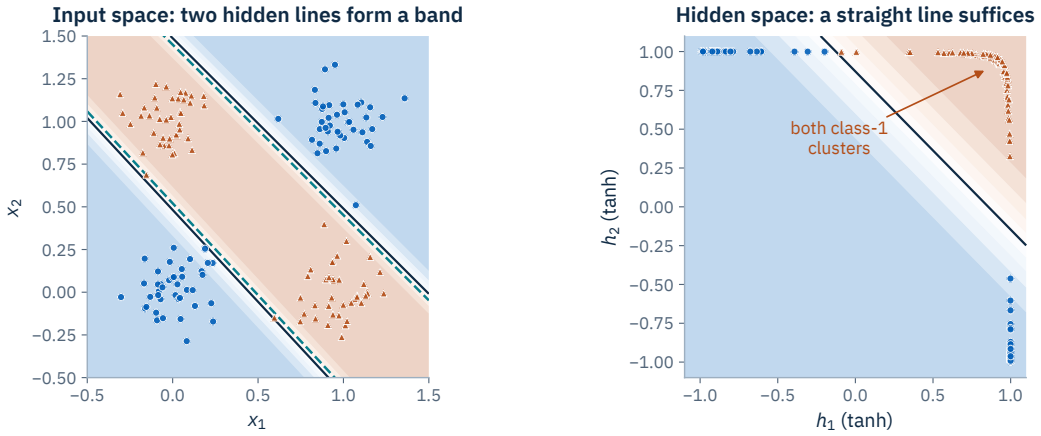
Each hidden unit draws one line in input space, the OR line and the AND line from the previous slides. The output unit combines them, and the region where it fires is the band between the two lines, a shape no single line can make.

These weights were set by hand, using step functions that have no useful gradient. Next we let gradient descent find its own weights with smooth activations.

The hidden layer re-maps inputs so that a line suffices

FROM NEURON TO PERCEPTRON

The hidden layer re-maps inputs so that a line suffices



Computed: noisy XOR (4 × 40 points, seed 0); 2–2–1 tanh network, initialization seed 2, 300 full-batch gradient steps. Solid lines: decision boundary (p = 0.5); dashed: where each hidden unit is zero.

[4 min] Left: input space with the learned probability. The trained network found a solution much like the hand-set one: each hidden unit draws a line (dashed), and the boundary, the band between them, is built from those two lines.

Right: every point plotted by its two hidden activations (h_1, h_2) instead of its inputs. Both class-1 clusters land in the same corner; the two class-0 clusters land in the other corners. In this new space one straight line, the output unit, separates the classes.

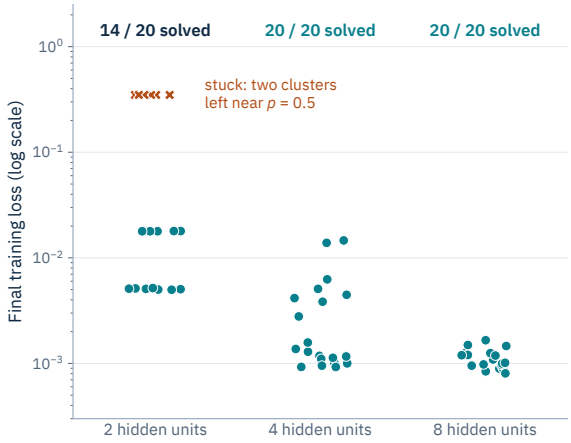
This is the central idea of representation learning: hidden layers transform the data until a simple final layer suffices. Deeper networks repeat the trick several times. The tanh units are nearly saturated (close to ±1) once the problem is solved, which makes them behave like the hand-set threshold units. Training accuracy is 1.000.

Gradient descent solves XOR, but not from every start

FROM NEURON TO PERCEPTRON

Gradient descent solves XOR, but not from every start

- Same data and training recipe; only the initial weights differ.
- 2 hidden units: 14 of 20 starts solve XOR.
- The others stall with two clusters left at $p \approx 0.5$.
- 4 or 8 hidden units: all 20 starts solve it.



Computed on the same noisy XOR data: 20 initialization seeds per width, 4000 full-batch steps, learning rate 1.0.

Neural Networks / Day 1 · Neural network foundations 42

[3 min] A network with barely enough capacity can get stuck: 6 of the 20 two-unit runs stalled at a training loss of 0.349, close to ½ ln 2 ≈ 0.347. The network outputs p ≈ 0.5 for two of the four clusters and gets the other two right, so training accuracy is only 0.64 to 0.72; whether a point in an undecided cluster lands just above or just below 0.5 is essentially arbitrary. With 4 or 8 hidden units every run solved the problem, with final losses between about 0.001 and 0.015.

Why it matters: the loss surface of a network is not a single bowl; there are flat regions and poor solutions. Extra units create more ways to reach a good solution, one reason practical networks are wider than the minimum needed, and one reason Day 2 spends time on initialization.

Practical habits: fix and report random seeds; when a run looks stuck, try several seeds before redesigning the model; and never report the best of many seeds as if it were typical.

Which of these can a single perceptron represent?

PRACTICE

3 min Pairs

Which of these can a single perceptron represent?

Inputs are 0 or 1 unless stated; a unit fires when $w \cdot x + b > 0$.

1. NAND of two inputs
2. XNOR: 1 when both inputs are equal
3. Majority vote of three inputs
4. "$x_1 > x_2$" for real-valued inputs

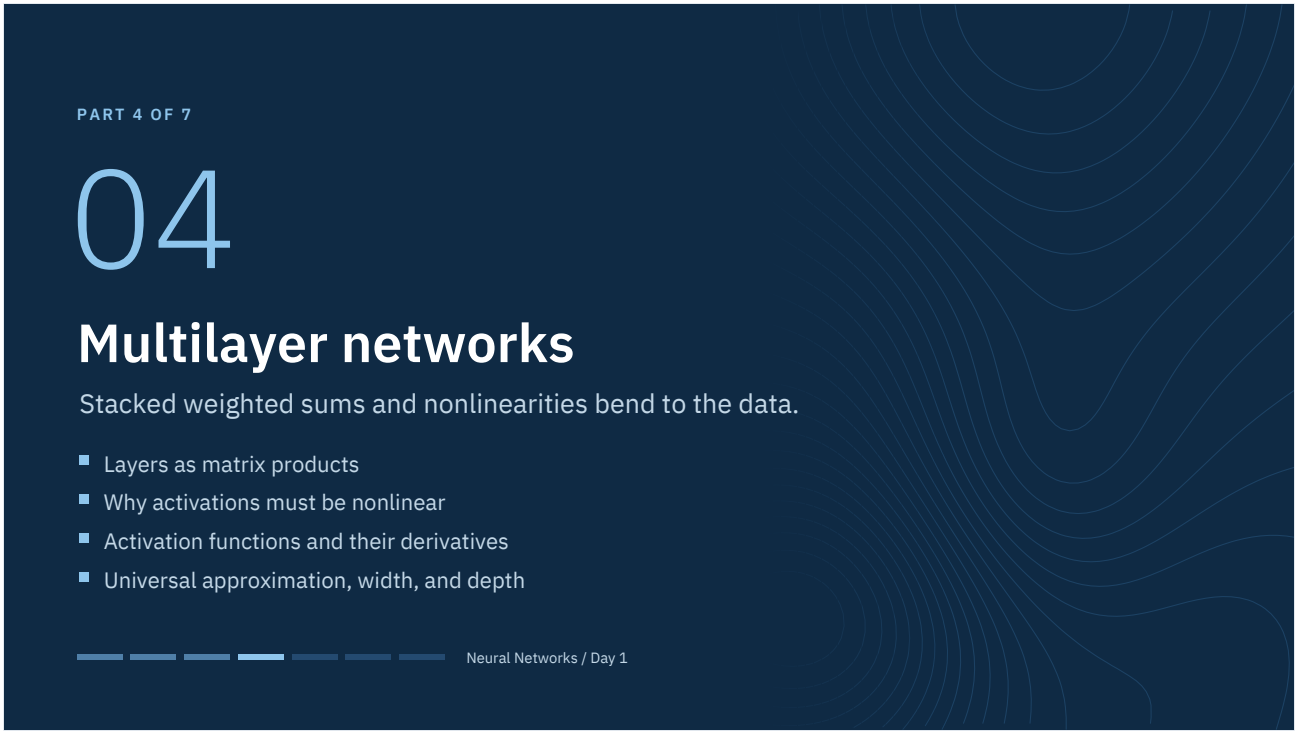
Neural Networks / Day 1 · Neural network foundations 43

[3 min] Worked answers.

1. NAND: yes. w = (−1, −1), b = 1.5 fires unless both inputs are 1 (then z = −0.5). 2. XNOR: no. It is NOT XOR, with the same diagonal geometry, so the proof from the previous exercise applies with the classes swapped. 3. Majority of three: yes. w = (1, 1, 1), b = −1.5 fires when at least two inputs are 1 (z = 0.5 or 1.5) and not otherwise (z = −0.5 or −1.5). 4. $x_1 > x_2$: yes. w = (1, −1), b = 0 gives z = $x_1 - x_2$ > 0 exactly when $x_1 > x_2$.

Close Part 3 at 12:30. Before lunch, preview the afternoon: we replace step functions with smooth activations, stack layers, and learn every weight with gradients.

Multilayer networks



[1 min] Part 4 runs 13:30–14:10. It generalizes the XOR network into the multilayer perceptron: layers written as matrix products, the reason nonlinear activations are required, the common activation functions, what "universal approximation" does and does not promise, and how width and depth change a decision boundary.

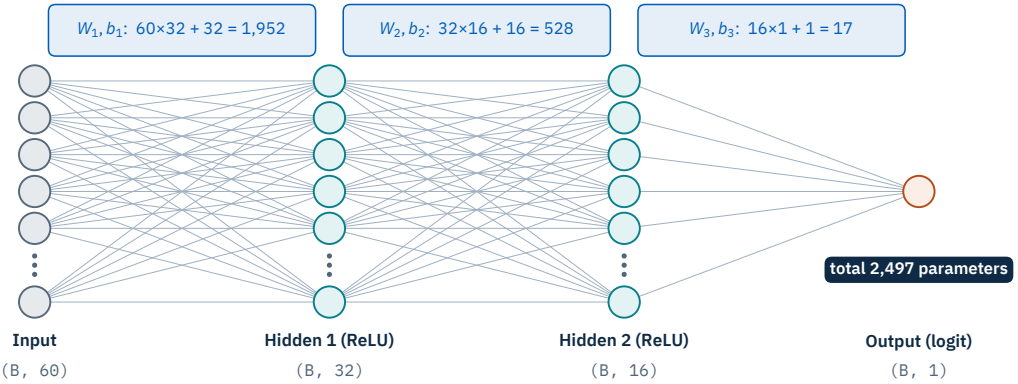
It closes with two hands-on items: a parameter count and a forward pass by hand. The worked network from the forward-pass slide is reused in Part 6 for the backward pass, so make sure everyone has its numbers written down.

A multilayer perceptron is a chain of dense layers

MULTILAYER NETWORKS

A multilayer perceptron is a chain of dense layers

Each layer maps (B, in) to (B, out) with a weight matrix (in, out) and one bias per output unit.



Schematic of a 60–32–16–1 network for Sonar-sized input; parameter counts are exact.

Neural Networks / Day 1 · Neural network foundations 45

[3 min] "Multilayer perceptron" (MLP) is the historical name for a fully connected feedforward network; modern MLPs use smooth or ReLU activations rather than step functions. Every unit in one layer connects to every unit in the next, which is why such layers are called dense or fully connected.

Count with the room: 60 × 32 + 32 = 1,952; 32 × 16 + 16 = 528; 16 × 1 + 1 = 17; total 2,497 parameters. With 166 training echoes, that is about 15 parameters per training example, one reason small tabular problems overfit so easily (Day 2 covers regularization).

The output layer has a single unit producing a raw score, the logit. A sigmoid turns it into a probability when we need one; the loss function in Part 5 works on the logit directly.

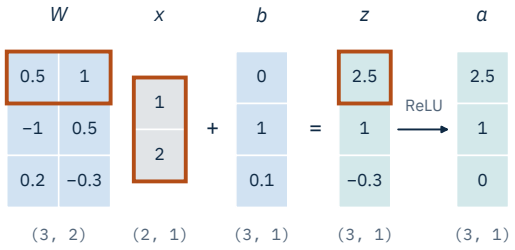
A dense layer is one matrix product plus a bias

MULTILAYER NETWORKS

A dense layer is one matrix product plus a bias

$$z = Wx + b, \qquad a = \mathrm{ReLU}(z)$$

- One example in as a column vector *x*, one vector *a* out.
- Row *j* of *W* holds the weights of unit *j*.
- A batch stacks examples as rows: $Z = XW^\top + b$.



$z_1 = 0.5 \cdot 1 + 1.0 \cdot 2 + 0 = 2.5$

row *j* of *W* holds the weights of unit *j*

Exact arithmetic: z = (2.5, 1.0, −0.3), so a = (2.5, 1.0, 0).

Neural Networks / Day 1 · Neural network foundations 46

[3 min] This is the single-example, column-vector convention common in textbooks: z = Wx + b, with W of shape (units, inputs) = (3, 2) here, so row j of W holds the weights of unit j. Part 2 wrote the same layer for a batch of rows, Z = XW + b with W of shape (inputs, units). The two matrices are transposes of each other: stacking examples as rows gives Z = X $W^\top$ + b, where $W^\top$ has shape (2, 3) and plays the role of Part 2's (D, H) matrix. PyTorch's nn.Linear(2, 3) stores its weight exactly as this (3, 2) W and computes x · $\text{weight}^\top$ + bias for a batch of rows.

The first two rows reproduce the two hidden units of the worked network we use this afternoon: weights (0.5, 1.0) with bias 0 and weights (−1.0, 0.5) with bias 1. The third unit shows ReLU switching off a negative pre-activation.

Ask: compute z_3. Answer: 0.2·1 + (−0.3)·2 + 0.1 = −0.3, and ReLU gives 0.

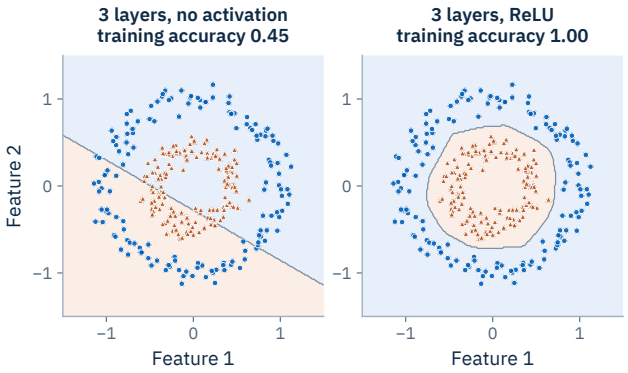
Stacking linear layers is still linear

MULTILAYER NETWORKS

Stacking linear layers is still linear

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

- Without activations, any stack of layers collapses into one.
- Nonlinear activations let hidden units bend the boundary.
- Same data, same three layers: only the activation differs.



Computed: synthetic circles (seed 0); scikit-learn MLPClassifier with three hidden layers of 16 units.

Neural Networks / Day 1 · Neural network foundations 47

[3 min] The algebra: two affine layers compose into one affine map with weight W_2W_1 and bias $W_2b_1 + b_2$. By induction, any number of affine layers is still one affine layer. This equation uses column vectors for compactness; the row-batch convention gives the same result after transposing weights.

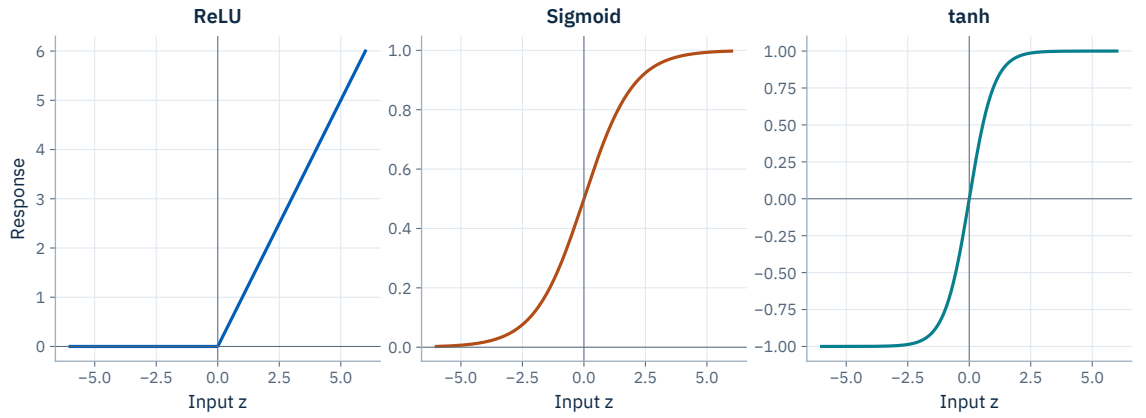
The figure makes it concrete. Both networks have three hidden layers of 16 units. With the identity activation the boundary is a straight line and training accuracy is 0.45: a line cannot separate a ring from its centre. With ReLU the boundary closes around the inner class and training accuracy is 1.00.

This is not a claim that every deep nonlinear model is easy to optimize or better than a shallow one; it only shows why the nonlinearity is essential. Transition: which nonlinearity?

Activation functions respond differently

MULTILAYER NETWORKS

Activation functions respond differently



Exact activation functions over a common input range. ReLU, sigmoid, and tanh have different ranges and saturation behaviour.

Neural Networks / Day 1 · Neural network foundations 48

[3 min] Sigmoid maps to (0, 1) and is the natural output for a binary probability; tanh maps to (−1, 1) and is zero-centred; ReLU is max(0, z), unbounded above and exactly zero for negative inputs.

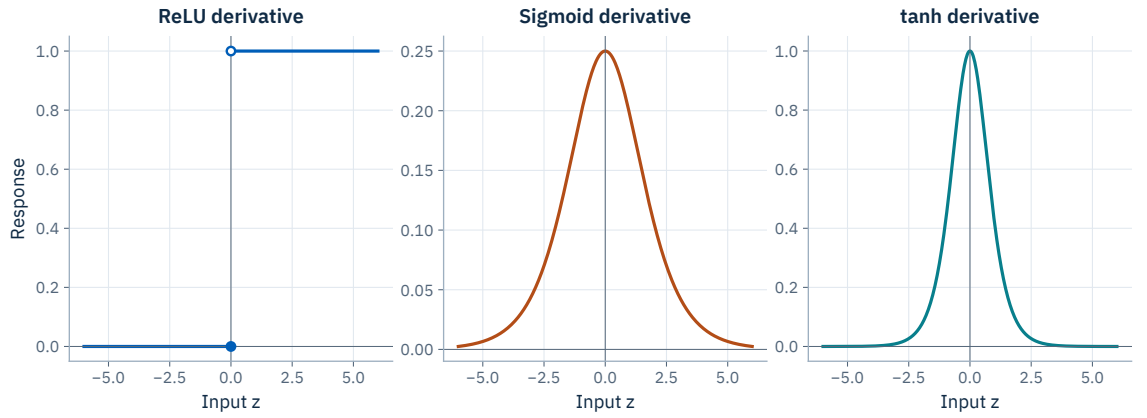
Hidden layers in modern networks commonly use ReLU-family functions because they are cheap and do not saturate for positive inputs. Sigmoid is useful for binary probabilities at inference, but the stable training loss consumes raw logits (Part 5), so the sigmoid is usually not a layer inside the model.

Lab 2's NumPy network uses sigmoid hidden units, as in many classic textbooks; notice how that choice affects its derivative on the next slide.

Derivatives explain saturation

MULTILAYER NETWORKS

Derivatives explain saturation



Analytical derivatives. ReLU's derivative at zero is undefined mathematically; software selects a convention.

Neural Networks / Day 1 · Neural network foundations 49

[3 min] Backpropagation multiplies derivatives like these (Part 6), so their size matters. The sigmoid derivative is $\sigma(z)(1 - \sigma(z))$, with maximum 0.25 at $z = 0$. The tanh derivative is $1 - \tanh^2(z)$, with maximum 1. Both are close to zero when $|z|$ is large: a saturated unit passes almost no gradient.

ReLU's derivative is 1 for positive inputs and 0 for negative ones. A ReLU unit that is negative for every training input receives no gradient at all and can stop learning, a "dead" unit. At exactly zero the derivative is undefined; libraries pick a convention (PyTorch uses 0).

Preview: multiplying many factors of at most 0.25 through a deep sigmoid network shrinks gradients quickly. Day 2 treats this properly under initialization and signal propagation.

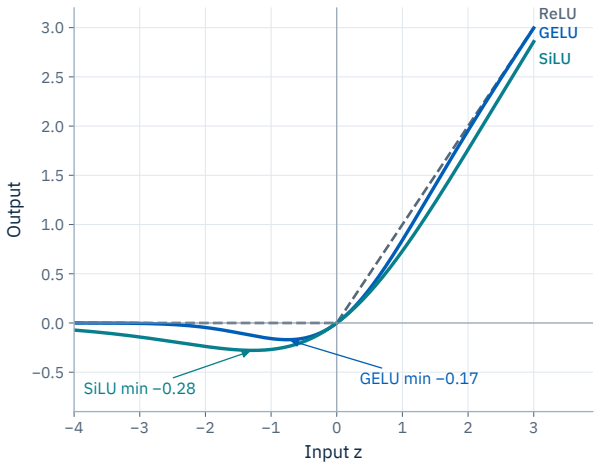
GELU and SiLU provide smooth alternatives

MULTILAYER NETWORKS

GELU and SiLU provide smooth alternatives

$$\mathrm{GELU}(z) = z\,\Phi(z), \qquad \mathrm{SiLU}(z) = z\,\sigma(z)$$

- GELU weights z by the standard-normal CDF Φ(*z*).
- SiLU, also called Swish, gates z with a sigmoid.
- Both are smooth and dip slightly below zero.
- Choose within a tested architecture, not in isolation.



Exact functions. Minima: GELU −0.17, SiLU −0.28.

Neural Networks / Day 1 · Neural network foundations 50

[2 min] These functions are common in modern networks; GELU appears in many transformer models. Both behave like ReLU for large positive inputs, but they are smooth at zero and allow small negative outputs (minimum about −0.17 for GELU near z ≈ −0.75 and −0.28 for SiLU near z ≈ −1.28), so gradients do not switch off abruptly.

SiLU is the Swish function with its slope parameter fixed at 1. Library GELU implementations may use the exact erf form or a tanh approximation; in PyTorch, nn.GELU(approximate="tanh") selects the latter.

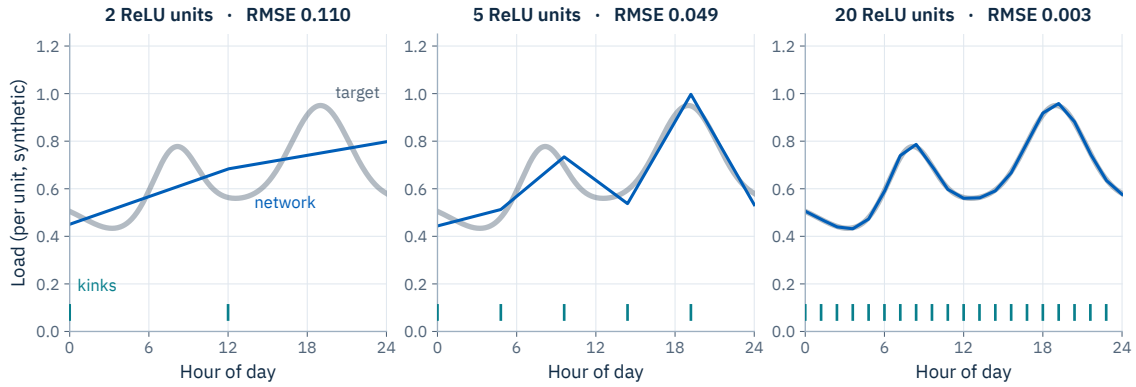
Avoid presenting one activation as universally best: differences are usually small compared with data, initialization and learning rate, and they interact with the architecture. Keep today's derivations with ReLU and sigmoid to reduce cognitive load.

Enough ReLU units can approximate a smooth curve

MULTILAYER NETWORKS

Enough ReLU units can approximate a smooth curve

Each ReLU unit adds one kink; the output layer weights and sums them.



Computed: synthetic daily-load-shaped curve, not measured data. One hidden layer, evenly spaced kinks, output weights by least squares.

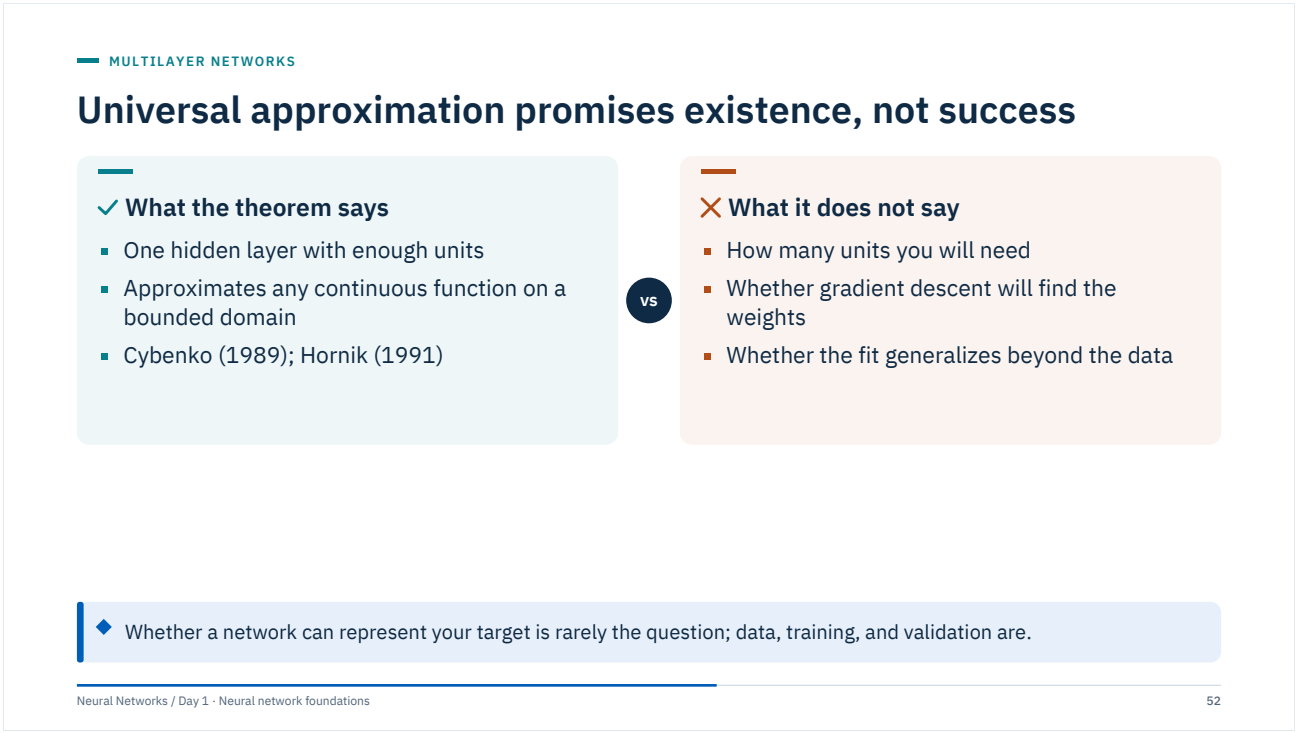
Neural Networks / Day 1 · Neural network foundations 51

[3 min] The network here is $f(t) = c + \Sigma_i a_i \, \mathrm{ReLU}(t - k_i)$: one hidden layer whose unit i has input weight 1 and bias $-k_i$, so it bends at $t = k_i$, and an output layer with weights a_i and bias c. With the kinks fixed, choosing the output weights is a linear least-squares problem, so we can compute the best fit exactly.

The error (RMSE, in the curve's per-unit scale) falls from 0.110 with 2 units to 0.049 with 5 and 0.003 with 20. More units, more kinks, a closer piecewise-linear fit.

In a trained network the kink positions and slopes are learned too, so fewer units can do the same job, but training must find them. The target is a synthetic daily-load shape chosen to look familiar; it is not measured data. This is the intuition behind the universal approximation theorem on the next slide.

Universal approximation promises existence, not success



[3 min] The precise statements are technical: for suitable activation functions, one hidden layer with enough units can approximate any continuous function on a compact (closed and bounded) domain to any accuracy. Cybenko (1989) proved it for sigmoidal activations and Hornik (1991) for a broad class of activations; later work extended it to non-polynomial activations such as ReLU.

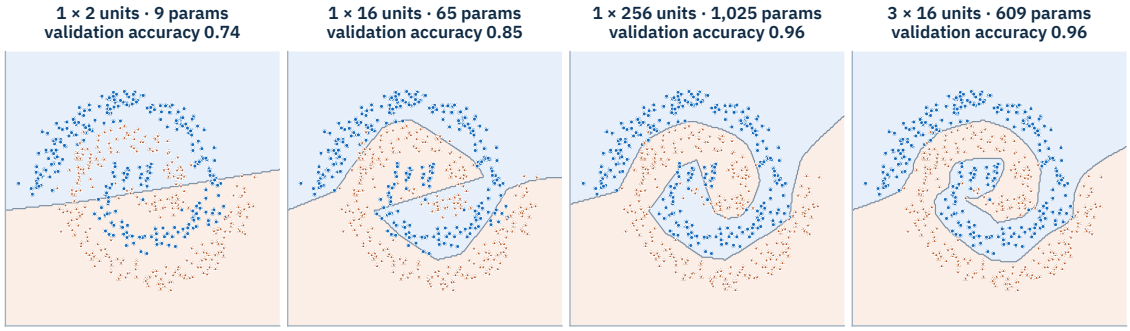
The right-hand card is what matters in practice. The number of units needed can be enormous. Existence says nothing about optimization: the XOR seeds already showed gradient descent failing from some starts. And fitting training points is not generalizing: a network flexible enough to fit anything can also fit noise.

Ask: the theorem says one hidden layer suffices, so why do we use deep networks? Answer: depth can represent many functions far more economically than width, and deep networks are often easier to train to good solutions in practice; the next slide shows one example.

Width and depth both add capacity to the boundary

MULTILAYER NETWORKS

Width and depth both add capacity to the boundary



Computed: synthetic two spirals (seed 0, 70/30 split); scikit-learn MLPClassifier defaults (ReLU, Adam), random_state 0. Dots: training points; accuracy on the 30% validation points.

Neural Networks / Day 1 · Neural network foundations 53

[3 min] Four networks on the same spirals. Two hidden units underfit badly (validation accuracy 0.74): two ReLU kinks cannot follow a spiral. Sixteen units carve a rough spiral (0.85). A single layer of 256 units (1,025 parameters) and three layers of 16 (609 parameters) both reach 0.96.

In this run the deeper network matched the wide one with fewer parameters. That is a common pattern for structured problems, not a law; with other seeds or solvers the ranking can change. Deeper networks compose features, bending already-bent boundaries, while wide shallow networks add many independent pieces. Depth brings its own optimization challenges, which Day 2 addresses with initialization and normalization.

Lesson for practice: do not choose architectures by eye or by slogan; compare candidates on validation data, as with any other hyperparameter.

Dimensions determine parameter count

PRACTICE

3 min · Individual

Dimensions determine parameter count

A dense network maps 6 inputs to 8 hidden units, then to 1 output.

1. Count the weights and biases in each layer.
2. State the output shape for a batch of 32 observations.
3. How does doubling the batch size change the parameter count?

[3 min] Worked answer.

1. First layer: 6 × 8 = 48 weights plus 8 biases = 56 parameters. Second layer: 8 × 1 = 8 weights plus 1 bias = 9 parameters. Total 65. 2. The output shape is (32, 1): one logit per observation. 3. It does not change: the batch size affects the memory used for activations and gradients during training, not the number of learned parameters.

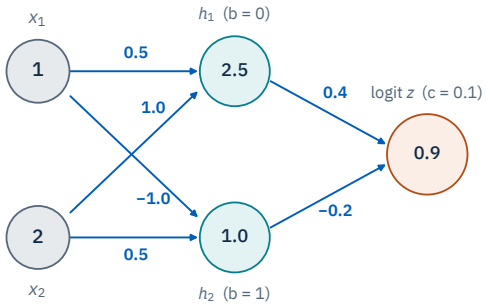
Common errors: forgetting the biases (answering 56 as 48); writing the output shape as (32,), which is valid only after squeezing and then no longer matches a (32, 1) target; thinking a larger batch means a larger model.

A two-hidden-unit forward pass by hand

MULTILAYER NETWORKS

A two-hidden-unit forward pass by hand

- $h_1 = \text{ReLU}(0.5 \cdot 1 + 1.0 \cdot 2 + 0) = 2.5$
- $h_2 = \text{ReLU}(-1.0 \cdot 1 + 0.5 \cdot 2 + 1) = 1.0$
- $z = 0.4 \cdot 2.5 - 0.2 \cdot 1.0 + 0.1 = 0.9$
- $p = \sigma(0.9) \approx 0.711$



$p = \sigma(0.9) = 0.711$ · ReLU keeps both hidden values

Write these numbers down: Part 6 sends gradients back through this exact network.

Exact arithmetic. The weights are chosen for clear arithmetic, not fitted to data.

Neural Networks / Day 1 · Neural network foundations 55

[3 min] Calculate each line with the class. Inputs x = (1, 2). Hidden unit 1 has weights (0.5, 1.0) and bias 0; hidden unit 2 has weights (−1.0, 0.5) and bias 1; both use ReLU. The output unit has weights (0.4, −0.2) and bias 0.1 and produces the logit z. The sigmoid turns z = 0.9 into the event probability p ≈ 0.711.

These values are chosen for arithmetic clarity; they are not fitted model parameters. Both hidden units are active (positive pre-activations), which keeps the backward example in Part 6 simple.

Ask everyone to write these numbers down: after the break we push gradients back through exactly this network, and in Part 7 PyTorch reproduces them.

Run the same network on a new input

PRACTICE

4 min · Individual

Run the same network on a new input

Use the weights from the previous slide with $x = (3, 1)$.

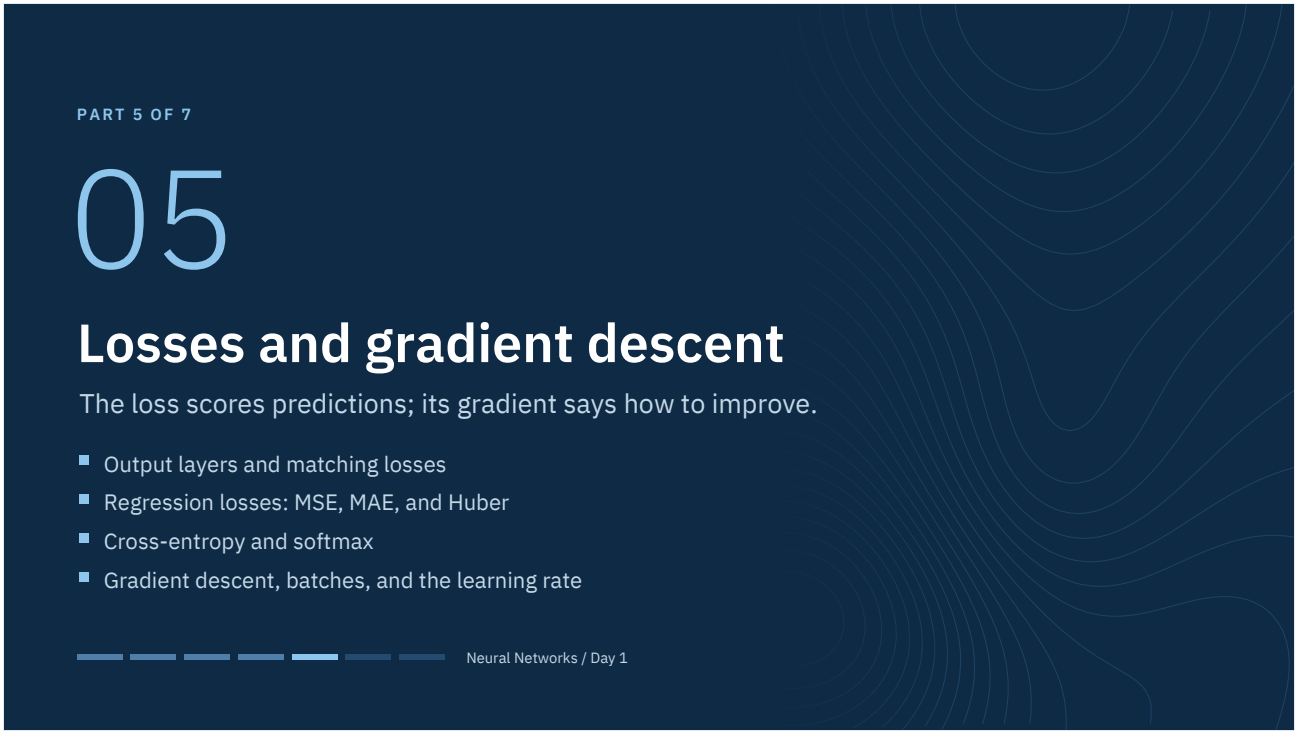
1. Compute h_1, h_2, the logit z, and p.
2. Which hidden unit is inactive, and why?
3. Which weights receive zero gradient from this example?

[4 min] Worked answer.

1. $h_1 = \text{ReLU}(0.5 \cdot 3 + 1.0 \cdot 1 + 0) = \text{ReLU}(2.5) = 2.5$. $h_2 = \text{ReLU}(-1.0 \cdot 3 + 0.5 \cdot 1 + 1) = \text{ReLU}(-1.5) = 0$. $z = 0.4 \cdot 2.5 - 0.2 \cdot 0 + 0.1 = 1.1$. $p = \sigma(1.1) \approx 0.750$. 2. Hidden unit 2 is inactive: its pre-activation is negative, so ReLU outputs 0. 3. Zero gradient: the two weights into h_2 (−1.0 and 0.5) and its bias, because ReLU's derivative is 0 there; and the output weight on h_2 (−0.2), because its input h_2 is 0. The weights into h_1, the output weight on h_1 and the output bias do receive gradient.

If a unit is inactive for every training example it stops learning altogether, a dead ReLU (Day 2). Close Part 4 at about 14:10.

Losses and gradient descent



[1 min] Part 5 runs 14:10–14:45. So far the weights have been hand-set. To learn them we need two things: a loss that turns "good predictions" into a single number, and a procedure that changes the weights to reduce it.

The first half matches losses to tasks (regression, binary, multiclass, multilabel) and shows how each loss treats errors. The second half is gradient descent on a real loss surface: full-batch, mini-batch and single-example updates, and a first look at the learning rate. Optimizers such as momentum and Adam, and learning-rate schedules, are Day 2 topics; keep today's treatment to plain gradient descent.

Output and loss must agree

LOSSES AND GRADIENT DESCENT

Output and loss must agree

Task	Model output	Training target and loss
Regression	Unconstrained numeric output	Numeric target, MSE or Huber
Binary classification	One raw logit per example	Float 0/1, BCEWithLogitsLoss
Multiclass classification	C raw logits per example	Integer class index, CrossEntropyLoss
Multilabel classification	One logit per independent label	Multiple 0/1 targets, binary losses

◆ Do not apply sigmoid or softmax before a loss that expects raw logits.

Neural Networks / Day 1 · Neural network foundations 58

[3 min] Walk the rows. Regression: the last layer has no activation, so any real value can be predicted; MSE or Huber compares it with a numeric target. Binary classification: one raw logit per example and a float 0/1 target of the same shape; BCEWithLogitsLoss applies the sigmoid internally in a numerically stable way. Multiclass: C logits and an integer class index; CrossEntropyLoss applies the softmax internally. Multilabel: one logit per label, several labels can be 1 at once, and each label gets its own binary loss.

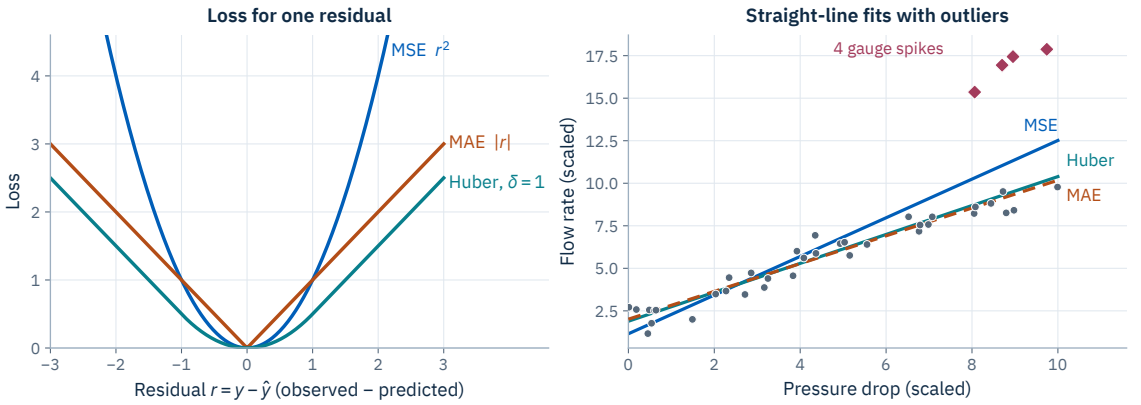
Multiclass classes are mutually exclusive; multilabel outcomes need not be. PyTorch's CrossEntropyLoss can also accept class-probability targets, but integer indices are the simpler core path.

The most common silent bug is a duplicated activation: a sigmoid layer inside the model followed by BCEWithLogitsLoss. The code runs, the loss decreases, and the probabilities are wrong.

MSE, MAE, and Huber treat large errors differently

LOSSES AND GRADIENT DESCENT

MSE, MAE, and Huber treat large errors differently



Left: exact loss functions. Right: synthetic data (seed 5), true slope 0.8, with 4 added spikes. Fitted slopes: MSE 1.14, MAE 0.82, Huber 0.85.

[3 min] Left: loss for a single residual r = y − ŷ, observed minus predicted, the convention used throughout the course. All three losses are symmetric, so the sign of r does not change them. MSE squares the residual, so large errors dominate. MAE grows linearly, so each error counts in proportion to its size. Huber is quadratic for small residuals and linear beyond δ, a compromise.

Right: the same effect on a fit. Forty synthetic points along a line with slope 0.8, plus four "gauge spikes". The MSE line is pulled up toward the spikes (slope 1.14); the MAE fit follows the median (0.82); Huber stays close to it (0.85). These are linear models fitted with scikit-learn (LinearRegression, QuantileRegressor at the median, HuberRegressor with its default settings), not networks, but a network trained with each loss is pulled in the same way.

Choose by the cost of errors and the nature of outliers: spikes that are measurement errors call for cleaning or a robust loss; rare but genuine extremes that matter may call for MSE. PyTorch provides nn.MSELoss, nn.L1Loss and nn.HuberLoss(delta=1.0).

Binary cross-entropy rewards calibrated confidence

LOSSES AND GRADIENT DESCENT

Binary cross-entropy rewards calibrated confidence

$$L = -[y \log p + (1 - y) \log(1 - p)]$$

- For y = 1, a low predicted probability incurs a high loss.
- For y = 0, a high predicted probability incurs a high loss.
- Stable implementations combine this with the sigmoid on raw logits.

◆ Worked example: y = 1 and p = 0.711 give L = −ln 0.711 ≈ 0.341.

Neural Networks / Day 1 · Neural network foundations 60

[3 min] Define the symbols: y ∈ {0, 1} is the target, p the predicted probability of the event (class 1), and log the natural logarithm, as everywhere in this course. Only one of the two terms is active for each example.

For the worked example, y = 1 and p ≈ 0.711 give L = −ln 0.711 ≈ 0.341. A confident correct prediction (p = 0.99) costs about 0.01; a confident wrong one (p = 0.01) costs about 4.6.

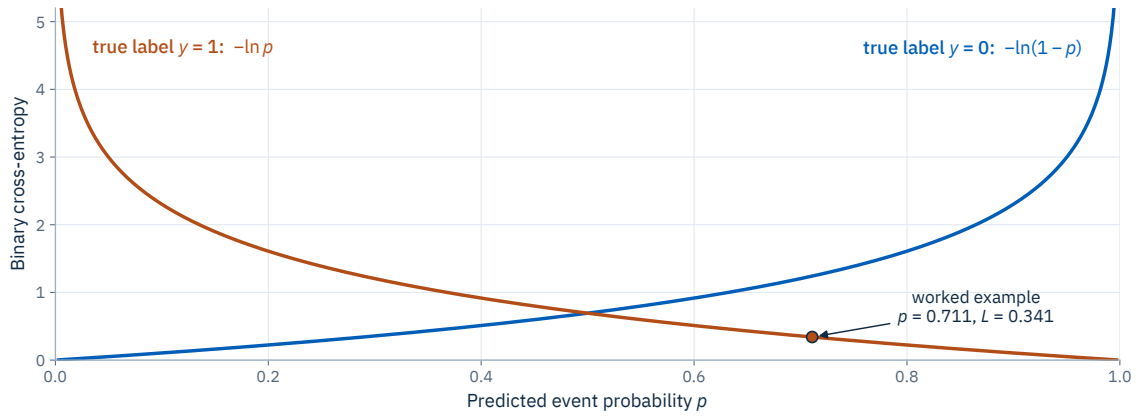
BCEWithLogitsLoss takes the logit z rather than p and computes the same quantity stably, avoiding log(0) when the sigmoid saturates. BCE does not guarantee empirical calibration after finite-data fitting, regularization or class weighting; evaluate probabilities directly when calibration matters.

Reference: https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html

Confident wrong answers cost the most

LOSSES AND GRADIENT DESCENT

Confident wrong answers cost the most



Exact binary cross-entropy for each possible label; the marked point is the worked example. Confident wrong probabilities incur much larger loss.

Neural Networks / Day 1 · Neural network foundations 61

[2 min] Read the plot, in the deck's class colours: when the true label is 1 (orange), the loss falls toward zero as the predicted probability rises toward 1, and grows without bound as it falls toward 0. The blue curve is the mirror image for label 0. The marked point is the worked example from the previous slide, p = 0.711 with y = 1, where L = 0.341.

Ask why two models with identical accuracy can have different cross-entropy. Answer: accuracy only checks which side of 0.5 each probability falls on; cross-entropy also scores how confident the model was. A model that is right at 0.51 and a model that is right at 0.99 have the same accuracy but very different losses, and one confident mistake can dominate the average.

Probabilities of exactly 0 or 1 need numerical care; stable logit-based implementations avoid taking these logarithms directly.

Softmax couples mutually exclusive classes

LOSSES AND GRADIENT DESCENT

Softmax couples mutually exclusive classes

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \qquad L = -\log p_y$$

- The probabilities are positive and sum to one across classes.
- The target index *y* selects the correct class's probability.
- Adding the same constant to every logit changes nothing.
- `CrossEntropyLoss` applies the softmax internally, stably.

◆ Libraries subtract the largest logit before exponentiating; the probabilities are unchanged.

Neural Networks / Day 1 · Neural network foundations 62

[3 min] Symbols: z_k is the logit of class k, p_k its softmax probability, and y the index of the correct class. The loss is the negative log-probability assigned to the correct class; the other classes enter only through the normalizing sum, which couples them: raising one class's probability lowers the others.

Work the simplest case: logits (0, 0, 0) give probabilities of one third each and a loss of ln 3 ≈ 1.099 whatever the target, the loss of an uninformed three-class model. Libraries subtract the maximum logit before exponentiating to avoid overflow; that is allowed precisely because adding a constant to every logit changes nothing.

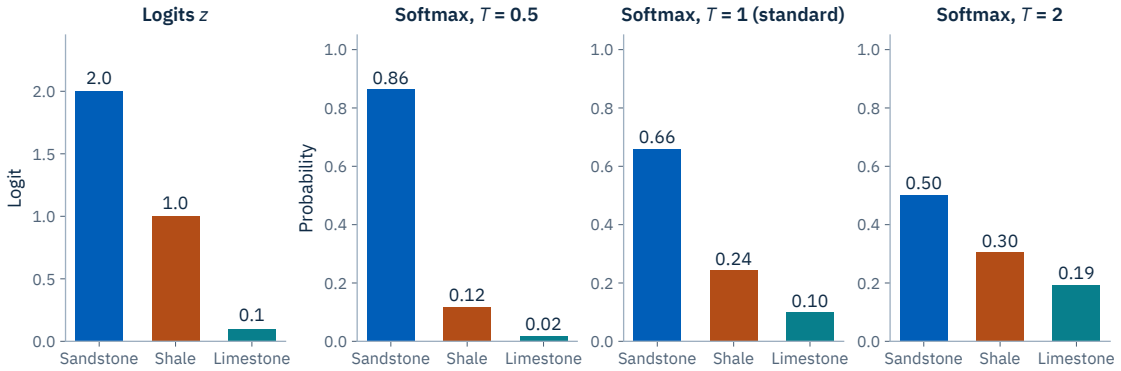
Transition: dividing the logits by a number is a different story; that is temperature.

Temperature sharpens or flattens the softmax

LOSSES AND GRADIENT DESCENT

Temperature sharpens or flattens the softmax

Divide the logits by T before the softmax: $T < 1$ sharpens, $T > 1$ flattens, and the ranking never changes.



Exact softmax of illustrative facies logits (2.0, 1.0, 0.1) at three temperatures.

Neural Networks / Day 1 · Neural network foundations 63

[2 min] Standard softmax (T = 1) turns logits (2.0, 1.0, 0.1) into probabilities 0.66, 0.24 and 0.10 for sandstone, shale and limestone. At T = 0.5 they sharpen to 0.86, 0.12 and 0.02; at T = 2 they flatten to 0.50, 0.30 and 0.19.

Temperature appears in three places you will meet later: calibration (temperature scaling fits one T on validation data after training), knowledge distillation, and sampling from generative models. Because dividing by a positive T never changes which logit is largest, the predicted class, and therefore accuracy, stays the same; only the stated confidence changes.

The facies names are illustrative labels for three logits, not a trained model.

Compute a softmax and its loss

PRACTICE

4 min Pairs

Compute a softmax and its loss

1. Logits (0, 0, 0): what are the probabilities, and the loss for any target?
2. Logits (2.0, 1.0, 0.1) with target Shale (index 1): what is the loss?
3. Add 5 to every logit: what changes?

Neural Networks / Day 1 · Neural network foundations 64

[4 min] Worked answer.

1. Each probability is 1/3; the loss is −ln(1/3) = ln 3 ≈ 1.099 for any target. 2. e^2.0 = 7.389, e^1.0 = 2.718, e^0.1 = 1.105; the sum is 11.212. Probabilities: 0.659, 0.242, 0.099. The loss for Shale is −ln 0.242 ≈ 1.417. 3. Nothing changes: every exponential is multiplied by e^5, which cancels in the ratio. This is why libraries can subtract the largest logit for numerical stability.

Common errors: applying softmax before CrossEntropyLoss (a double softmax flattens the probabilities and slows learning); passing one-hot vectors where the loss expects class indices; using log base 10 instead of the natural logarithm.

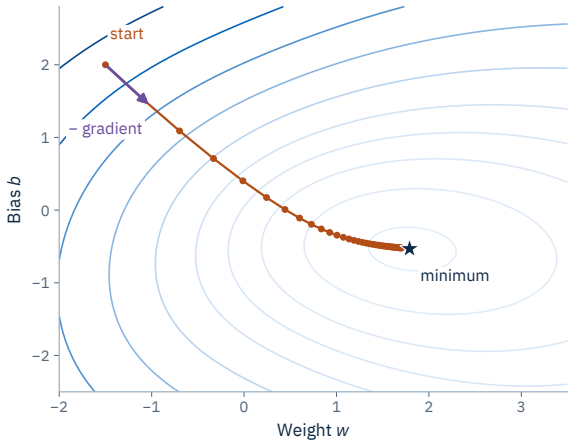
Gradient descent steps against the gradient

LOSSES AND GRADIENT DESCENT

Gradient descent steps against the gradient

$$w_{t+1} = w_t - \eta \nabla_w L(w_t)$$

- The gradient points uphill; step the other way.
- The learning rate η sets the step size.
- Steps shrink as the surface flattens near the minimum.



Computed: loss surface of a one-feature logistic model on synthetic data (seed 7); 40 full-batch steps, learning rate 1.0.

Neural Networks / Day 1 · Neural network foundations 65

[3 min] Symbols: w_t are the parameters after t steps, ∇_w L the gradient (the vector of partial derivatives of the loss), η the learning rate.

This is a real loss surface: the mean binary cross-entropy of p = σ(w·x + b) on 200 synthetic points. Think of it as gas-lock probability against one scaled pump reading, but the data are simulated. The contours are the loss as a function of the two parameters; the minimum is at w ≈ 1.79, b ≈ −0.53, with loss 0.493. Forty steps from (−1.5, 2.0) reach loss 0.4930 against the optimum 0.4928. The steps shrink near the minimum because the gradient itself shrinks there, even though η is constant.

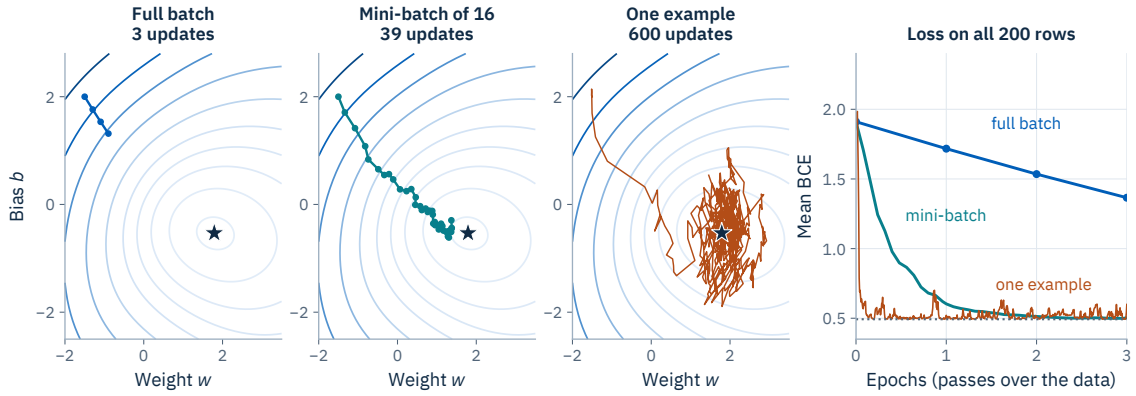
A network has thousands or millions of parameters rather than two, and its surface is not a single bowl (recall the XOR seeds), but the update rule is exactly this, applied to every parameter. Part 6 computes the gradient with backpropagation.

Batch size trades gradient noise for update count

LOSSES AND GRADIENT DESCENT

Batch size trades gradient noise for update count

Same data, start, learning rate, and 3 epochs: smaller batches make more, noisier updates.



Computed on the synthetic logistic problem: full batch (3 updates), mini-batches of 16 (39), single examples (600). Dotted line: minimum loss.

[3 min] Vocabulary. An epoch is one pass over the training data. Full-batch gradient descent uses all rows for every update. Mini-batch gradient descent estimates the gradient from a random subset, here 16 rows, so 200 rows give ⌈200/16⌉ = 13 updates per epoch and 39 in three epochs. Stochastic gradient descent in the strict sense uses one example per update; in practice "SGD" usually means mini-batches.

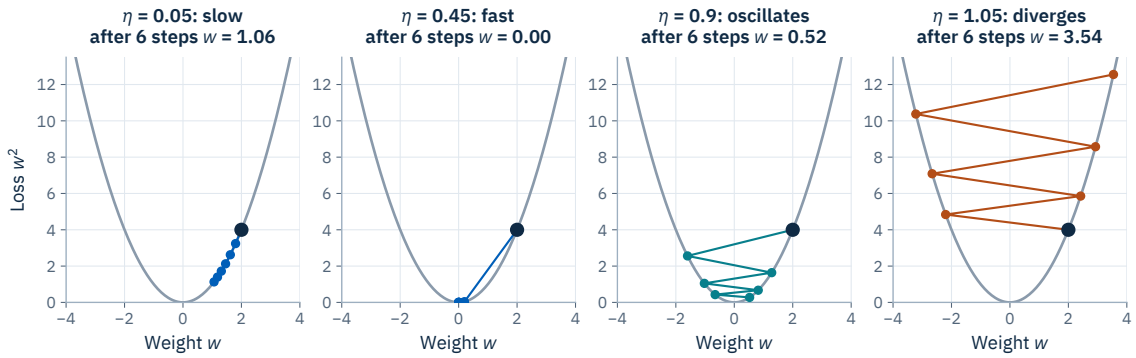
After three epochs, full batch has taken only three steps and is still far away. Mini-batches have nearly reached the minimum. Single examples got there fastest but keep jittering around it, because each update follows one noisy example.

Mini-batches are the default: they use vectorized hardware well, and their noise is tolerable or even helpful. Batch size interacts with the learning rate and with generalization; Day 2 treats that properly.

The learning rate decides whether descent converges

LOSSES AND GRADIENT DESCENT

The learning rate decides whether descent converges



Exact gradient descent on L(w) = w² from w = 2; each step multiplies w by (1 − 2η). Schedules and adaptive optimizers follow on Day 2.

Neural Networks / Day 1 · Neural network foundations 67

[3 min] For this loss the update is w ← w − η·2w = (1 − 2η)w, so the learning rate alone decides the behaviour. η = 0.05: factor 0.9, a slow creep (w = 1.06 after six steps). η = 0.45: factor 0.1, w is essentially 0 after two steps. η = 0.9: factor −0.8, the sign flips every step while shrinking slowly (0.52 after six). η = 1.05: factor −1.1, each step overshoots further and the loss grows: divergence (3.54 after six).

In a real network, a learning rate that is too large shows as a loss that spikes, oscillates or becomes NaN; too small shows as a loss that barely moves. Lab 2's experiments with learning rates show the same pattern on Sonar.

Day 2 covers learning-rate range tests, schedules, momentum and adaptive optimizers such as Adam. Today: plain gradient descent, and the habit of plotting the loss.

Match each output layer to its loss

PRACTICE

4 min Pairs

Match each output layer to its loss

For a batch of 64, give the output shape and the PyTorch loss for each task.

1. Daily gas rate of a well
2. Gas lock: yes or no
3. Facies: one of five classes
4. Five alarms that can fire together

Neural Networks / Day 1 · Neural network foundations 68

[4 min] Worked answer.

1. Gas rate: output (64, 1) with no activation; nn.MSELoss or nn.HuberLoss; float target of shape (64, 1). 2. Gas lock: output (64, 1), one logit; nn.BCEWithLogitsLoss; float 0/1 target of shape (64, 1). 3. Facies: output (64, 5), five logits; nn.CrossEntropyLoss; integer class-index target of shape (64,) with dtype long. 4. Five alarms: output (64, 5), five logits; nn.BCEWithLogitsLoss; float 0/1 target of shape (64, 5). Each alarm gets its own independent sigmoid, not a softmax.

Common errors: adding a sigmoid before BCEWithLogitsLoss or a softmax before CrossEntropyLoss; passing one-hot float targets where indices are expected; treating the alarms as five mutually exclusive classes. Close Part 5 at 14:45 and take the break.

Backpropagation



[1 min] Part 6 runs 15:00–15:30. Gradient descent needs the gradient of the loss with respect to every weight. Backpropagation computes all of them in one backward sweep through the network, reusing the values stored during the forward pass.

We work entirely on the two-hidden-unit network from Part 4, so every number can be checked by hand, then verify the result numerically. In Part 7, PyTorch's autograd reproduces the same numbers, closing the loop between hand calculation and framework.

Backpropagation applies the chain rule

BACKPROPAGATION

Backpropagation applies the chain rule

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z}\,\frac{\partial z}{\partial w}$$

- Forward pass: compute and store every intermediate value.
- Backward pass: propagate sensitivities from the loss to the parameters.
- An optimizer then uses the gradients to update the parameters.

◆ For $z = v \cdot h + c$, the local derivative $\partial z / \partial v_1$ is simply h_1, a value stored by the forward pass.

Neural Networks / Day 1 · Neural network foundations 70

[3 min] Notation for the worked network from Part 4: h = (h₁, h₂) are the hidden activations, v = (0.4, −0.2) the output weights, c = 0.1 the output bias and z the logit. The chain rule says a change in w affects L only through z, so the sensitivities multiply. For an output weight, z = v·h + c, so ∂z/∂v₁ = h₁: the gradient of an incoming weight is the upstream gradient times the value that the weight multiplied in the forward pass. That is why the forward pass must store its intermediate values.

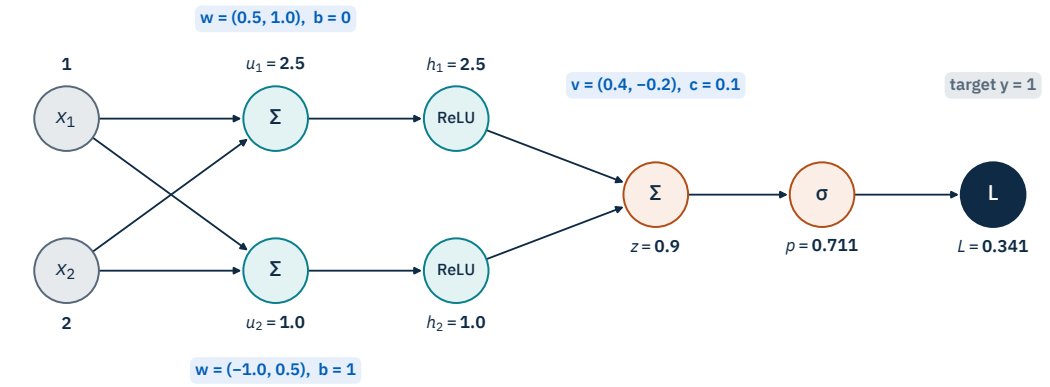
Deeper weights repeat the pattern: each layer multiplies the incoming gradient by its own local derivative and passes the result further back.

Backpropagation computes gradients; it is not itself the parameter-update algorithm. Automatic differentiation follows the computational graph rather than estimating each derivative with finite differences. Finite differences remain a useful small-scale check, later in this part.

A computational graph records the forward pass

BACKPROPAGATION

A computational graph records the forward pass



Exact values for the worked network with x = (1, 2) and target y = 1. Σ: weighted sum; σ: sigmoid; L: binary cross-entropy.

Neural Networks / Day 1 · Neural network foundations 71

[3 min] The same network as the forward-pass slide, drawn as a graph of operations. Each Σ node computes a weighted sum with its own parameters (blue chips); the hidden sums are called u_1 and u_2 (the pre-activations), so that z can keep its meaning as the logit; each ReLU node applies the activation; σ turns the logit into a probability; L compares it with the target.

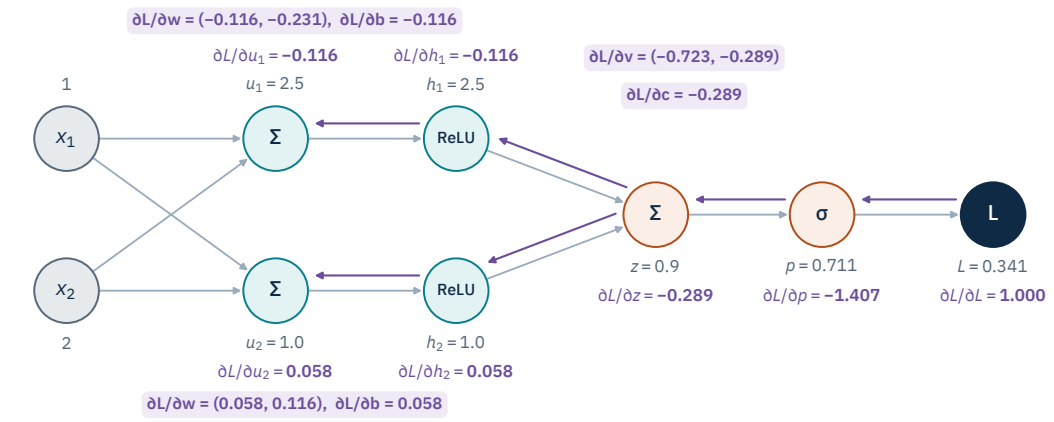
Forward pass, left to right: $u_1 = 2.5$ and $u_2 = 1.0$; both ReLUs pass their inputs, so $h_1 = 2.5$ and $h_2 = 1.0$; $z = 0.4 \cdot 2.5 - 0.2 \cdot 1.0 + 0.1 = 0.9$; $p = \sigma(0.9) = 0.711$; $L = -\ln 0.711 = 0.341$.

Every node keeps its output because the backward pass needs it: $\partial z / \partial v_1 = h_1$, for example. This storage is why training needs more memory than prediction, and why frameworks let you switch it off with torch.no_grad() when you only predict.

Gradients flow backward through the same graph

BACKPROPAGATION

Gradients flow backward through the same graph



Exact reverse-mode gradients. Each purple value is the upstream gradient times the node's local derivative.

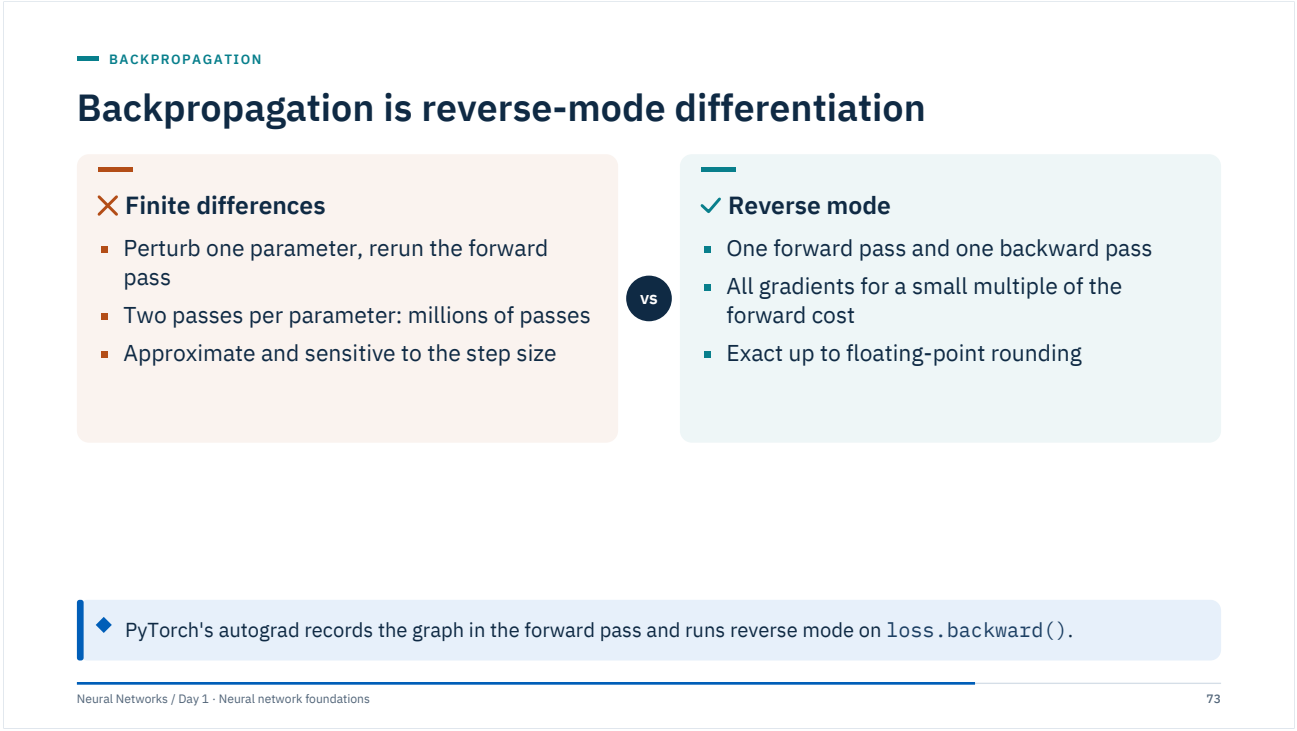
Neural Networks / Day 1 · Neural network foundations 72

[5 min] Walk right to left with the room. ∂L/∂L = 1. ∂L/∂p = −1/p = −1.407. The sigmoid's local derivative is p(1 − p) = 0.206, so ∂L/∂z = −1.407 × 0.206 = −0.289, which equals p − y: the famous simplification for sigmoid plus cross-entropy.

At the output Σ node: ∂L/∂v = ∂L/∂z × h = (−0.723, −0.289) and ∂L/∂c = −0.289. Going further back, $\partial L/\partial h_1 = -0.289 \times 0.4 = -0.116$ and $\partial L/\partial h_2 = -0.289 \times (-0.2) = 0.058$. ReLU passes the gradient unchanged where it was active (both units here), so ∂L/∂u = (−0.116, 0.058).

Finally ∂L/∂w = ∂L/∂u × x: row 1 is (−0.116, −0.231) and row 2 is (0.058, 0.116), and each bias gradient equals its ∂L/∂u. Negative gradients mean "increase this weight to reduce the loss".

Backpropagation is reverse-mode differentiation



[3 min] Why backpropagation made neural networks practical. A network has many inputs to differentiate with respect to (its parameters) and one output (the scalar loss). Reverse mode is efficient exactly in that case: one backward sweep gives every partial derivative, at a cost that is a small constant multiple of the forward pass.

Finite differences would perturb each parameter separately: a central difference needs two extra forward passes per parameter, which is hopeless for a network with millions of parameters, and the result is only an approximation whose accuracy depends on the step size.

Forward-mode differentiation also exists; it is efficient when there are few inputs and many outputs, the opposite of training. Reverse mode's price is memory: every intermediate value must be kept until the backward pass.

One backward calculation updates one weight

BACKPROPAGATION

One backward calculation updates one weight

$$\frac{\partial L}{\partial z} = p - y$$

- In the worked example, p = 0.711 and y = 1, so ∂L/∂z = −0.289.
- The output weight on h_1 has gradient −0.289 × 2.5 = −0.723.
- A learning rate of 0.1 moves that weight from 0.4 to about 0.472.

◆ Sigmoid with cross-entropy, like softmax with cross-entropy, gives "prediction minus target".

Neural Networks / Day 1 · Neural network foundations 74

[2 min] This derivative, p − y, holds for sigmoid plus binary cross-entropy; the same form appears for softmax plus cross-entropy. It is one reason these pairs are standard: the gradient is simply "prediction minus target". Mind the sign: this is the negative of the residual y − ŷ (observed minus predicted) that the course uses for regression errors.

The update: $v_1 \leftarrow v_1 - \eta \cdot \partial L/\partial v_1 = 0.4 - 0.1 \times (-0.723) = 0.472$. Ask why the weight increases. Answer: the target is 1 and the prediction 0.711 is too low; raising v_1 raises the logit (because $h_1 = 2.5 > 0$), which raises p toward 1.

Round only for exposition; in code keep full precision. The hidden-layer weights also move in the same step; the next exercise computes two of them.

Push the gradient one layer further

PRACTICE

5 min Pairs

Push the gradient one layer further

Worked network, $x = (1, 2)$, $y = 1$, and $p - y = -0.289$.

1. Find $\partial L/\partial w$ for the weight from x_2 to h_1 (value 1.0).
2. Update it with learning rate 0.1.
3. Repeat both steps for the weight from x_2 to h_2 (value 0.5).

Neural Networks / Day 1 · Neural network foundations 75

[5 min] Worked answer.

1. $\partial L/\partial w = (p - y) \times v_1 \times \text{ReLU}'(u_1) \times x_2 = -0.289 \times 0.4 \times 1 \times 2 = -0.231$. 2. New value $1.0 - 0.1 \times (-0.231) = 1.023$. 3. For the weight from x_2 to h_2: $-0.289 \times (-0.2) \times 1 \times 2 = 0.116$, so the new value is $0.5 - 0.1 \times 0.116 = 0.488$.

Interpretation: raising the first weight raises h_1, which raises z because v_1 is positive, which moves p toward the target, so the gradient is negative and the weight increases. The output weight on h_2 is negative, so raising h_2 would lower p; its incoming weight therefore decreases.

Common errors: dropping the output weight v from the chain; forgetting the ReLU derivative (1 here, 0 if the unit were inactive, as in the x = (3, 1) exercise); using x_1 instead of x_2.

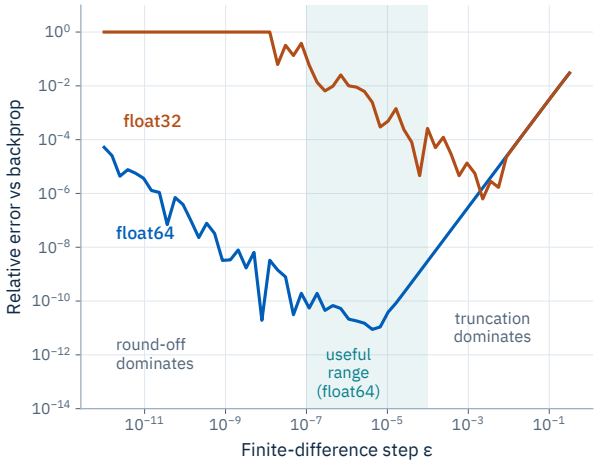
A numerical gradient check catches silent errors

BACKPROPAGATION

A numerical gradient check catches silent errors

$$g_{\text{num}} = \frac{L(w+\epsilon) - L(w-\epsilon)}{2\epsilon}$$

- Compare with backpropagation using a relative error.
- Use float64 and ε around 10^{-5} to 10^{-6}.
- Too large ε: truncation error; too small: round-off.
- Avoid kinks, such as ReLU exactly at zero.



Computed for ∂L/∂v₁ of the worked network: relative error against the step ε.

Neural Networks / Day 1 · Neural network foundations 76

[3 min] The central difference estimates the slope from two nearby loss values. Compare it with the backpropagated value using the relative error |g_num – g_bp| / max(|g_num|, |g_bp|).

The plot shows the trade-off. With large ε the estimate suffers from truncation error (the curve is not a straight line over that distance); with tiny ε the two losses are nearly equal and subtracting them loses precision (round-off). In float64 the best error here is about 1e-11 near ε ≈ 4e-6. In float32 the error never gets much below 1e-6 (best about 6e-7), and below ε ≈ 1e-8 the check fails completely because w ± ε rounds back to w.

Use a small deterministic network in double precision, switch off dropout and other randomness, and avoid points where the function has a kink. As a rough guide, float64 relative errors of 1e-7 or smaller usually indicate a correct backward pass, while errors near 1 indicate a bug. Do not run this on every weight of a large model: check a few entries per parameter array.

A gradient check for the Lab 2 network

BACKPROPAGATION

A gradient check for the Lab 2 network

```python
def grad_check(loss_fn, params, grads, key, idx, eps=1e-5):
    old = params[key][idx]
    params[key][idx] = old + eps
    up = loss_fn(params)
    params[key][idx] = old - eps
    down = loss_fn(params)
    params[key][idx] = old                          # restore the weight
    num, ana = (up - down) / (2 * eps), grads["d" + key][idx]
    return abs(num - ana) / max(abs(num), abs(ana), 1e-12)

loss_fn = lambda p: binary_cross_entropy(y_train, forward(X_train, p)["a2"])
grads = backward(X_train, y_train, params, forward(X_train, params))
for key, idx in [("W1", (0, 0)), ("b1", (0, 3)), ("W2", (5, 0))]:
    print(key, idx, f"{grad_check(loss_fn, params, grads, key, idx):.1e}")
```

OUTPUT

```
W1 (0, 0) 3.8e-10
b1 (0, 3) 1.1e-08
W2 (5, 0) 2.5e-11
```

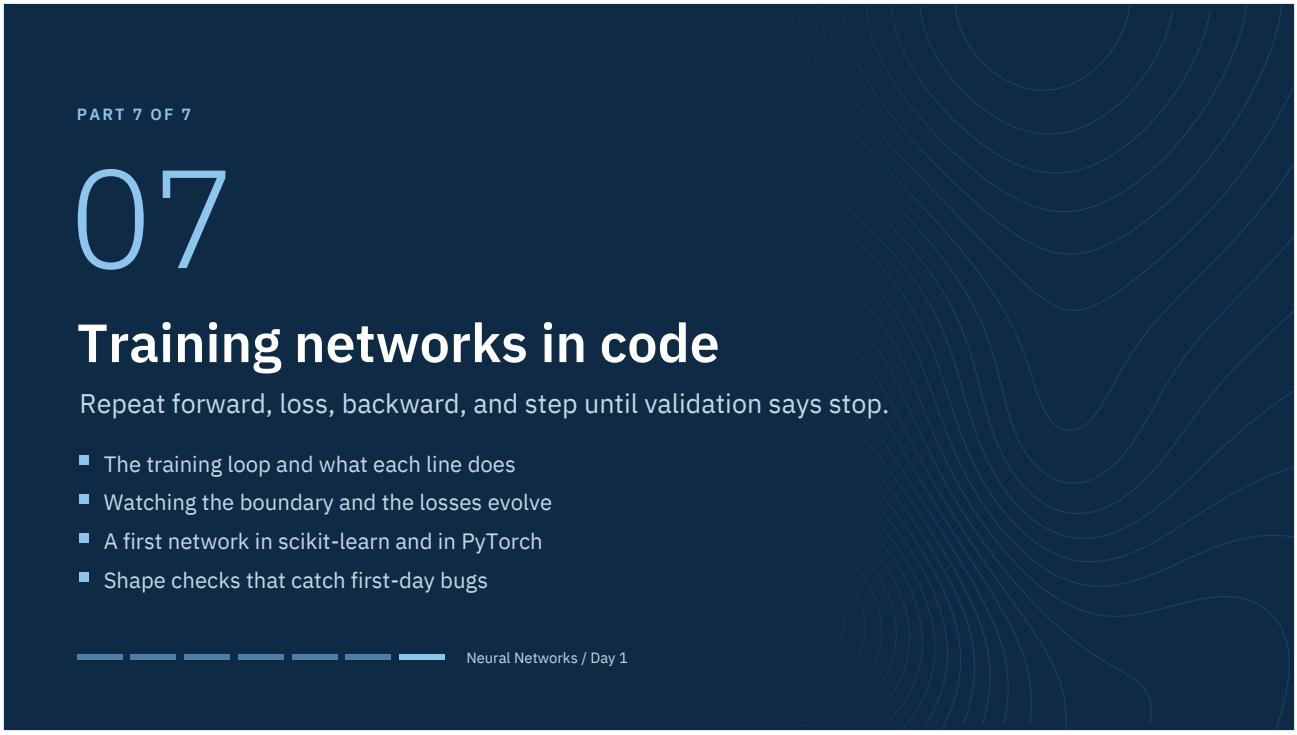
Neural Networks / Day 1 · Neural network foundations 77

[3 min] This is the check learners add to Lab 2, in the cell after the backward pass in section 7. It uses the notebook's own functions (forward, backward and binary_cross_entropy) and parameter dictionary, whose gradient keys carry a "d" prefix (dW1 for W1). It perturbs one entry, recomputes the full training loss twice, restores the entry, and compares the slope with the backpropagated gradient.

The output was computed on the Lab 2 training rows at initialization (np.random.seed(42), 32 hidden units): relative errors between 1e-11 and 1e-8, so the backward pass is right. As a demonstration, a backward pass that forgets the sigmoid derivative in the hidden layer gives a relative error of 0.81 for the same W1 entry: the kind of bug that still lets training "work", just badly.

Common errors: forgetting to restore the parameter (every later check is then wrong); comparing against gradients computed before a parameter update; using float32 arrays.

Training networks in code



[1 min] Part 7 runs 15:30–15:55 and leads straight into Lab 2. Everything so far comes together in a few lines of code: a loop that repeats the forward pass, the loss, the backward pass and the parameter update over mini-batches and epochs.

We then write the same kind of network three ways: by hand in NumPy (Lab 2), with scikit-learn's MLPClassifier in a pipeline, and as a PyTorch nn.Module whose autograd reproduces the hand-computed gradients from Part 6. The part closes with shape checks, the cheapest defence against silent bugs.

The training loop has four responsibilities

TRAINING NETWORKS IN CODE

The training loop has four responsibilities

```python
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()       # 1 clear
        logits = model(X_batch)     # 2 forward
        loss = criterion(logits, y_batch)
        loss.backward()             # 3 gradients
        optimizer.step()            # 4 update
    model.eval()
    with torch.no_grad():
        val_loss = criterion(model(X_val), y_val)
```

- Clear old gradients: PyTorch accumulates them.
- Forward pass and loss on one mini-batch.
- `backward()` fills every parameter's `.grad`.
- `step()` applies the update rule.
- Validate without building a graph.

Neural Networks / Day 1 · Neural network foundations 79

[3 min] This is the conceptual PyTorch loop; model, train_loader, criterion and optimizer must already exist. A loader can be built with DataLoader(TensorDataset(X_t, y_t), batch_size=32, shuffle=True).

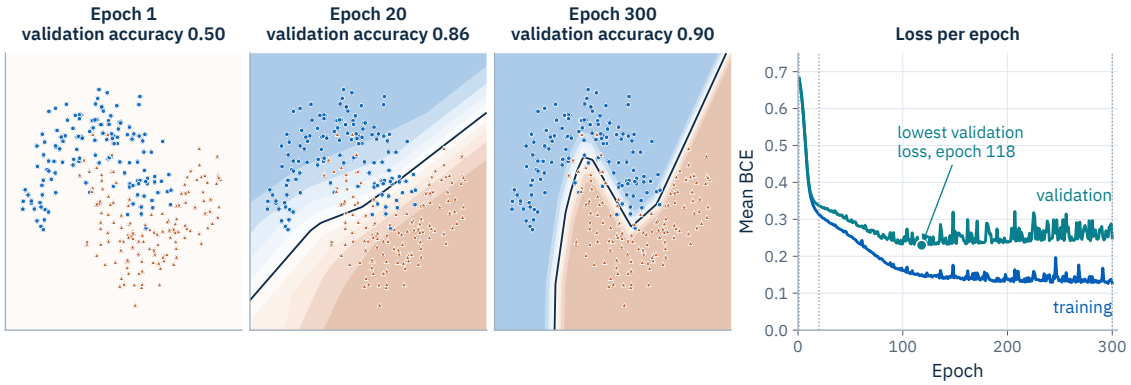
Walk the four responsibilities. zero_grad clears gradients because PyTorch adds new gradients to old ones unless told otherwise; forgetting it is a classic bug that makes updates grow over time. The forward pass produces logits, and the criterion compares them with the targets. backward runs reverse mode and stores ðloss/ðparameter in each parameter's .grad. step applies the update, for plain SGD exactly w ← w – η·grad.

model.train() and model.eval() switch layers such as dropout and batch normalization (Day 2); in today's networks they change nothing but are good habits. torch.no_grad() skips graph building during validation. Match target and output shapes explicitly to avoid accidental broadcasting.

The decision boundary takes shape during training

TRAINING NETWORKS IN CODE

The decision boundary takes shape during training



Computed: synthetic moons (seed 1); PyTorch 2–16–16–1 ReLU network, SGD with learning rate 0.1 and batch 32. Dotted lines: the three snapshot epochs. Validation loss is lowest at epoch 118.

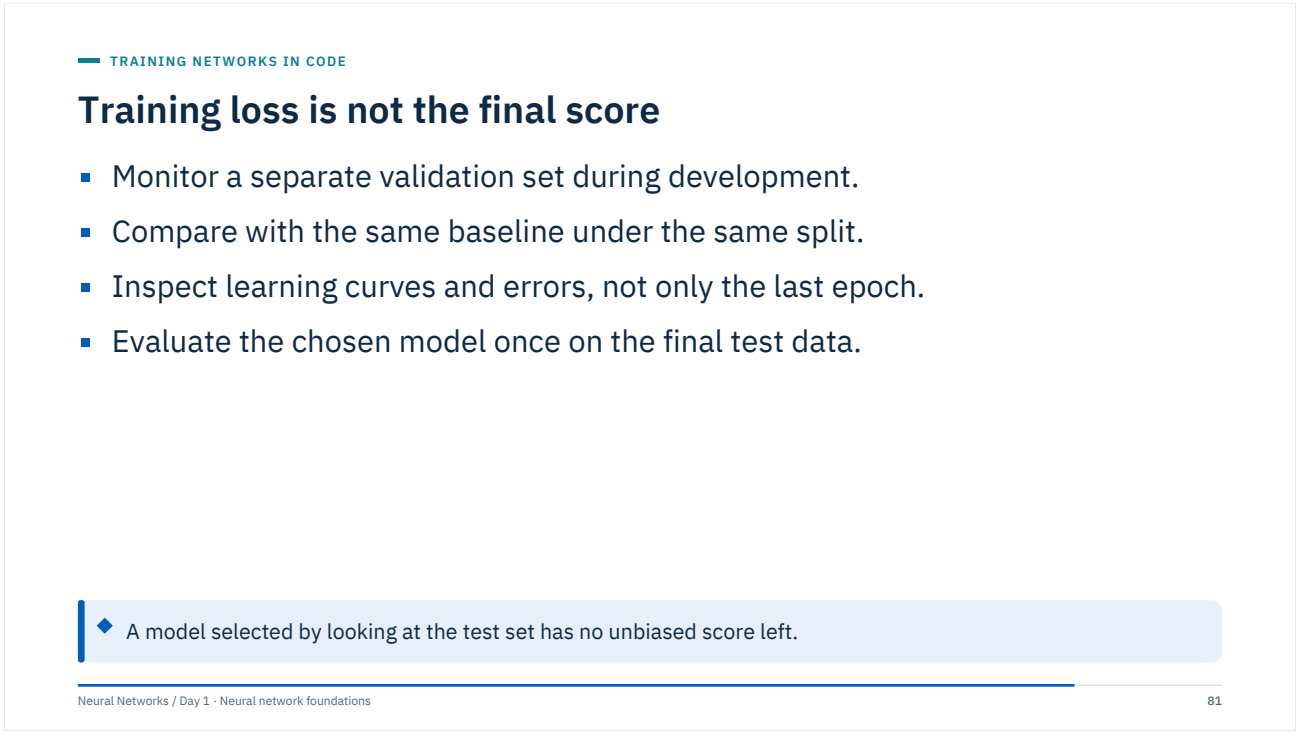
Neural Networks / Day 1 · Neural network foundations 80

[3 min] At epoch 1 the network predicts about 0.5 everywhere: the random initial weights have not yet formed a boundary, and validation accuracy is 0.50. By epoch 20 a bent line separates most points (0.86). By epoch 300 the boundary follows the two moons (0.90).

The right panel is the habit to build: plot training and validation loss every epoch. Training loss keeps falling, to 0.127 at epoch 300, because that is what the optimizer minimizes. Validation loss bottoms out at epoch 118 (0.230) and then creeps up to 0.250, the first sign of overfitting. The epoch-to-epoch jitter comes from mini-batch noise.

Day 2 turns this into a method with early stopping and checkpointing. Transition: why the training loss can never be the final score.

Training loss is not the final score



[2 min] The training loss measures how well the model fits the rows it was optimized on; it falls by design, as the previous slide showed. Validation data support early stopping, architecture choices and thresholds. The baseline must be evaluated on the same folds or split, otherwise the comparison mixes model differences with split differences.

Repeatedly tuning on the final test set converts it into another validation set, which is why Lab 1 chose and tuned its model with cross-validation and touched its 42 test rows once. Keep a record of every decision made with validation data, and touch the test set once.

Day 2 adds checkpointing, early stopping and reproducible training details. Ask: in Lab 2, which data will you use to choose the learning rate? Answer: a validation split or CV folds taken from the training rows, not the 42 test echoes.

A first network in scikit-learn

TRAINING NETWORKS IN CODE

A first network in scikit-learn

```
from sklearn.neural_network import MLPClassifier

mlp = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(32,), alpha=1e-3,
                  max_iter=2000, random_state=0),
)
scores = cross_val_score(mlp, X_train, y_train, cv=cv)
print(f"CV accuracy {scores.mean():.3f} +/- {scores.std():.3f}")
```

OUTPUT

```
CV accuracy 0.843 +/- 0.052
```

Same folds as the logistic baseline (0.753 ± 0.061): compare fold by fold before celebrating.

Neural Networks / Day 1 · Neural network foundations 82

[3 min] This reuses the imports, the training rows and the cv object from the baseline slide in Part 2, so the five folds are identical. MLPClassifier's defaults are ReLU hidden units and the Adam optimizer; alpha is an L2 penalty on the weights (Day 2 calls it weight decay). Putting the scaler inside the pipeline matters even more for networks than for logistic regression, because unscaled inputs slow gradient descent badly.

The MLP averages 0.843 against 0.753 for the baseline. Fold by fold it is higher in four folds and tied in one. That is encouraging but modest evidence: five folds of about 33 echoes, and the random forest and SVM from Lab 1 both scored 0.831 on the same folds. The network is competitive with good baselines, not clearly better.

MLPClassifier and MLPRegressor fit into pipelines, cross-validation and grid search exactly like any other estimator, which makes them the fastest route to a fair first comparison.

A first network in PyTorch

TRAINING NETWORKS IN CODE

A first network in PyTorch

```
import torch
from torch import nn

class SonarNet(nn.Module):
    def __init__(self, n_in=60, n_hidden=32):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(n_in, n_hidden), nn.ReLU(),
            nn.Linear(n_hidden, 1))              # one raw logit

    def forward(self, x):                        # x: (B, 60) float32
        return self.layers(x)                    # (B, 1) logits

model = SonarNet()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
print(sum(p.numel() for p in model.parameters()))
```

OUTPUT

```
1985
```

Neural Networks / Day 1 · Neural network foundations 83

[3 min] An nn.Module declares its layers in __init__ and describes the forward pass in forward; PyTorch derives the backward pass automatically. nn.Sequential chains layers in order. nn.Linear(60, 32) owns a 32 × 60 weight matrix and 32 biases.

The model returns one raw logit per echo, with no sigmoid, because the loss will be BCEWithLogitsLoss. Apply torch.sigmoid only when you need probabilities for reporting or thresholds.

Count the parameters with the room: 60 × 32 + 32 = 1,952 in the first layer and 32 × 1 + 1 = 33 in the second, 1,985 in total, as printed. model.parameters() hands every learnable tensor to the optimizer; plain SGD with learning rate 0.1 is today's gradient descent, and Day 2 introduces Adam and schedules.

Trained with the loop from this part (standardized inputs, batch 32, 100 epochs, seed 0), this network reached 0.881 accuracy on the 42 test echoes (37 of 42): the same uncertainty caveats apply.

Autograd reproduces the gradients computed by hand

TRAINING NETWORKS IN CODE

Autograd reproduces the gradients computed by hand

```python
import torch

x = torch.tensor([1.0, 2.0])
W1 = torch.tensor([[0.5, 1.0], [-1.0, 0.5]], requires_grad=True)
b1 = torch.tensor([0.0, 1.0], requires_grad=True)
v = torch.tensor([0.4, -0.2], requires_grad=True)
c = torch.tensor(0.1, requires_grad=True)
h = torch.relu(W1 @ x + b1)                  # (2.5, 1.0)
z = v @ h + c                                # logit 0.9
loss = torch.nn.functional.binary_cross_entropy_with_logits(
    z, torch.tensor(1.0))
loss.backward()
print(round(loss.item(), 4), v.grad, W1.grad, sep="\n")
```

OUTPUT

```
0.3412
tensor([-0.7226, -0.2891])
tensor([[-0.1156, -0.2312],
        [ 0.0578,  0.1156]])
```

Neural Networks / Day 1 · Neural network foundations 84

[3 min] The worked network from Parts 4 and 6, written with tensors that require gradients. W1 holds one row per hidden unit, the column-vector form of Part 4 and the layout nn.Linear uses for its weight, so the forward pass is W1 @ x. The forward pass builds a graph as it runs; loss.backward() runs reverse mode through it and fills the .grad field of every leaf tensor that requires gradients.

Compare with the backward-pass slide: the loss 0.3412, ∂L/∂v = (−0.723, −0.289) and ∂L/∂W1 = ((−0.116, −0.231), (0.058, 0.116)) match to rounding. b1.grad would print (−0.116, 0.058) and c.grad −0.289.

binary_cross_entropy_with_logits takes the logit, not the probability, exactly as BCEWithLogitsLoss does. This is the strongest argument for doing one example by hand: once the numbers agree, you trust the framework for the millions of gradients you will never check by hand.

Shape checks turn silent bugs into loud ones

TRAINING NETWORKS IN CODE

Shape checks turn silent bugs into loud ones

```
X_t = torch.as_tensor(X_train, dtype=torch.float32)   # (166, 60)
y_t = torch.as_tensor(y_train, dtype=torch.float32)   # (166,)
criterion = nn.BCEWithLogitsLoss()
logits = model(X_t)                                   # (166, 1)
assert X_t.shape[1] == 60 and torch.isfinite(X_t).all()
print(logits.shape, y_t.shape)
loss = criterion(logits, y_t)          # raises: the shapes differ
```

OUTPUT

```
torch.Size([166, 1]) torch.Size([166])
ValueError: Target size (torch.Size([166])) must be the same as
input size (torch.Size([166, 1]))
```

Fix with `y_t = y_t.reshape(-1, 1)`. `MSELoss` would only warn, then broadcast to (166, 166).

[3 min] X_train and y_train come from the Lab 1 split: float64 features and an int64 label vector. torch.as_tensor converts both to float32, the dtype PyTorch layers expect; the assert documents the feature count and rejects NaN or infinite values before they poison a gradient.

The printed shapes reveal the classic mismatch from Part 2: logits (166, 1) against targets (166,). BCEWithLogitsLoss refuses and raises the error shown (wrapped here onto two lines). That is the good case. MSELoss, used for regression, accepts the same mismatch with only a warning and silently computes a (166, 166) grid of differences, the broadcasting bug from Part 2.

Habits: print shapes once, assert them in the training code, reshape targets explicitly, and read warnings rather than silencing them. Two minutes of checks here saves an afternoon of debugging a model that "trains" without learning.

Choose scikit-learn for speed, PyTorch for control

TRAINING NETWORKS IN CODE

Choose scikit-learn for speed, PyTorch for control

Aspect	scikit-learn MLPClassifier	PyTorch
Architectures	Dense layers only	Any differentiable model
Training loop	Built into `fit`	You write it
Losses and outputs	Fixed by the estimator	You choose and check them
Hardware	CPU	CPU or GPU
Best use	Quick tabular baseline	Custom models, Days 2 to 5

◆ Lab 2 builds the same network a third way, by hand in NumPy, to see every step.

Neural Networks / Day 1 · Neural network foundations 86

[2 min] scikit-learn's MLPClassifier and MLPRegressor are ideal for a first comparison on tabular data: they fit into pipelines, cross-validation and grid search like any other estimator, and they hide the training loop. The price is control: dense layers only, a fixed loss for each estimator, CPU training, and limited options for custom inputs such as images or sequences.

PyTorch makes you write the loop, choose the loss and check the shapes, and in exchange supports any differentiable architecture, custom losses, GPUs and pretrained models. Days 2 to 5 use it throughout; Day 2 also shows the Keras equivalent, which sits between the two in convenience.

Transition: Lab 2 builds the same network a third way, by hand in NumPy, so that every step you just saw in PyTorch is visible.

Lab 2: a NumPy network you can explain

LABORATORY

45 min

Lab 2: a NumPy network you can explain

- ☐ Implement sigmoid, initialization, forward pass, BCE, and backward pass.
- ☐ Assert that every gradient has the shape of its parameter.
- ☐ Add the gradient check and report three relative errors.
- ☐ Train, plot the loss, and evaluate once on the test rows.
- ☐ Compare with the Lab 1 baseline; sections 12–13 are extensions.

NOTEBOOK

`exercises/02_nn_from_scratch/NN_From_Scratch_Student.ipynb`

EVIDENCE TO SUBMIT

- Shapes of W1, b1, W2, b2 and of their gradients
- Gradient-check relative errors in float64
- Loss curve, and train and test accuracy against the baseline

Neural Networks / Day 1 · Neural network foundations 87

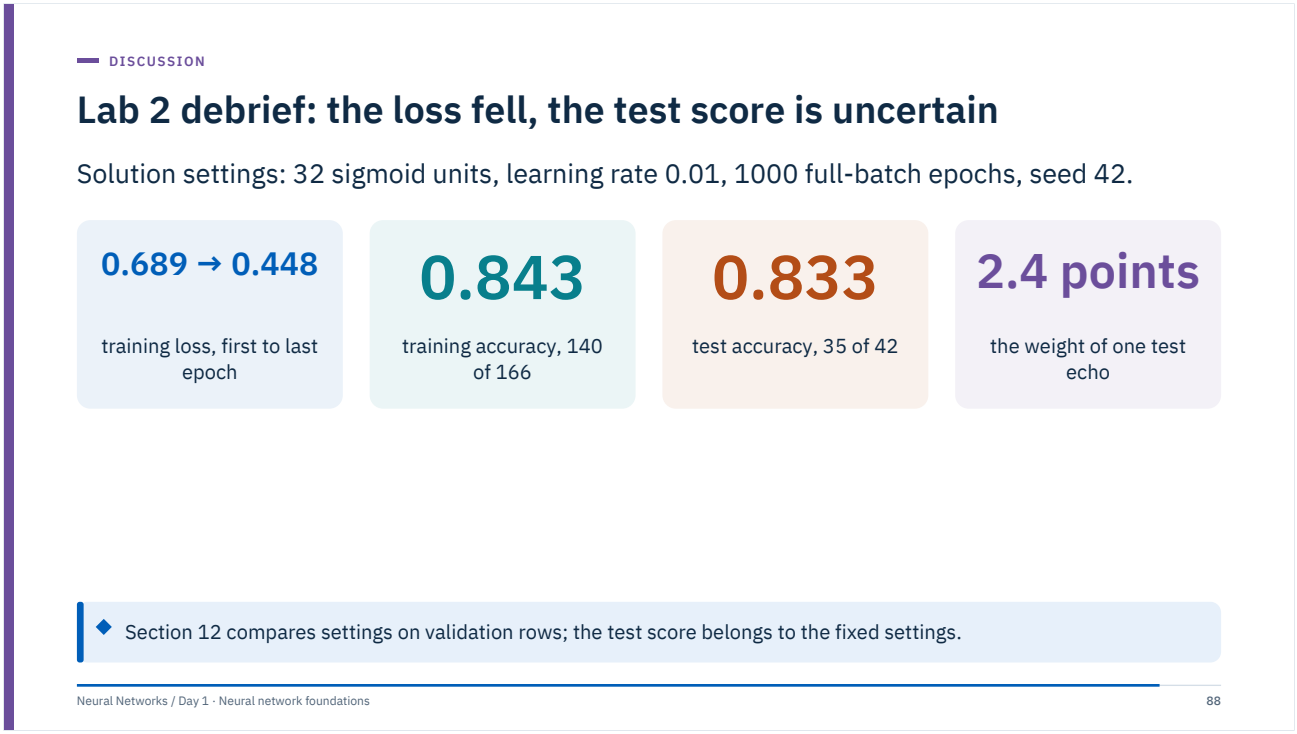
[45 min lab, then 10 min debrief] Suggested timing: 5 minutes to load and standardize the data (section 2); 15 minutes for sigmoid, initialization, forward pass and BCE (sections 3–6); 10 minutes for the backward pass, shape assertions and the gradient check from Part 6 (section 7 and the gradient-check cell that follows it); 10 minutes to train, plot and evaluate (sections 8–11); 5 minutes for the write-up. Sections 12 (hyperparameter experiments) and 13 (MLPClassifier comparison) are extensions for fast finishers.

The notebook's network has 32 sigmoid hidden units, a sigmoid output and full-batch gradient descent (learning rate 0.01, 1000 epochs, seed 42). Its backward pass uses dz2 = a2 − y, which is the p − y from Part 6.

Common problems: reshaping y to (m, 1) is required, otherwise a2 − y broadcasts to (m, m); forgetting to re-seed before training changes the results; the gradient check must use float64 arrays and restore each perturbed weight; sigmoid_derivative takes the activation a, not z. Both Day 1 notebooks map Mine = 1 and Rock = 0 and make the same split, so the 42 test echoes are the ones Lab 1 held out.

Protocol: the section 9 settings are fixed in advance, so section 11 is the notebook's single test evaluation. Section 12 compares hidden size, learning rate and epochs on a validation split carved from the training rows (124 training and 42 validation rows, with the scaler refitted on the 124), and section 13 compares with MLPClassifier on the same validation rows; neither touches the test set. A setting adopted from section 12 is reported with its validation score, not re-tested on the same test echoes. Expected numbers are in the debrief notes.

Lab 2 debrief: the loss fell, the test score is uncertain

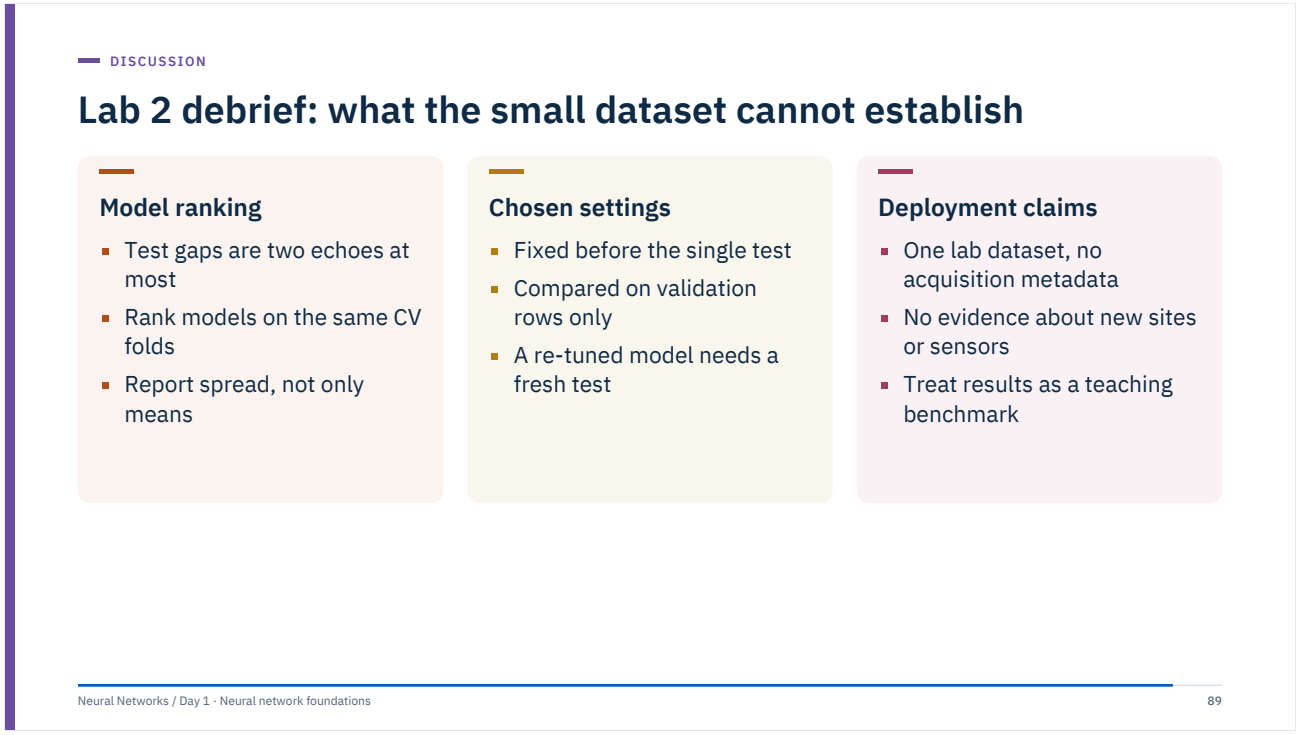


[5 min] With the solution settings the training loss falls from 0.689, close to ln 2 ≈ 0.693 for an uninformed binary model, to 0.448 after 1000 epochs, and it is still falling: learning rate 0.01 with full-batch gradient descent is slow. Training accuracy is 0.843 (140 of 166) and test accuracy 0.833 (35 of 42), on the same 42 test echoes as Lab 1. Each test echo moves accuracy by 2.4 points.

The notebook's validation experiments (124 training and 42 validation rows) illustrate the learning-rate slide: with learning rate 0.001 the training loss barely moves (0.649), with 0.05 it falls to 0.221, and with 0.1 it reaches 0.105 while the validation loss rises from 0.43 to 0.66, the overfitting signature. Validation accuracy stays between 0.69 and 0.79 across the learning rates. The best single validation score, 0.881 for 16 hidden units, is one of many noisy scores on 42 rows and therefore optimistic, like a grid-search maximum; that is why a setting chosen there is not re-tested on the test echoes.

Collect the gradient-check results: relative errors of 1e-7 or smaller mean the backward pass is right. Anyone with errors near 1 has a bug, usually a missing sigmoid derivative or a transposed matrix.

Lab 2 debrief: what the small dataset cannot establish



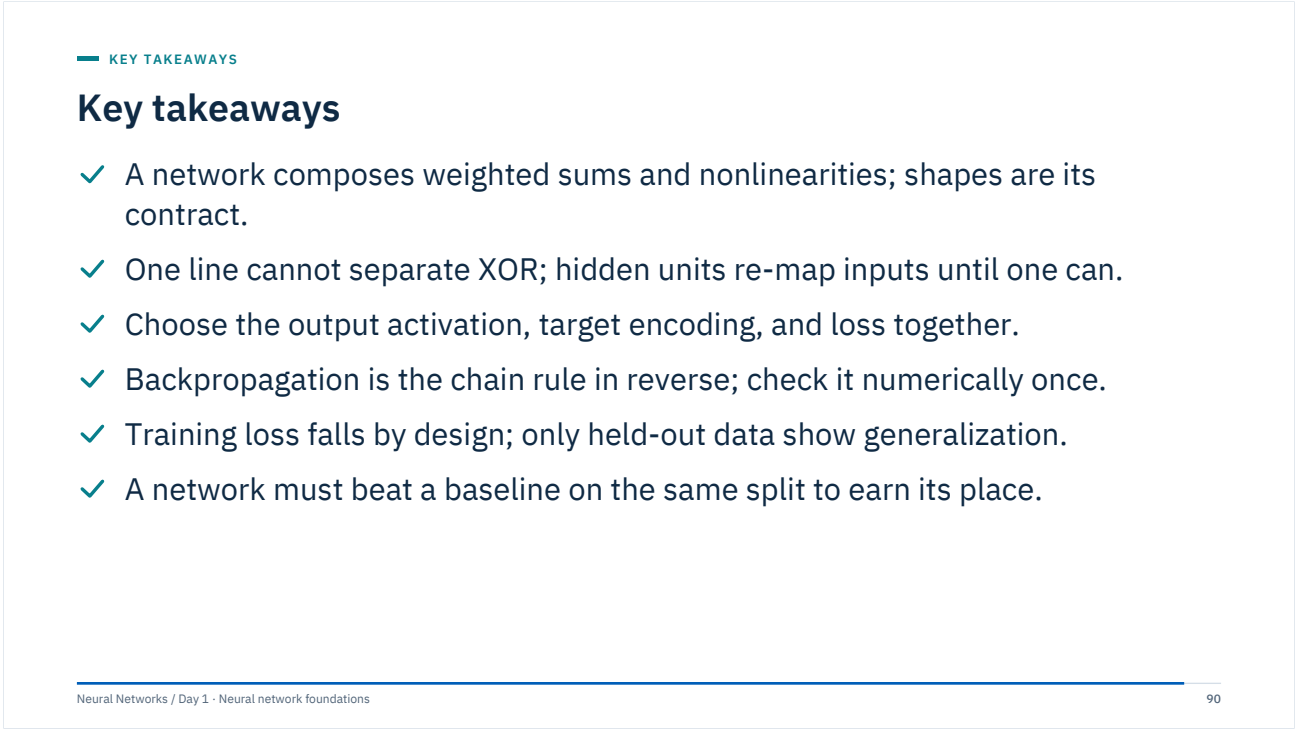
[5 min] Close the lab with what the evidence does and does not support.

Model ranking: on the same 42 test echoes the NumPy network scores 0.833 (35 of 42) and Lab 1's tuned random forest 0.881 (37 of 42), two echoes apart. On the shared CV folds the logistic baseline scored 0.753, the random forest and the SVM 0.831 each, and the scikit-learn MLP from Part 7 0.843. The honest summary is "competitive with good baselines", not "the network wins".

Why the scikit-learn MLP in section 13 fits the training rows much better than the NumPy network with seemingly the same settings (training accuracy 0.99 against 0.84): its sgd solver uses momentum (0.9, Nesterov) by default, so it travels further in 1000 iterations, and it still warns that it has not converged. It does not score higher on the validation rows (0.76 against 0.79): a closer fit to 124 rows is not better generalization. Day 2 explains momentum.

Deployment: the Sonar file has no dates, sites or acquisition angles, so we cannot test grouped or time-ordered generalization. Treat all of today's scores as a teaching benchmark, never as a field capability.

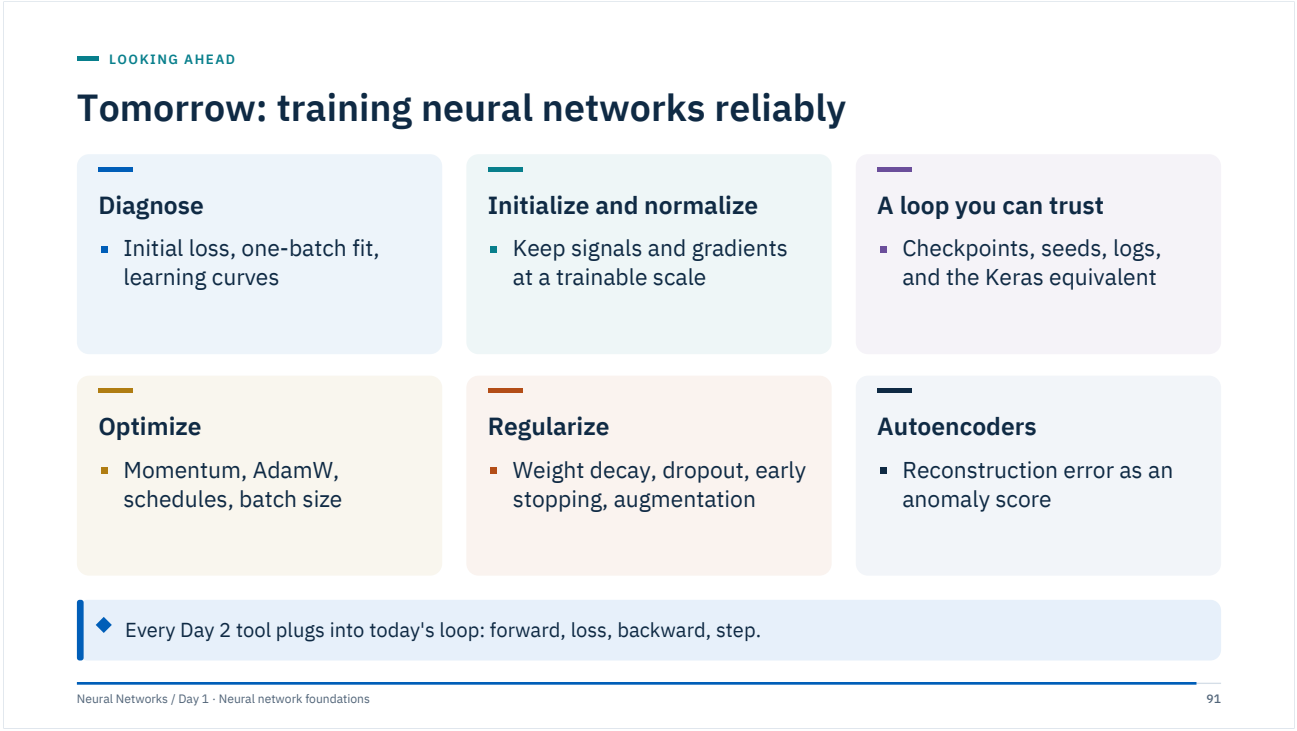
Key takeaways



[3 min] Read the takeaways aloud and link each to a moment in the day. Shapes: the broadcasting bug and the BCEWithLogitsLoss error. XOR: the proof exercise and the hidden-space figure. Output, target and loss: the matching exercise with the four energy tasks. Backpropagation: the computational graph, the autograd printout and the Lab 2 gradient check. Held-out data: the boundary-evolution curves and the Lab 1 protocol comparison. Baselines: the Sonar results, where the network was competitive but not clearly better.

Ask each participant to pick the takeaway they would explain to a colleague tomorrow morning, and to name the slide or lab step that convinced them.

Tomorrow: training neural networks reliably



[1 min] Preview Day 2, Training neural networks reliably, so learners can connect today's open questions to tomorrow's answers. The cards follow its five parts. The saturation of sigmoid derivatives and the stuck XOR seeds motivate diagnostics, initialization and normalization. The seed sensitivity motivates a training loop with checkpoints, fixed seeds and logs; Day 2's first lab rebuilds today's Sonar task as a reproducible PyTorch pipeline, and the same loop is shown in Keras. The slow NumPy training in Lab 2 motivates momentum, adaptive optimizers and learning-rate schedules. The creeping validation loss in the boundary-evolution figure motivates regularization and early stopping. The day closes with autoencoders, which turn reconstruction error into an anomaly score.

Suggest a short review before tomorrow: rerun the Lab 2 notebook top to bottom with the gradient check in place.

Exit check

PRACTICE

5 min · Individual

Exit check

1. Why do dense layers need nonlinear activations?
2. Why must the raw-logit loss and the output activation agree?
3. Which shape error can pass silently, and how do you catch it?
4. How can a gradient check reveal a silent implementation error?

[5 min] Collect answers on paper or in a quick poll; review two or three aloud.

1. Without nonlinear activations, any stack of affine layers collapses into a single affine map, so depth adds no expressive power; the circles example stayed at a straight line. 2. BCEWithLogitsLoss and CrossEntropyLoss apply the sigmoid or softmax internally; adding another activation in the model applies it twice, which distorts the probabilities and the gradients while the code still runs. 3. Predictions of shape (B, 1) against targets of shape (B,) broadcast to (B, B) in NumPy and in MSELoss; catch it by printing and asserting shapes and by reshaping targets explicitly. 4. Perturbing a parameter slightly and measuring the change in the loss gives an independent estimate of the gradient; if it disagrees with backpropagation, the backward code does not match the forward code.

Ask one learner to explain the difference between backpropagation (computing gradients) and optimization (using them to update parameters).

References and further study

FURTHER STUDY

References and further study

- Goodfellow, Bengio, Courville (2016). *Deep Learning*. MIT Press. Chapters 6 and 8.
- Rumelhart, Hinton, Williams (1986). Learning representations by back-propagating errors. *Nature* 323.
- Rosenblatt (1958). The perceptron. *Psychological Review* 65.
- Minsky, Papert (1969). *Perceptrons*. MIT Press.
- Cybenko (1989); Hornik (1991): universal approximation theorems.
- PyTorch tutorials: tensors, autograd, and the training loop. scikit-learn User Guide: neural network models.
- UCI Machine Learning Repository: Connectionist Bench (Sonar, Mines vs. Rocks).

Neural Networks / Day 1 · Neural network foundations 93

Goodfellow, Bengio and Courville's Deep Learning is freely readable online; chapter 6 covers feedforward networks and backpropagation and chapter 8 optimization. The historical papers are short and readable: Rumelhart, Hinton and Williams (Nature, 1986) for backpropagation and Rosenblatt (Psychological Review, 1958) for the perceptron. Minsky and Papert's Perceptrons (1969) analyses the limits of single-layer networks. Cybenko (Mathematics of Control, Signals and Systems, 1989) and Hornik (Neural Networks, 1991) state the approximation theorems.

For practice, the PyTorch "Learn the Basics" tutorials cover tensors, autograd and the training loop, and the scikit-learn user guide describes MLPClassifier and MLPRegressor. The UCI dataset page documents the Sonar data: consult its description before interpreting any result as a field classification capability. All arithmetic examples in this day are deliberately small and reproducible.

Reference: https://www.deeplearningbook.org/ ; https://docs.pytorch.org/tutorials/beginner/basics/intro.html ; https://scikit-learn.org/stable/modules/neural_networks_supervised.html ; https://archive.ics.uci.edu/dataset/151/connectionist+bench+sonar+mines+vs+rocks